



SAS[®] Viya[®] Platform: Deploying SAS[®] Viya[®] Monitoring for Kubernetes

1.2.47 - 1.2.53

This document might apply to additional versions of the software. Open this document in [SAS Help Center](#) and click on the version in the banner to see all available versions.

About SAS Viya Monitoring for Kubernetes	3
Overview	3
Considerations	4
Log Monitoring	5
Metric Monitoring	6
Getting Started	9
System Requirements	10
Kubernetes Cluster Support	10
Prerequisite Software	10
Minimum Resource Requirements	11
Storage Requirements for OpenSearch	14
Security	15
Pre-Deployment	16
Pre-Deployment Checklist	16
Security Recommendations	17
Create the Deployment Directory	18
Customizations	24
Red Hat OpenShift Considerations	62
Transition from Deploying SAS Viya Monitoring through the DaC Project	68

Deploy	72
Deploy SAS Viya Monitoring for Kubernetes	72
Deploy SAS Viya Monitoring for Kubernetes on Red Hat OpenShift	74
Deploy the External Alertmanager Configuration	77
Post-Deployment	77
Access Log-Monitoring and Metric-Monitoring Web Applications	77
Enable Access to Grafana Dashboards	78
Use OpenSearch Dashboards to Validate Logging	79
Configure SAS Environment Manager to Access Monitoring	79
Modify the Deployment	80
Migrate from Ingress NGINX to Contour	80
Change Administrator Passwords after Deployment	82
Control User Access to Log Monitoring	84
Configure Access to Web Applications via NodePorts	89
Configure Access to OpenSearch (OpenShift Only)	91
Limit Access in a Multi-Tenant Environment	92
Adjust the Retention Period for Log Messages	93
Increase Storage for Log Messages	96
Display Log Messages in Grafana Dashboards	100
Add More Grafana Dashboards	101
Additional Metric-Monitoring Customizations	104
Upgrade in Place	105
Remove the Monitoring Components	106
Renew Transport Layer Security (TLS) Certificates	107
Troubleshooting	110
Log Messages Require More Storage	110
Fluent Bit Log Messages Are Not Available	111
Logging Deployment Fails Due to OpenSearch REST Endpoint Not Being Accessible	112
Grafana Log In Fails after Updating the Grafana Admin Password	113
Metric-Monitoring Pods Are Not Running	114
The Kubernetes-Proxy Dashboard Does Not Display Any Data	116
SAS Launched Jobs Dashboards Do Not Work	116
Avoid Higher Resource Use (Istio Only)	117
Reference	117
Components for Deployment	117
Sample Files for Customization	119
Grafana Alerts	121
Helm Charts	125
Internal Logging Users	127
ServiceMonitors Provided for SAS Viya	128
Recording Rules Provided for SAS Workload Manager	129
OpenSearch Dashboards Provided by SAS	130
Grafana Dashboards Descriptions	131
Grafana Dashboards Location and File Names	139
Monitoring and Logging on Cloud-Provider Platforms	144
Choosing a Monitoring Strategy for an Azure Environment	146

About SAS Viya Monitoring for Kubernetes

Overview

The [SAS Viya Monitoring for Kubernetes](#) project is a monitoring solution for the SAS Viya platform that is built from highly regarded open-source observability tools. The project provides scripts and customization options to streamline the configuration and deployment process. The result is a comprehensive monitoring system that is set up quickly and easily.

SAS Viya Monitoring for Kubernetes includes the following monitoring capabilities:

Log messages

Log messages that are generated by SAS Viya applications and other applications that run on the Kubernetes cluster are collected, standardized, and stored for review by administrators.

Metrics

Performance and health metrics from applications are collected, stored, and made accessible to administrators.

Traces

Traces provide detailed information about the processing that occurs within an application and the interactions between components.

Note: Support for tracing within SAS Viya Monitoring for Kubernetes is experimental at this time. In addition, SAS Viya does not currently support exporting trace information.

SAS Viya Monitoring for Kubernetes uses the following open-sources tools:

- Fluent Bit, OpenSearch, and OpenSearch Dashboards for log-monitoring capabilities. See [“Log Monitoring” on page 5](#) for details.
- Prometheus and Grafana for metric-monitoring capabilities. See [“Metric Monitoring” on page 6](#) for details.

Additional open-source observability tools that provide other specialized capabilities are also included in SAS Viya Monitoring for Kubernetes.

SAS Viya Monitoring for Kubernetes also serves as a repository for additional content that can integrate with other Kubernetes monitoring tools. For example, it includes an alternative logging stack of Fluent Bit and Azure Monitor. Because the SAS Viya platform is designed and instrumented to be cloud-native and it is deployed to a Kubernetes cluster, any Kubernetes monitoring solution is compatible.

Note: This documentation uses “SAS Viya” to refer to the SAS Viya platform and compatible SAS Viya applications, including these products:

- SAS® Viya® (previously called SAS Visual Machine Learning)
 - SAS® Viya® Advanced (previously called SAS Visual Data Science)
 - SAS® Viya® Programming (previously called SAS Visual Data Science Programming)
 - SAS® Viya® Enterprise (previously called SAS Visual Data Science Decisioning)
-

Considerations

Be sure to understand the following considerations:

- The log-monitoring and metric-monitoring components can be deployed independently or together. There are no hard dependencies between the two.
- The experimental tracing feature requires functionality from both log-monitoring and metric-monitoring components. Therefore, you must deploy both sets of components in order to use tracing.
- There are no release dependencies between SAS Viya Monitoring for Kubernetes and SAS Viya, but you are encouraged to use the most recent release of SAS Viya Monitoring for Kubernetes.
- You should update your SAS Viya Monitoring for Kubernetes deployment at least every six months. This ensures that the latest bug fixes, security updates, and feature enhancements are deployed.
- Upgrading to a more recent version of Kubernetes does not require you to redeploy SAS Viya Monitoring for Kubernetes. However, you might want to perform an [“Upgrade in Place” on page 105](#) of SAS Viya Monitoring for Kubernetes that coincides with an upgrade of Kubernetes.
- SAS Viya Monitoring for Kubernetes deploys and integrates many open-source observability tools. However, administrators retain the ability and obligation to modify the configuration to meet their organization's unique situation and needs. Administrators are expected to familiarize themselves with the tools and how to use them. In some cases, they might need to use features of these tools that are not part of the out-of-the-box configuration of SAS Viya Monitoring for Kubernetes. Most of the components of the project are deployed through Helm Charts. These Helm charts provide the mechanism for administrators to modify a component's configuration options in order to comply with their organization's requirements. See [“Helm Charts” on page 125](#) for the list of Helm Charts that are used.
- The SAS Viya Monitoring for Kubernetes solution deploys open-source components. Your use of each component is governed by its applicable open-source license and you are responsible for understanding those license agreements.
- [SAS Viya Monitoring for Kubernetes](#) is available on GitHub under an Apache 2.0 license and there is no cost. It is supported in the following ways:
 - Customers who use SAS Viya 4 can open a support request with SAS Technical Support, and these requests are handled under the standard SAS support policies.
 - You can open issues and submit feature requests in the GitHub repository. GitHub issues are handled as time and priorities permit.

Log Monitoring

Introduction

SAS Viya 4 is a cloud-native Kubernetes application, and SAS Viya servers and services operate within Kubernetes pods. Because Kubernetes pods are transient, log messages cannot be written to static log files. Instead, log messages are streamed to the container's standard output locations (stdout or stderr). Kubernetes routes these log messages to temporary files on the Kubernetes node on which the pod is running. This makes them available to Kubernetes commands and to monitoring solutions. SAS Viya Monitoring for Kubernetes collects and aggregates these log messages so that administrators can review them in OpenSearch Dashboards.

Note: See [“Helm Charts Used by the Log-Monitoring Components” on page 125](#) for the list of Helm Charts that are used to deploy the components.

How Log Messages Are Collected

The log-monitoring capabilities of SAS Viya Monitoring for Kubernetes are built using Fluent Bit and OpenSearch open-source tools. OpenSearch Dashboards provide a web application that enables administrators to search for and review log messages that are generated in their Kubernetes cluster. Additional open-source observability tools that provide other specialized capabilities are also included in SAS Viya Monitoring for Kubernetes.

The following steps describe the log-message collection process:

- 1 Fluent Bit collects and distributes log messages that are streamed from all Kubernetes pods on the cluster. SAS provides configuration files so that Fluent Bit can locate messages from SAS Viya sources. As Fluent Bit collects the log messages, it performs some limited transformations and then passes them to OpenSearch. See the [Fluent Bit documentation](#) for more information about Fluent Bit.

IMPORTANT The log-monitoring infrastructure collects log messages from all pods in a Kubernetes cluster, not only the pods used for SAS Viya 4.

- 2 OpenSearch receives the collected messages from Fluent Bit, indexes the message contents, and directs the messages to the persistent store. SAS Viya Monitoring for Kubernetes is implemented with OpenSearch storage and search technology. For more information, see the [OpenSearch documentation](#).
- 3 OpenSearch Dashboards provides a graphical user interface, which enables users to search, filter, chart, and display the log messages that are stored in OpenSearch. See [“Performing Basic Tasks in OpenSearch Dashboards” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#) for more information about how to use OpenSearch Dashboards.

SAS provides a set of dashboards for SAS Viya, and you can create your own. For information about these dashboards, see [“OpenSearch Dashboards Provided by SAS” on page 130](#).

In addition, ElasticSearch Exporter collects performance and operational metrics from OpenSearch and makes them available to Prometheus. This enables administrators to monitor the health and performance of this key log-monitoring component. See the [Elasticsearch Exporter community on GitHub](#).

Note: See [“Log-Monitoring Components” on page 117](#) for a comprehensive list of the open source components that are integrated into the SAS Viya Monitoring for Kubernetes project.

Metric Monitoring

Introduction

The metric-monitoring capabilities use Prometheus and Grafana open-source projects. Administrators use the visualization and reporting capabilities of Grafana to explore the metric data. Additional open-source observability tools that provide more specialized capabilities are also included in the solution.

Metric monitoring collects metric data from the following sources:

- SAS Viya servers and services, SAS Cloud Analytic Services (CAS), and the stateful servers in your SAS Viya deployment
- nodes, services, pods, containers and other Kubernetes resources of the cluster where SAS Viya is deployed

There are two approaches to collecting metric data, a *push model* and a *pull model*. As a cloud-native application, SAS Viya 4 makes metrics available via a pull model. Monitoring applications, such as Prometheus, issue requests to SAS Viya resources at known locations and then process the collected data.

Push model

In a push model, resources load their metric data to a file or table at a specified interval. The monitoring application reads the metric data from that file or table.

SAS Viya 3.5 and earlier used the push model.

Pull model

In a pull model, the monitoring application queries the resources directly. The resource provides the metric data in response to the query.

Typically, SAS Viya 4 uses a pull model to collect metric data. Monitoring applications (such as Prometheus) issue requests to SAS Viya resources at known locations and then process the collected data.

Note: See [“Helm Charts Used by the Metric-Monitoring Components” on page 126](#) for the list of Helm Charts that are used to deploy the components.

Discovering Resources

The Prometheus Operator integrates Prometheus with Kubernetes so that Prometheus can discover resources and the endpoints for these resources, from which it collects metric data. The Prometheus Operator does the following tasks:

- defines PodMonitor and ServiceMonitor resources in order to create Prometheus jobs that discover, collect, and label metric data.
- updates the Prometheus configuration when PodMonitors and ServiceMonitors are created, updated, and deleted. The configuration modifications are in response to changes in the monitored resources.

See the [Prometheus API documentation](#) for more information about these resources.

SAS Viya Monitoring for Kubernetes provides pod monitor and service monitor definitions for SAS Viya resources. Because SAS Viya resources are created in Kubernetes, Prometheus uses the information from the pod monitors to automatically discover the resources and collect their metrics.

Note: The Prometheus Operator manages the Prometheus instance and the Alertmanager instance in the Kubernetes cluster.

See the [Prometheus Operator GitHub website](#) for information.

How SAS Viya Monitoring for Kubernetes Collects and Processes Metric Data

SAS Viya Monitoring for Kubernetes collects and processes metric data from SAS Viya and Kubernetes. The following steps describe the process:

- 1 Prometheus locates resources, collects and stores metrics, and generates alerts based on metrics. It accesses SAS Viya resources in the following ways:

Metric-data endpoints

Metric data for most SAS Viya resources is available through Prometheus HTTP and HTTPS metric endpoints. The endpoint for a resource provides metric data in a standard format that Prometheus can process. When Prometheus discovers the resource, it knows the location of the endpoint. For details about the metric-data format, see the [Prometheus Exposition Formats documentation](#).

Exporters

Exporters collect metric data and convert it to a Prometheus-endpoint format. Some exporters that are used include Crunchy Data, node exporters, and kube-state-exporters.

- Crunchy Data is an optional open-source Postgres provider for SAS Viya. Crunchy Data does not produce metric data in Prometheus-endpoint format. The SAS Viya metric-monitoring infrastructure includes an exporter for Crunchy Data.
- Node Exporter collects hardware and operating system metrics from Linux kernels and makes the metrics available to Prometheus. For more information, see the [Node Exporter GitHub website](#).

- Kube State Metrics collects metrics about the status of the deployment, its nodes, and its pods and makes the metrics available to Prometheus. For more information, see the [kube-state-metrics GitHub website](#).

Pushgateway

Some transient processes (such as SAS jobs) do not exist long enough for the metric data to be collected. Thus, a pull model fails to collect their metric data. Instead, these processes push their metrics to the Prometheus Pushgateway. Prometheus collects the data from the Pushgateway endpoint. For more information, see the [Prometheus Pushgateway GitHub website](#).

- 2 After Prometheus collects the metric data, Grafana queries the collected data in order to visualize the metric data. Grafana uses dashboards to display different types of metric data.
 - SAS provides a set of dashboards to visualize metrics from SAS Viya resources. For information about these dashboards, see [“Dashboards for Kubernetes Resources” on page 134](#).
 - The repository also provides a curated selection of dashboards to visualize metrics from Kubernetes resources.
 - Grafana provides standard dashboards to visualize Kubernetes metrics.
 - You can define your own dashboards.

See the [Grafana documentation](#) for more information.

Note: See [“Metric-Monitoring Components” on page 118](#) for a comprehensive list of the open source components that are integrated into the SAS Viya Monitoring for Kubernetes project.

Alerting

SAS Viya Monitoring for Kubernetes includes alerting capabilities through Alertmanager and Grafana. Both applications enable you to do the following tasks:

- Define alerts that are based on the same Prometheus metrics using Prometheus Query Language (PromQL).
- Manage these alerts, including silencing, inhibition, aggregation, and sending out notifications through email, on-call notifications systems, and chat platforms.

Alertmanager

A number of Kubernetes-focused alerts are defined out-of-the-box. These alerts are already grouped for ease of use, which categorizes alerts of similar nature into a single notification. For more information about configuring and using Alertmanager, see the [Alertmanager documentation](#).

Prometheus and Alertmanager web applications are disabled by default because they do not have a built-in user authentication mechanism. (They are not required for metric gathering or alert notification.) For more information about configuring access, see the following topics:

- [“Configure Access to Web Applications” on page 24](#)
- [“Configure Access to Prometheus and Alertmanager via NodePorts” on page 36](#)

Grafana Alerting

SAS Viya Monitoring for Kubernetes supports defining *alert rules*, *contact points*, and *notification policies* through YAML files as part of the pre-deployment process. For more information, see [“Customize Alerts Managed by Grafana” on page 56](#).

You can also use the Grafana **Alerting** page to interactively define these alerting configurations. For more information, see the following documentation:

- [Grafana Alerting documentation](#)
- [“Using Grafana Alerting” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#)

For a comparison of Alertmanager and Grafana, see the SAS Communities article [Comparing Alerting Capabilities of Alertmanager with Grafana](#).

How It Works

- 1 An alert instance fires based on the *alert rule*. The alert rule contains the query expression, the condition that must be met, and labels, such as: `'alertname="HighCPU"'` and `'severity="critical"'`.
- 2 The alert is routed based on matching labels in the *notification policies*. Grafana then evaluates the following:
 - which *contact point* the alert should go to
 - whether to delay (*group_wait*), batch (*group_by*), or suppress it (*repeat_interval*)
- 3 The alert is sent to the matching *contact point*, such as Slack or email.

Note: SAS Viya Monitoring for Kubernetes also supports sending alerts to external Alertmanagers. For more information, see [“Configure Alertmanager” on page 61](#).

Getting Started

The following steps outline the [SAS Viya Monitoring for Kubernetes](#) deployment and customization process for both the log-monitoring and the metric-monitoring components.

- 1 Verify that [“System Requirements”](#) are met.
- 2 [“Create the Deployment Directory”](#) for your SAS Viya Monitoring for Kubernetes solution.
- 3 Review the [“Customizations”](#) that are relevant to your deployment. The deployment scripts use customization files and some customizations require adding environment variables.

If you deploy on OpenShift, review the additional [“Red Hat OpenShift Considerations” on page 62](#).

TIP Use the [“Pre-Deployment Checklist”](#) for guidance.

- 4 “Deploy SAS Viya Monitoring for Kubernetes”.
- 5 Access and start using the log-monitoring and metric-monitoring applications. For information, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

System Requirements

Kubernetes Cluster Support

SAS Viya Monitoring for Kubernetes supports the same Kubernetes clusters as SAS Viya 4. For supported platforms, see [SAS Viya Platform Operations: Provisioning Resources](#).

For cluster requirements, see [“Virtual Infrastructure Requirements” in System Requirements for the SAS Viya Platform](#). Be sure to select your release of SAS Viya 4 in the Help Center banner.

Note: You must have Kubernetes cluster-wide access to any cluster in which you deploy SAS Viya Monitoring for Kubernetes software. You cannot deploy successfully if you have access only to a namespace or a subset of namespaces. This access also enables you to use the scripts provided by the SAS Viya Monitoring for Kubernetes solution.

For information about SAS Viya Monitoring for Kubernetes on cloud-provider platforms, see the following information:

- [“Monitoring and Logging on Cloud-Provider Platforms” on page 144](#).
- The GitHub repository contains a sample that shows how to deploy and customize the SAS Viya Monitoring for Kubernetes solution on Microsoft Azure. Other samples show how to use the native monitoring solutions to monitor SAS Viya. For descriptions of the samples and links to the sample directories, see [Sample Files on page 119](#).
- For a comparison between SAS Viya Monitoring for Kubernetes and Microsoft Azure Monitor, see [“Choosing a Monitoring Strategy for an Azure Environment” on page 146](#).

Prerequisite Software

You must have the following software with the correct versions:

- `kubectl` (a version that meets the prerequisites for SAS Viya 4 platform deployment) with Kubernetes cluster-wide access
- [Helm](#) version 3 or later

Note:

- See the [Helm version documentation](#) to determine the specific version to use with the version of Kubernetes running on your cluster.
 - For OpenShift deployments, version 3.5 or later is recommended.
 - For air-gap deployments, version 3.8 or later is required.
-
- [Git](#) version 1.8 or later
 - [Bash](#)
 - OpenSSL, cert-manager, or your organization can generate its own certificates
 - [yq utility](#) 4.45.1 version or later

Additional Prerequisites for Deployment on Red Hat OpenShift

The following additional prerequisites are required for deployments on Red Hat OpenShift:

- [OpenShift](#) (a version that meets the prerequisites for SAS Viya 4 platform deployment)
- [OpenShift oc command-line interface](#) version 4.0 or later

Minimum Resource Requirements

The components of SAS Viya Monitoring for Kubernetes have the following minimum resource requirements.

Considerations

- The CPU and memory minimums are implemented as Kubernetes resource requests (spec.resources.requests.cpu and spec.resources.requests.memory) for the pods.
- In some cases, CPU and memory maximums are implemented as Kubernetes resource limits (spec.resources.limits.cpu and spec.resources.limits.memory).
- The storage minimums are implemented as part of the corresponding PVC request.

Log-Monitoring Components

Elasticsearch Exporter

CPU:	100mcore
Memory:	128MiB

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Note: There might be another pod deployed with each instance of SAS Viya.

Event Router

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Fluent Bit

CPU:	100mcore
Memory:	128MiB
Storage:	Uses the host file system for buffering and to maintain state information.

One (1) pod per Kubernetes node (deployed via DaemonSet).

OpenSearch

CPU:	1
Memory:	100MiB (Note 1)
Storage:	1 (30GiB) PVC per OpenSearch node (Note 2)

Note:

- 1 Although the resource requests for the OpenSearch pods are set to 100MiB, the request setting is intended only to ensure that the pod gets scheduled on a node. When running, the OpenSearch pods typically require 6GiB of memory when configured with the default Java heap settings of 4GiB.
 - 2 30GiB is the default value. You can adjust to meet your needs.
-

Three (3) pods per deployment of SAS Viya Monitoring for Kubernetes.

Note: Do not co-locate with another instance of OpenSearch.

OpenSearch Dashboards

CPU:	requests (minimum): 100mcore	limits (maximum): 100mcore
Memory:	requests (minimum): 512MiB	limits (maximum): 512MiB

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Metric-Monitoring Components

Alertmanager

CPU:	requests: 50mcore
------	-------------------

Memory: 50MiB
Storage: 1 (10GiB) PVC

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Note: Do not co-locate with another instance of Alertmanager.

Grafana

CPU: requests: 350mcore
Memory: 350MiB
Storage: 1 (5GiB) PVC

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Kube State Metrics

CPU: requests: 25mcore
Memory: 50MiB

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Node Exporter

CPU: requests (minimum): 50mcore limits (maximum): 250mcore
Memory: requests (minimum): 50MiB limits (maximum): 100MiB

One (1) pod per Kubernetes node (deployed via DaemonSet).

Prometheus

CPU: requests: 1
Memory: 2GiB
Storage: 1 (25GiB) PVC

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Note: Do not co-locate with another instance of Prometheus.

Prometheus Operator

CPU: requests (minimum): 100mcore limits (maximum): 500mcore
Memory: requests (minimum): 50MiB limits (maximum): 2GiB

One (1) pod per deployment of SAS Viya Monitoring for Kubernetes.

Storage Requirements for OpenSearch

Deploying OpenSearch requires a StorageClass that provides persistent block storage or a local file system mount in order to store the search indices. Remote file systems, such as NFS, are not supported for this purpose. However, the default storage class in cloud platforms is typically adequate.

On-cloud or virtualization platforms

SAS recommends provisioning the required storage using features that are provided by the platform, such as vSphere volumes.

For physical machines or for virtualization platforms that lack dynamic provisioning

SAS recommends using local storage: persistent volumes with disks of sufficient size and a Kubernetes provisioner for local storage.

Table 1 *Storage Options per Platform*

Platform	StorageClass
Microsoft Azure	default ■ (Kubernetes 1.26 and later) disk.csi.azure.com ■ (Kubernetes 1.25 and earlier) kubernetes.io/azure-disk
Amazon Web Services	gp2 (kubernetes.io/aws-ebs)
Google Cloud Platform	standard (kubernetes.io/gce-pd)
Open source Kubernetes	Customer-provided storage class for local disk storage as appropriate for the target cloud platform or infrastructure
Red Hat OpenShift	thin (kubernetes.io/vsphere-volume)

By default, SAS Viya Monitoring for Kubernetes deploys three OpenSearch pods. Each pod requires a PVC with a default size of 30GiB.

For more information about StorageClass, see [“Customize StorageClass” on page 40](#).

Security

Overview

SAS Viya Monitoring for Kubernetes provides security in the following ways:

- [Transport Layer Security \(TLS\), digital certificates, and Kubernetes secrets on page 15](#)
- [Roles and role mappings \(RBACs\) for user accounts on page 84](#)
- The OpenSearch security plug-in is configured to provide a default security configuration. This configuration meets most customers' needs. See ["Customize Security Configuration Files" on page 53](#).
- [Permissions that control access to Grafana dashboards on page 78](#)

Note: The user interfaces of Prometheus and Alertmanager are not secure because they do not require user authentication. Therefore, SAS Viya Monitoring for Kubernetes does not enable these user interfaces by default. The Prometheus metric data and alerts are accessed through Grafana. Grafana provides secure access to the data because it includes user authentication.

Transport Layer Security (TLS): Digital Certificates and Kubernetes Secrets

Transport Layer Security (TLS) is used by the monitoring components and TLS requires the use of digital security certificates to secure communications. Secure communications within the SAS Viya Monitoring for Kubernetes solution are these two types:

- TLS for communications into the cluster

For communications into the cluster, the certificates enable your web browser to verify the identities of the OpenSearch, OpenSearch Dashboards, Prometheus, Grafana, and Alertmanager applications. For the Kubernetes ingress controller, you must supply your own certificates to enforce TLS. For more information, see ["Configure Access to Web Applications" on page 24](#).
- TLS for in-cluster communications

For in-cluster communications, the certificates enable identity verification when establishing communication:

 - ☐ between OpenSearch nodes
 - ☐ between OpenSearch and other log-monitoring components
 - ☐ between Prometheus and Grafana
 - ☐ between Prometheus (or Grafana) and other metric-monitoring components

When you deploy the SAS Viya Monitoring for Kubernetes solution the following steps occur:

- 1 The deployment process detects whether pre-existing certificates are present. You have the option to use your own certificates for in-cluster communications.
- 2 If no certificates are detected, the deployment process uses OpenSSL to generate the TLS certificates that it requires. You have the option to use cert manager to generate the certificates.

For information, see [“TLS for In-Cluster Communications” on page 37](#).

Note: TLS is enabled by default when deploying the SAS Viya Monitoring for Kubernetes solution.

Certificates are stored as Kubernetes secrets for subsequent use. Kubernetes mounts these secrets onto the various log-monitoring and metric-monitoring components' pods where they appear as files on the disk. The name and location of these files is fixed by the Helm charts and cannot be changed.

Pre-Deployment

Pre-Deployment Checklist

Before you deploy SAS Viya Monitoring for Kubernetes, review this checklist to ensure that you have reviewed the applicable information for your deployment.

- ☐ All deployments should review these topics:
 - ☐ [“System Requirements” on page 10](#)
 - ☐ [“Security Recommendations” on page 17](#)
 - ☐ [“Create the Deployment Directory” on page 18](#)
- ☐ Are you deploying on Microsoft Azure, Amazon EKS, or Google GKE?
See [“Monitoring and Logging on Cloud-Provider Platforms” on page 144](#).
- ☐ Are you deploying on Red Hat OpenShift?
 - ☐ [“Deploy SAS Viya Monitoring for Kubernetes on Red Hat OpenShift” on page 74](#)
 - ☐ [“Customize Metric-Monitoring” on page 64](#)
 - ☐ [“OpenShift Authentication for Grafana” on page 64](#)
 - ☐ [“Customize Routes on OpenShift” on page 65](#)
 - ☐ [“Enable Direct Access to OpenSearch on OpenShift \(Optional\)” on page 67](#)
- ☐ Are you deploying into an air-gapped environment?
See [“Prepare to Deploy SAS Viya Monitoring for Kubernetes in an Air Gap Environment” on page 43](#).

Note: These environments might also be called *dark sites*.

- ☐ Have you chosen your security configurations and determined how users access each application?
 - ☐ [“Configure Access to Web Applications” on page 24](#)
 - ☐ [“TLS for In-Cluster Communications” on page 37](#)
 - ☐ [“Control User Access to Log Monitoring” on page 84](#)
 - ☐ [“Set Administrator User Passwords” on page 55](#)
 - ☐ [Customize Security Configuration Files for OpenSearch on page 53](#)
 - ☐ [“Enable Access to Grafana Dashboards” on page 78](#)
- ☐ Have you assessed your storage needs?
 - ☐ [“Customize StorageClass” on page 40](#)
 - ☐ [“Increase Storage for Log Messages” on page 96](#)
 - ☐ [“Adjust the Retention Period for Log Messages” on page 93](#)
- ☐ Does your organization require that you use a workload-node-placement strategy?
See [“Customize Workload-Node Placement” on page 42](#).
- ☐ Will you use Grafana alerting capabilities?
See [“Customize Alerts Managed by Grafana” on page 56](#).
- ☐ Does your environment already include an instance of Alertmanager?
See [“Configure Alertmanager” on page 61](#).
- ☐ Will your deployment scripts run in a high-security environment on a system whose network traffic is routed through a proxy server?
See [“Prevent Routing Network Traffic through a Proxy Server” on page 55](#).

Security Recommendations

SAS strongly encourages the following security best practices for your SAS Viya Monitoring for Kubernetes deployment:

- Use a source-control management system for your directory that includes customization files. This enforces version control and increased security through access control.
- Limit access to customization and deployment files to a select group, such as deployment engineers.
- Do not use plaintext passwords in configuration files. Instead, encrypt these passwords, or use a password management tool for a more secure method.
- If a jump host was used for the deployment, remove all deployment content and artifacts from the machine after a successful deployment.

- Limit access to the logging and monitoring namespaces.

Create the Deployment Directory

Overview of Directory Structure

Create a directory for your [SAS Viya Monitoring for Kubernetes](#) deployment that is similar to the following:

```
/local_directory/
├─ viya4-monitoring-kubernetes
├─ my-viya4mon-user-directory/
│   ├─ user.env
│   ├─ logging
│   │   └─ customization files
│   │   └─ user.env
│   └─ monitoring
│       └─ customization files
│       └─ user.env
```

Where:

- `/local_directory/viya4-monitoring-kubernetes` contains a local copy of the [SAS Viya Monitoring for Kubernetes](#) repository. See [“Copy the Repository” on page 18](#).
- `/local_directory/my-viya4mon-user-directory` is the directory that contains your customization files. You must create the `$USER_DIR` environment variable to reference this directory because the deployment scripts use the `$USER_DIR` environment variable. See [“Set the \\$USER_DIR Environment Variable” on page 20](#).

Note: The `$USER_DIR` directory is not a subdirectory of the SAS Viya Monitoring for Kubernetes repository. It is outside of the repository.

- customization files are yaml files that include your customization configurations. The project provides samples that you can copy to the `$USER_DIR` directory as a starting point. See [“Copy Customization Sample Files” on page 20](#).
- The `user.env` files contain environment variables that are necessary for your customizations. See [“Customize Variables and Parameters” on page 22](#).

IMPORTANT SAS recommends that you use a source-control management system for your directory that includes customization files. This enforces version control and increases security through access control.

Copy the Repository

You can use either of the following methods to create a local copy of the repository:

- Clone the repository to a new local directory.
- Download a compressed copy of the repository and expand the copy in a new local directory.

Clone the Repository

Complete these steps:

- 1 Create a local directory into which you want to clone the repository.
- 2 Change to the directory where you want to create the local copy, and enter the following command:

```
git clone https://github.com/sassoftware/viya4-monitoring-kubernetes.git
```

- 3 Change to the directory **viya4-monitoring-kubernetes**.
- 4 Enter the command:

```
git checkout stable
```

Download a Compressed Copy of the Repository

Complete these steps:

- 1 Go to the [SAS Viya Monitoring for Kubernetes](#) repository on GitHub.
- 2 Confirm the release number of the stable branch.
 - a Click the **main** list and select **stable**.
 - b View **CHANGELOG.md**. The release number is displayed at the top of the file.

Note: Typically, the release number associated with the stable branch is the latest release available.

- 3 Return to the main page of the repository. On the right side of the page, click **Releases**. The [Releases page](#) appears.
- 4 Locate the release number that you want to deploy.
- 5 Below the release notes, expand **Assets** if it is not already expanded.
- 6 Select either **Source code (zip)** or **Source code (tar.gz)** to download a compressed file of the repository.
- 7 Decompress the downloaded file to create a local copy of the repository. The repository is created in a directory named **viya4-monitoring-kubernetes-release_number** where *release_number* is the release that you selected.

Create the Directory for Your Customization Files

The deployment scripts use customization files. Create a directory that is outside of the local repository that contains these customization files. Maintaining these customization files in their own directory that is outside of the local repository provides the following advantages:

- The customization files can be in a location that is under version control.
- The customization files are preserved when you deploy new versions of the log- and metric-monitoring components.
- Different sets of customization files can be created for different cluster types. You can specify which set of customization files to use depending on the type of cluster to which you are deploying.

Use the following command to create the directory:

```
mkdir local_directory/my-viya4mon-user-dir
```

Where *my-viya4mon-user-dir* is the directory of your customization files.

Note: The directory for your customization files might need to be in a shared location if your organization has multiple administrators that maintain the [SAS Viya Monitoring for Kubernetes](#) solution.

Set the \$USER_DIR Environment Variable

The \$USER_DIR environment variable identifies the directory in which you put various files to customize the deployment. The deployment scripts refer to the \$USER_DIR directory and its subdirectories as they execute to access these files. Enter the following command to create the \$USER_DIR environment variable:

```
export USER_DIR=/local_directory/my-viya4mon-user-dir
```

Copy Customization Sample Files

About the Sample Files

Samples are provided to demonstrate how to customize the deployment of the log- and metric-monitoring components for several common deployment scenarios as well as a generic sample. Each sample provides instructions and example `yaml` files that you modify to fit your environment. Although each example focuses on a specific scenario, you can combine multiple samples by merging the applicable values in each deployment file.

When evaluating the samples, consider these key points:

- If your situation matches one of the specialized samples, copy the customization files for the sample that most closely matches your environment from the repository to your customization file directory. Using an existing configuration file enables you to start your customization with a

set of values that are valid for your situation. You can then make further modifications to the files.

- If your situation does not match any of the specialized samples, copy the [generic base sample](#) as a basis for your customization files, and then change the values or copy values from other samples to match your environment.
- If more than one sample applies to your environment, you can manually copy the values from the other sample files to the files in your customization directory.

Note: A sample file provides only the `user.env` files that are necessary for the sample scenario. You might need to create `user.env` files for environment variables that pertain to other customizations. For the `user.env` files that might be needed, see [“About Environment Variables” on page 23](#).

The following samples are available:

CAUTION

Sample files show plain text passwords as examples. For your production environment, SAS recommends that you set these passwords with a more secure mechanism.

[Azure Deployment of Log-Monitoring and Metric-Monitoring Components](#)

shows the customizations that are necessary if you deploy on Microsoft Azure Kubernetes Service (AKS). The sample assumes that NGINX Ingress is used, but you can modify the sample for other solutions.

[External Alertmanager](#)

shows how to configure monitoring to use an external instance of Alertmanager.

In a typical deployment, each cluster uses a separate instance of Alertmanager. If you have multiple clusters, you might want alerts from all clusters to come from a single instance of Alertmanager. You might also have an existing instance of Alertmanager and want alerts to come from the existing instance.

[Generic Base for Log-Monitoring and Metric-Monitoring Components](#)

provides a template `$USER_DIR` containing a full set of customization files with explanatory comments.

[Grafana Alerts](#)

includes the customizations for adding Grafana alert rules for monitoring SAS Viya environments. The alert `YAML` files are organized into subdirectories by component or category.

Ingress Configuration

The following samples are available for configuring access to the web applications that are deployed as part of the SAS Viya Monitoring for Kubernetes solution:

- [Contour Configuration](#)

demonstrates how to configure Contour as the ingress controller for accessing the web applications that are deployed. Options for either host-based and path-based ingress are included.

- [Ingress NGINX Configuration](#)

demonstrates how to configure Ingress NGINX as the ingress controller for accessing the web applications. Options for either host-based and path-based ingress are included.

Note: You can choose to automate this process. For information, see [“Configure Access to Web Applications” on page 24](#).

Note: In November, 2025, the Kubernetes community announced that it would retire Ingress NGINX. For information, see [Ingress NGINX Retirement: What You Need to Know](#). With the project’s imminent retirement, Contour is now recommended. However, SAS Viya Monitoring for Kubernetes will continue to support the use of Ingress NGINX for the foreseeable future. For SAS Viya’s policy, see [“Overview” in SAS Viya Platform: Changing Ingress Controllers in a Deployment](#).

Note: Migrating from Ingress NGINX to Contour is accomplished by redeploying SAS Viya Monitoring for Kubernetes after you make adjustments for Contour. If you used the existing ingress auto-generation capabilities and your `$USER_DIR` is in place, the changes that you need to make are to set `INGRESS_TYPE` to `contour` and to set `INGRESS_USE_SEPARATE_CERTS` to `false`. If you manually configured review the Contour sample and the Ingress NGINX sample to understand what needs to be modified.

Namespace Monitoring

demonstrates how to customize a monitoring deployment to separate cluster monitoring from SAS Viya (namespace) monitoring.

SAS SpeedyStore Monitoring

demonstrates how to extend SAS Viya Monitoring for Kubernetes to monitor the SingleStore instance that is embedded within SAS SpeedyStore. This allows administrators to monitor their SingleStore cluster using the same Grafana instance that they use to monitor the rest of their SAS Viya deployment.

For more information on logging and monitoring your SAS Viya Speedy Store deployment, see [“Logging and Monitoring” in SAS SpeedyStore: Administration and Configuration Guide](#).

Copy Sample Files

Enter the following command from the top-level directory of your copy of the repository:

```
cp samples/sample_subdirectory/* $USER_DIR -r
```

Where:

- `sample_subdirectory` is the subdirectory of your selected sample.
 - `$USER_DIR` is the environment variable that contains the directory path to your customization files. See [“Set the \\$USER_DIR Environment Variable” on page 20](#).
-

Note: To enable the sample alerts you must copy the samples to `$USER_DIR/monitoring/alerting/` directory.

Customize Variables and Parameters

After you have your local directory that contains your sample files, you can customize the components that you deploy by specifying environment variables and Helm chart parameters.

About Environment Variables

Environment variables control script behavior and high-level options such as TLS and workload-node placement. Although you can specify environment variables on a command line, it is recommended that you specify the variables in the `user.env` file.

Typically, a copy of the `user.env` file is in the following locations:

- `$USER_DIR/user.env`
- `$USER_DIR/logging/user.env`
- `$USER_DIR/monitoring/user.env`

where `$USER_DIR` is the environment variable that contains the directory of your customization files. See [“Set the \\$USER_DIR Environment Variable” on page 20](#).

How an environment variable is used depends on which file contains the variable:

- Log-monitoring scripts typically load environment variables from the top-level `user.env` file and the `logging/user.env` file.
- Metric-monitoring scripts typically load environment variables from the top-level `user.env` file and the `monitoring/user.env` file.
- Typically, if you want a value available to both log- and metric-monitoring, put that variable in the top-level `user.env` file.
- If the variable is specific to log monitoring or metric monitoring, put that variable in the corresponding `user.env` file.
- Avoid putting the same environment variable in multiple `user.env` files. This might lead to unexpected results.

Note: In `user.env` files, any line where the first character is “#” is treated as a comment and ignored.

Note: A `user.env` file is necessary for only the environment variables for your customizations. You do not need to create the file if you are not including environment variables. For example, if you are not deploying the log-monitoring components, you do not need the `$USER_DIR/logging/user.env` file.

About Helm Chart Parameter Values

Note: For a list of the Helm charts used, see [“Helm Charts” on page 125](#).

Note: In the following lists, the `$USER_DIR` environment variable contains the location of your `my-viya4mon-user-dir` directory.

To customize Helm chart parameter values for log-monitoring components, use the following YAML customization files:

- For OpenSearch: `$USER_DIR/logging/user-values-opensearch.yaml`

- For OpenSearch Dashboards: `$USER_DIR/logging/user-values-osd.yaml`
- For Elasticsearch Exporter: `$USER_DIR/logging/user-values-es-exporter.yaml`
- For Fluent Bit: `$USER_DIR/logging/user-values-fluent-bit-open.yaml`

To customize Helm chart parameter values for metric-monitoring components, use the following YAML customization files:

- For Grafana and metric-monitoring components:
`$USER_DIR/monitoring/user-values-prom-operator.yaml`
- For the Prometheus Pushgateway component deployed into the SAS Viya namespace:
`$USER_DIR/monitoring/user-values-push-gateway.yaml`

TIP

- Avoid keeping other files in these directories.
- These directories might contain additional YAML files.
- You do not have to specify values in all of these YAML files.
- In order to minimize the potential for errors, you should not manually create the customization files. Use one of the sets of sample files as the starting point for your own customizations. See [“Copy Customization Sample Files” on page 20](#).

IMPORTANT If you edit a YAML file, ensure that the parent item of any item that you uncomment is also uncommented. For example, in the `user-values-opensearch.yaml` file, if you uncommented the `storageClass` item for the OpenSearch primary nodes, you must also uncomment the `persistence` item, as shown below:

```
# Sample user-values-opensearch.yaml

persistence:  # Uncommented because 'storageClass' is uncommented.
  storageClass: alt-storage # Uncommented to direct OpenSearch to an alternate storageClass
```

Customizations

Follow the instructions in this chapter to update the customization files to specify the customization variables and parameters.

Configure Access to Web Applications

Introduction

The SAS Viya Monitoring for Kubernetes project includes the following web applications:

- OpenSearch Dashboards: to view collected log messages
- Grafana: to visualize the collected metrics and traces

In addition, you can choose to make the following components accessible:

- Prometheus web application
- Alertmanager web application
- OpenSearch API endpoint

These web applications can be made available through the following mechanisms:

Ingress Configuration

The following ingress controllers are available for configuring access to the web applications that are deployed as part of the SAS Viya Monitoring for Kubernetes solution:

- [Contour](#) ingress controller

Contour is the preferred method for production environments.

- Ingress NGINX ingress controller

Due to the Ingress NGINX project's imminent retirement, Contour is now recommended. However, SAS Viya Monitoring for Kubernetes will continue to support the use of Ingress NGINX for the foreseeable future.

Note: In November 2025, the Kubernetes community announced that it would retire Ingress NGINX. For information, see [Ingress NGINX Retirement: What You Need to Know](#).

NodePorts

NodePorts are typically used in test environments and other non-production uses. TLS is configured by default. For instructions on setting up NodePorts, see the following topics:

- [“Configure Access to OpenSearch Dashboards via NodePorts” on page 36](#)
- [“Configure Access to Prometheus and Alertmanager via NodePorts” on page 36](#)

Note: Access to Grafana via NodePorts is enabled unless Ingress is used.

Note: In addition to the above methods to access web applications, you can use Kubernetes port-forwarding capabilities. These are useful only for quick tests.

Choose one of the following methods to generate ingress definitions and Kubernetes Secrets that are needed for TLS in the deployment process:

- Automatically generate Ingress definitions and the Kubernetes Secrets that are needed for TLS through environment variables. For instructions, see [“Automatically Generate Ingress Definitions and Kubernetes Secrets” on page 26](#).
- Use the [Contour](#) sample or the [Ingress NGINX](#) sample for guidance on how to manually generate Ingress definitions. Secondly, create a set of Kubernetes secrets to hold the TLS digital certificates. For instructions, see [“Manually Configure Ingress Definitions and Enable TLS” on page 33](#).

Note: Migrating from Ingress NGINX to Contour is accomplished by redeploying SAS Viya Monitoring for Kubernetes after you make adjustments for Contour. If you used the existing ingress autogeneration capabilities and your `$USER_DIR` is in place, the changes that you need to make are to set `INGRESS_TYPE` to `contour` and to set `INGRESS_USE_SEPARATE_CERTS` to `false`. If you manually configured ingress, review the Contour sample and Ingress NGINX sample to understand what needs to be modified.

Automatically Generate Ingress Definitions and Kubernetes Secrets

Review the following notes before you create your environment variables:

- You must have the `yq` utility available on the host where the deployment scripts are run.
- This automation eliminates the need to manually define these common customizations in the appropriate `yaml` file within the `$USER_DIR` directory.
- The deployment automation leverages the [Contour](#) or [Ingress NGINX](#) samples by combining them with the appropriate host names and other information that is provided below to create the corresponding Contour or Ingress NGINX resources. The automation supports only the specific ingress configuration options that are included in the samples.
- Make sure the values are correct. Incorrect or mismatched information results in the web application being unreachable.

Use the following steps to provide this information to the appropriate Helm chart prior to the deployment process:

- 1 Make sure you have set up your `$USER_DIR` directory that contains your `yaml` files and `user.env` files. See [“Create the Deployment Directory” on page 18](#) for more information.
- 2 To generate ingress definitions for OpenSearch Dashboards and Grafana, add the following environment variables to the `$USER_DIR/user.env` file:

```
AUTOGENERATE_INGRESS=true
BASE_DOMAIN=host_domain_for_application_URLs
ROUTING=host OR path
```

If you are using **Contour**, set the `INGRESS_TYPE` environment variable to `contour`:

```
INGRESS_TYPE=contour
```

If you are using **Ingress NGINX**, set the `INGRESS_TYPE` environment variable to `ingress-nginx`:

```
INGRESS_TYPE=ingress-nginx
```

- 3 Add the following environment variables to provide the location of the TLS certificate file and key file:

```
INGRESS_CERT=TLS_certificate_location
INGRESS_KEY=key_location
```

When you provide `INGRESS_CERT` and `INGRESS_KEY` environment variables, the certificate/key pair is used to create one Kubernetes Secret for all of the project's web applications, or the certificate/key pair is used to create a separate Kubernetes Secret for each web application. The `USE_SEPARATE_CERTS` environment variable determines which approach is used.

- When the `USE_SEPARATE_CERTS` environment variable is set to `false`, one Kubernetes Secret that is called `v4m-ingress-tls-secret` is created and used to protect all of the web applications.

This is the default when `INGRESS_TYPE` is set to `contour`.

- When the `USE_SEPARATE_CERTS` environment variable is set to `true`, distinct Kubernetes secrets are created for each web application. For example, the Kubernetes secret `grafana-ingress-tls-secret` is created and used when accessing Grafana, and the Kubernetes secret `kibana-ingress-tls-secret` is created and used when accessing OpenSearch Dashboards.

This is the default when `INGRESS_TYPE` is set to `ingress-nginx`.

Decide on your preferred approach and add the `USE_SEPARATE_CERTS` environment variable and set it to `false` or `true`:

```
USE_SEPARATE_CERTS=false/true
```

Note: You can manually create these secrets after the deployment process completes. For information, see [“Manually Configure Ingress Definitions and Enable TLS” on page 33](#).

- 4 Use this step **ONLY** if you are configuring path-based routing with Contour and you want to modify the following default behavior:

When configuring path-based routing with Contour, an additional HTTPProxy resource, referred to as a *root* HTTPProxy, is created in the same namespace as the HTTPProxy resources that are created for each accessible web application. This *root* HTTPProxy resource prefixes the specified `BASE_DOMAIN` with either `logging` (for OpenSearch and OpenSearch Dashboards) or `monitoring` (for Grafana, Prometheus and Alertmanager). The generated URLs to access Grafana and OpenSearch Dashboards are similar to the following:

- `https://monitoring.myserver.example.com/grafana`
- `https://logging.myserver.example.com/dashboards`

This approach provides a workable solution that minimizes potential conflicts with other HTTPProxy resources that might be deployed.

However, you might want to modify this behavior to work within your organization's policies. Here are some examples of why you might want to modify the behavior:

- To restrict the creation of *root* HTTPProxy resources to certain users or namespaces.
- To have more control over the URL that is needed to reach the web applications, such as eliminating the `logging` or `monitoring` prefix.

You can disable the creation of the *root* HTTPProxy resources by setting the environment variable `INGRESS_CREATE_ROOT_PROXY` to `false`:

```
INGRESS_CREATE_ROOT_PROXY=false
```

You can then create or update an existing *root* HTTPProxy to make the SAS Viya Monitoring for Kubernetes web applications accessible from your preferred URL.

Note: When using path-based routing with Contour, there must be one *root* HTTPProxy that references the HTTPProxy resource for a web application. If there is not, the web application is not accessible to users.

Note: The `INGRESS_CREATE_ROOT_PROXY` environment variable is only relevant when configuring ingress with Contour.

Optional: Provide a fully qualified domain name (FQDN) for any of the web applications.

You can overwrite the `BASE_DOMAIN` for any of the applications by setting the appropriate FQDN environment variable. However, you must still provide a `BASE_DOMAIN` even if you plan to provide only an FQDN for OpenSearch Dashboards and Grafana.

- 1 Add either of the following environment variables to the `$USER_DIR/user.env` file:

```
GRAFANA_FQDN=fully_qualified_hostname
```

```
OSD_FQDN=fully_qualified_hostname
```

The following table shows examples of the URL variations for Grafana, based on which environment variables you use.

Table 2 Example Grafana URL Variations

ROUTING	BASE_DOMAIN	GRAFANA_FQDN	GRAFANA_PATH	Grafana URL
host	myserver.example.com			https://grafana.myserver.example.com
path	myserver.example.com			https://myserver.example.com/grafana
host	myserver.example.com		k8smetrics	https://k8smetrics.myserver.example.com
path	myserver.example.com		k8smetrics	https://myserver.example.com/k8smetrics
host	myserver.example.com	viyaprod.example.com		https://viyaprod.example.com
path	myserver.example.com	viyaprod.example.com	grafana	https://viyaprod.example.com/grafana

For a list of environment variables and their default values that are used for generating ingress definitions, see [“Reference: Environment Variables Used for Ingress Definitions”](#) on page 29.

Optional: Enable ingress access for Alertmanager, Prometheus, or OpenSearch API endpoint.

Access to these applications is disabled by default, even if `AUTOGENERATE_INGRESS` is set to `true`.

Note: The developers of this project do not recommend making Prometheus and Alertmanager accessible. The Prometheus and Alertmanager applications do not include native authentication mechanisms by default. Exposing such an application without other restrictions in place is unsecure. In addition, the developers do not recommend making the OpenSearch API endpoint accessible. Although the OpenSearch API endpoint does require authentication, there are limited use cases where access to it is required.

To enable access to any of these applications and define ingress resources, you must set the appropriate environment variable.

1 Add the following environment variables to the `$USER_DIR/user.env` file:

- For Alertmanager:

```
ALERTMANAGER_INGRESS_ENABLE=true
```

- For Prometheus:

```
PROMETHEUS_INGRESS_ENABLE=true
```

- For OpenSearch:

```
OPENSEARCH_INGRESS_ENABLE=true
```

2 The `INGRESS_CERT` and `INGRESS_KEY` environment variables create secrets for every web application. You can optionally specify certificates for these applications. Use the following environment variables if you choose to do so:

```
ALERTMANAGER_INGRESS_CERT=TLS_certificate_location
ALERTMANAGER_INGRESS_KEY=key_location
PROMETHEUS_INGRESS_CERT=TLS_certificate_location
PROMETHEUS_INGRESS_KEY=key_location
OPENSEARCH_INGRESS_CERT=TLS_certificate_location
OPENSEARCH_INGRESS_KEY=key_location
```

Reference: Environment Variables Used for Ingress Definitions

The following table provides details about the environment variables that are used for configuring Ingress access to web applications and for TLS certificate and key files.

Table 3 Environment Variables for Ingress Definitions

Environment Variable	Default Value	Notes
<code>AUTOGENERATE_INGRESS</code>	<code>false</code>	Ingress definitions are auto-generated when this value is set to <code>true</code> .
<code>INGRESS_TYPE</code>	<code>ingress-nginx</code>	Note: You can configure <i>Contour HTTPProxy</i> resources for the ingress controller. Set this value to <code>contour</code> .

Environment Variable	Default Value	Notes
		Note: In November, 2025, the Kubernetes community announced that it would retire Ingress NGINX. For information, see Ingress NGINX Retirement: What You Need to Know. With the project's imminent retirement, Contour is now recommended. However, SAS Viya Monitoring for Kubernetes will continue to support the use of Ingress NGINX for the foreseeable future. SAS recommends that you change the ingress controller in existing SAS Viya platform deployments to Contour ingress. For information, see "Overview" in SAS Viya Platform: Changing Ingress Controllers in a Deployment.
BASE_DOMAIN	<i>none</i>	REQUIRED The host domain that is used for constructing URLs to web applications on this cluster. <code>BASE_DOMAIN</code> is required even if you provide FQDN for the applications.
ROUTING	host	The following values are used to indicate whether host-based or path-based Ingress should be configured: ■ host ■ path Any other value results in an error.
INGRESS_CERT	<i>none</i>	File containing TLS certificate to use for ingress of SAS Viya Monitoring web applications
INGRESS_KEY	<i>none</i>	File containing TLS key to use for ingress of SAS Viya Monitoring web applications
USE_SEPARATE_CERTS	When <code>INGRESS_TYPE</code> is <code>ingress-nginx</code> , the default value is <code>true</code> .	When <code>true</code> , a separate Kubernetes Secret containing ingress TLS certificates is used for each web application.

Environment Variable	Default Value	Notes
	When <code>INGRESS_TYPE</code> is <code>contour</code> , the default value is <code>false</code> .	When <code>false</code> , a single Kubernetes Secret is used for every web application.
<code>INGRESS_CREATE_ROOT_PROXY</code>	<code>true</code>	This environment variable is only applicable when configuring path-based routing with Contour ingress. When <code>true</code> , a root HTTPProxy resource is created in the logging or monitoring namespace in addition to the application-specific HTTPProxy resources. See this step on page 27 for more information.
<code>GRAFANA_INGRESS_ENABLE</code>	<code>true</code>	<code>true/false</code> Enable Ingress for Grafana
<code>OSD_INGRESS_ENABLE</code>	<code>true</code>	<code>true/false</code> Enable Ingress for OpenSearch Dashboards
<code>ALERTMANAGER_INGRESS_ENABLE</code>	<code>false</code>	<code>true/false</code> Enable Ingress for Alertmanager
<code>OPENSEARCH_INGRESS_ENABLE</code>	<code>false</code>	<code>true/false</code> Enable ingress for OpenSearch
<code>PROMETHEUS_INGRESS_ENABLE</code>	<code>false</code>	<code>true/false</code> Enable ingress for Prometheus
<code>GRAFANA_FQDN</code>	for host-based routing: <code>\$GRAFANA_PATH.\$BASE_DOMAIN</code> for path-based routing: <code>BASE_DOMAIN</code>	Fully Qualified Domain Name to access Grafana
<code>GRAFANA_PATH</code>	<code>grafana</code>	host/path segment used to construct FQDN and Paths
<code>OSD_FQDN</code>	for host-based routing: <code>\$OSD_PATH.\$BASE_DOMAIN</code> for path-based routing: <code>BASE_DOMAIN</code>	Fully Qualified Domain Name to access OpenSearch Dashboards
<code>OSD_PATH</code>	<code>dashboards</code>	host/path segment used to construct FQDN and Paths

Environment Variable	Default Value	Notes
ALERTMANAGER_FQDN	for host-based routing: \$ALERTMANAGER_PATH.\$BASE_DOMAIN for path-based routing: BASE_DOMAIN	Fully Qualified Domain Name to access Alertmanager
ALERTMANAGER_PATH	alertmanager	host/path segment used to construct FQDN and Paths
PROMETHEUS_FQDN	for host-based routing: \$PROMETHEUS_PATH.\$BASE_DOMAIN for path-based routing: BASE_DOMAIN	Fully Qualified Domain Name to access Prometheus
PROMETHEUS_PATH	prometheus	host/path segment used to construct FQDN and Paths
OPENSEARCH_FQDN	for host-based routing: \$OPENSEARCH_PATH.\$BASE_DOMAIN for path-based routing: BASE_DOMAIN	Fully Qualified Domain Name to access OpenSearch
OPENSEARCH_PATH	search	host/path segment used to construct FQDN and Paths
GRAFANA_INGRESS_CERT	\$INGRESS_CERT	File containing TLS certificate to use for Grafana Ingress
GRAFANA_INGRESS_KEY	\$INGRESS_KEY	File containing TLS key to use for Grafana Ingress
OSD_INGRESS_CERT	\$INGRESS_CERT	File containing TLS certificate to use for OpenSearch Dashboards Ingress
OSD_INGRESS_KEY	\$INGRESS_KEY	File containing TLS key to use for OpenSearch Dashboards Ingress
ALERTMANAGER_INGRESS_CERT	\$INGRESS_CERT	File containing TLS certificate to use for Alertmanager Ingress
ALERTMANAGER_INGRESS_KEY	\$INGRESS_KEY	File containing TLS key to use for Alertmanager Ingress

Environment Variable	Default Value	Notes
PROMETHEUS_INGRESS_CERT	\$INGRESS_CERT	File containing TLS certificate to use for Prometheus Ingress
PROMETHEUS_INGRESS_KEY	\$INGRESS_KEY	File containing TLS key to use for Prometheus Ingress
OPENSEARCH_INGRESS_CERT	\$INGRESS_CERT	File containing TLS certificate to use for OpenSearch Ingress
OPENSEARCH_INGRESS_KEY	\$INGRESS_KEY	File containing TLS key to use for OpenSearch Ingress

Manually Configure Ingress Definitions and Enable TLS

You can manually generate either host-based or path-based Kubernetes ingress definitions that are TLS-enabled for accessing SAS Viya Monitoring for Kubernetes web applications. You have the option of using Contour or Ingress NGINX as the ingress controller. Regardless of the ingress controller that you use, you must manually create a set of Kubernetes secrets to hold the TLS digital certificates. These certificates are not generated for ingress by default. However, enabling TLS is a best practice. For an overview of TLS for SAS Viya Monitoring for Kubernetes, see [“Transport Layer Security \(TLS\): Digital Certificates and Kubernetes Secrets”](#) on page 15.

Use the following steps:

Use either Contour or Ingress NGINX sample for your Kubernetes ingress controller.

- 1 Set up either host-based or path-based ingress by using the README.md from one of the following samples:
 - The [Contour](#) sample in the repository, which uses [Kubernetes Contour](#) to make web applications accessible to users.
 - The [Ingress NGINX](#) sample in the repository, which uses [Kubernetes ingress resources](#) to make web applications accessible to users.

Note: In November, 2025, the Kubernetes community announced that it would retire Ingress NGINX. For information, see [Ingress NGINX Retirement: What You Need to Know](#). With the project's imminent retirement, Contour is now recommended. However, SAS Viya Monitoring for Kubernetes will continue to support the use of Ingress NGINX for the foreseeable future. SAS recommends that you change the ingress controller in existing SAS Viya platform deployments to Contour ingress. For information, see the [“Overview” in SAS Viya Platform: Changing Ingress Controllers in a Deployment](#).

Create Kubernetes Secret to enable TLS for communications into the cluster.

- 1 Review the following notes before you create a set of Kubernetes secrets to enable TLS for communications into the cluster:
 - These certificates are separate from the secrets that are used to secure the in-cluster communications.

- Unlike TLS for in-cluster communications, the deployment scripts do not automatically generate TLS digital certificates for you. You must provide the TLS certificates for communications into the cluster. (The deployment scripts can create the secrets if you use the [autogeneration on page 26](#) method.)
 - When direct connections to OpenSearch are enabled (either via ingress or by enabling a NodePort), HTTPS must be used instead of HTTP when connecting. This is true regardless of whether HTTPS is required for OpenSearch Dashboards.
 - Secrets for Prometheus and Alertmanager are optional. This project does not make these applications accessible by default because they do not have a built-in user-authentication mechanism. If you choose to make them accessible, use the secrets that are listed in this [table on page 35](#) to define the appropriate set of TLS certificates.
 - To renew certificates, see [“Renew Transport Layer Security \(TLS\) Certificates” on page 107](#).
- 2 Use the following steps if you choose to use a single Kubernetes Secret resource that contains the ingress TLS certificates for all of the web applications that are deployed in a given namespace.

Note: As an alternative, you can choose to create a separate Kubernetes Secret for each web application. This would allow you to use different TLS certificates for different web applications. For a list of secrets that you create, see the [Table 5 on page 35](#) table. This project originally used this approach, but recommends the simpler strategy if it fits within your organization's policies.

Make sure you have the following certificate files:

TLS certificate (the "cert")

used to generate the `tls.crt` file. It must be an X.509 PEM certificate.

TLS key (the "key")

used to generate the `tls.key` file. It must be an X.509 PEM key.

- 3 Create the following namespaces, depending on if you deploy the log-monitoring components, metric-monitoring components, or both:

```
kubectl create namespace monitoring
```

```
kubectl create namespace logging
```

- 4 Use the following commands to generate the secret for your logging and/or monitoring namespaces:

```
kubectl create secret tls v4m-ingress-tls-secret --namespace logging --key=cert_key_file --cert=cert_file
```

```
kubectl create secret tls v4m-ingress-tls-secret --namespace monitoring --key=cert_key_file --cert=cert_file
```

Where:

- `v4m-ingress-tls-secret` is the name of the secret that is used.
- `namespace` is the namespace that is used during the deployment process. The following values are used by default:

Log monitoring	<code>logging</code>
----------------	----------------------

Metric monitoring	<i>monitoring</i>
-------------------	-------------------

If you change the default, be sure to specify the name of your namespace instead of the default.

- *cert_key_file* is the TLS certificate key file associated with the secret name.
- *cert_file* is the TLS certificate file associated with the secret name.

The following tables list the Kubernetes secrets that are used for secured connections into the cluster.

Table 4 *Single Kubernetes Secret Used for All Web Applications*

Kubernetes Secret	Description
v4m-ingress-tls-secret	Secures the connection into any web application ingress resource that you have deployed.

Table 5 *Separate Kubernetes Secrets For Each Web Application*

Kubernetes Secret	Description
kibana-ingress-tls-secret ⁽¹⁾	Secures the connection into the OpenSearch Dashboards ingress resource.
elasticsearch-ingress-tls-secret ^{(1) (2)}	Secures the connection into the OpenSearch ingress resource.
grafana-ingress-tls-secret	Secures the connection into the Grafana ingress resource.
prometheus-ingress-tls-secret ⁽²⁾	Secures the connection into the Prometheus ingress resource.
alertmanager-ingress-tls-secret ⁽²⁾	Secures the connection into the Alertmanager ingress resource.

⁽¹⁾To maintain compatibility, these options have not been renamed to indicate their support of OpenSearch Dashboards and OpenSearch.

⁽²⁾Secrets for Prometheus, Alertmanager, and OpenSearch are optional. This project does not make these applications accessible by default because they do not have a built-in user-authentication mechanism. If you choose to make them accessible, use the secrets that are listed above to define the appropriate set of TLS certificates.

Configure Access to OpenSearch Dashboards via NodePorts

SAS Viya Monitoring for Kubernetes does not create a NodePort for accessing OpenSearch Dashboards by default. If Ingress is not configured and the `KB_KNOWN_NODEPORT_ENABLE` environment variable is not set to `true`, OpenSearch Dashboards is accessible via port forwarding only.

OpenSearch Dashboards can be made available through Ingress, which is the preferred method for production environments. For information about how to do this, see [“Automatically Generate Ingress Definitions and Kubernetes Secrets” on page 26](#).

To make **OpenSearch Dashboards** available via NodePorts, complete the following steps:

- 1 Open the `$USER_DIR/logging/user.env` file where `$USER_DIR` is the location of your customization files.
- 2 Add or update this line:

```
KB_KNOWN_NODEPORT_ENABLE=true
```

Note: If you choose to configure access via NodePorts after you deploy SAS Viya Monitoring for Kubernetes, use the `configure_nodeport.sh` script. For information, see [“Configure Access to Web Applications via NodePorts” on page 89](#).

Configure Access to Prometheus and Alertmanager via NodePorts

The user interfaces of Prometheus and Alertmanager are not required for metric gathering or alert notification. Thus, both user interfaces are disabled by default in the SAS Viya Monitoring for Kubernetes solution.

IMPORTANT Although it is possible to make Prometheus and Alertmanager web applications accessible, you might not want to because these web applications do not include built-in user authentication mechanisms. Therefore, making them accessible without implementing an authentication mechanism is a security risk.

If you choose to use NodePorts to access Prometheus and Alertmanager, complete the following steps:

- 1 Open the `$USER_DIR/monitoring/user.env` file where `$USER_DIR` is the location of your customization files.
- 2 Add or update this line:

```
PROM_NODEPORT_ENABLE = true
```

To access Prometheus and Alertmanager via NodePorts, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

Note: If you choose to configure access via NodePorts after you deploy SAS Viya Monitoring for Kubernetes, use the `configure_nodeport.sh` script. For information, see [“Configure Access to Web Applications via NodePorts” on page 89](#).

TLS for In-Cluster Communications

Introduction

For an overview of TLS for SAS Viya Monitoring for Kubernetes, see [“Transport Layer Security \(TLS\): Digital Certificates and Kubernetes Secrets” on page 15](#).

In-cluster communications is communication between components within the cluster. The following types of communication are examples:

- between OpenSearch nodes
- between OpenSearch and other log-monitoring components
- between Prometheus and Grafana
- between Prometheus (or Grafana) and other metric-monitoring components

By default, TLS is configured for in-cluster communications by using OpenSSL to generate the certificates. Log-monitoring components must always communicate through TLS-enabled connections. For the metric-monitoring components, the use of TLS for in-cluster communications can be disabled.

Note: The Prometheus node exporter and kube-state-metrics exporters currently do not support TLS. These components do not communicate outside the cluster (that is, they are not exposed over Ingress).

Note: Prior to release 1.2.1, cert-manager was used to generate certificates. You can still use cert-manager to generate the TLS certificates. See [“Enable cert-manager Instead of OpenSSL to Generate the TLS Certificates” on page 37](#).

If your organization requires that its own certificates be used, you must create the associated Kubernetes secrets before starting the deployment process. See [“Use Your Own Certificates to Create Kubernetes Secrets” on page 38](#).

Enable cert-manager Instead of OpenSSL to Generate the TLS Certificates

If you are not providing your own certificates, the deployment process can generate the required certificates for you. By default, the certificates are generated by using OpenSSL. However, the deployment process can use cert-manager if it is available in the cluster.

The default setting is `openssl`.

To use cert-manager, complete these steps before you run the deployment script:

- 1 Open the `$USER_DIR/logging/user.env` file where `$USER_DIR` is the location of your customization files.

- 2 Add or update this line:

```
CERT_GENERATOR=cert-manager
```

Use Your Own Certificates to Create Kubernetes Secrets

IMPORTANT Before you start this procedure, make sure you have set up your `$USER_DIR` directory. See [“Create the Directory for Your Customization Files” on page 20](#).

Your organization might have certificates already. For each application that you want to secure, make sure you have the certificates for that application's Kubernetes secret. You must pre-populate the Kubernetes secrets with your certificates before running the deployment script.

Complete the following steps:

- 1 You need the following three certificate files for each secret:

TLS certificate (the "cert")

used to generate the `tls.crt` file. It must be an X.509 PEM certificate.

TLS key (the "key")

used to generate the `tls.key` file. It must be an X.509 PEM key.

Certificate from the signing certificate authority (the "CA certificate")

used to generate the `ca.crt` file. It must be an X.509 PEM certificate.

Note: The certificate that you use for the `es-admin-tls-secret` secret cannot be reused for any of the other TLS certificates.

- 2 Create the following namespaces, depending on if you deploy the log-monitoring components, metric-monitoring components, or both:

```
kubectl create namespace monitoring
```

```
kubectl create namespace logging
```

- 3 Use the following command to generate the secrets. Run the command for each secret that you require.

```
kubectl -n namespace create secret generic secret_name --from-file=tls.crt=tls_cert_name.pem
--from-file=tls.key=key_name.key --from-file=ca.crt=CA_cert_name.pem
```

Where:

- `namespace` is the namespace that is used during the deployment process. The following values are used by default:

Log monitoring	<i>logging</i>
Metric monitoring	<i>monitoring</i>

If you change the default, be sure to specify the name of your namespace instead of the default.

- `secret_name` is one of the following values:

For log monitoring:

- `es-transport-tls-secret`

- es-rest-tls-secret
- es-admin-tls-secret
- kibana-tls-secret

For metric-monitoring:

- prometheus-tls-secret
- alertmanager-tls-secret
- grafana-tls-secret
- *tls_cert_name*, *key_name*, and *CA_cert_name* is a set of files that you obtained in step 1.

Note: You need all three files for each secret that is being generated.

- 4 Repeat step 2 until you have created all seven of the required Kubernetes secrets.

Kubernetes Secrets That Are Used for Secured In-Cluster Connections

The following table lists the Kubernetes secrets that are used for secured connections within the cluster.

Table 6 *Kubernetes Secrets That Are Used for Secured In-Cluster Connections*

Log-Monitoring Kubernetes Secrets	
es-transport-tls-secret	Secures communications between OpenSearch nodes.
es-rest-tls-secret	Secures communications between OpenSearch and incoming REST-call traffic.
es-admin-tls-secret	Secures communications for internal administration actions that are performed locally on the OpenSearch nodes.
kibana-tls-secret	Secures communications between OpenSearch Dashboards and either the Ingress object or the user's web browser (if NodePorts are used).
Note: To maintain compatibility, these options have not been renamed to indicate their support of OpenSearch and OpenSearch Dashboards.	
Metric-Monitoring Kubernetes Secrets	
prometheus-tls-secret	Secures communications between Prometheus and other components.
alertmanager-tls-secret	Secures communications between Alertmanager and other components.
grafana-tls-secret	Secures communications between Grafana and other components.

Customize StorageClass

Configuring StorageClass for Persistent Volume Claims (PVCs)

Kubernetes makes long-term storage available in your cluster in the form of Persistent Volume (PV) resources. StorageClass resources provide a way for administrators to describe the classes of storage that they offer and every PV is associated with a StorageClass. A Persistent Volume Claim (PVC) resource makes a PV available to a particular pod. PVCs are configured for the following five components that are deployed by SAS Viya Monitoring for Kubernetes:

- OpenSearch
- Alertmanager
- Grafana
- Prometheus
- Prometheus Pushgateway

The deployment script assumes that your cluster has a default StorageClass that it uses to support the SAS Viya Monitoring for Kubernetes PVCs. A warning is issued if no default StorageClass is available. You can set the StorageClass by using either of the two methods that follow and the warning still appears, but you can safely ignore it. If you have not specified a different StorageClass to use for a PVC and there is no default StorageClass, the deployment process fails .

To prevent the monitoring applications and your SAS Viya deployment from competing for the same disk space, consider directing the monitoring applications' PVCs to a different Kubernetes StorageClass. This prevents contention and insulates each one from storage issues that are caused by the other. For example, if you use a different StorageClass for OpenSearch and your SAS Viya deployment runs out of disk space, OpenSearch continues to operate. Alternatively, if your OpenSearch runs out of disk space, SAS Viya continues to operate.

You can explicitly specify the StorageClass with either of the following methods:

- Modify the applicable `YAML` file that is used for Helm processing. See [“Specify the StorageClass” on page 40](#) for the manual customization steps.
- Automatically generate StorageClass references with environment variables. See [“Automatically Generate Storage Class References” on page 41](#).

Specify the StorageClass

To specify an alternate StorageClass for use, complete the following steps:

- 1 Open the applicable file used for Helm processing:
 - For OpenSearch, edit `logging/user-values-opensearch.yaml`.
 - For OpenSearch Dashboards, edit `logging/user-values-osd.yaml`.
 - For Grafana, edit `monitoring/user-values-prom-operator.yaml`
- 2 Uncomment the applicable lines in the file to specify the StorageClass that is used for that PVC.

IMPORTANT If you edit a YAML file, ensure that the parent item of any item that you uncomment is also uncommented. For example, in the `user-values-opensearch.yaml` file, if you uncommented the `storageClass` item for the OpenSearch primary nodes, you must also uncomment the `persistence` item, as shown below:

```
# Sample user-values-opensearch.yaml

persistence:  # Uncommented because 'storageClass' is uncommented.
  storageClass: alt-storage # Uncommented to direct OpenSearch to an alternate storageClass
```

- 3 Confirm that the `storageClassName` matches your preferred StorageClass.

Automatically Generate Storage Class References

You can generate a StorageClass for PVCs during the SAS Viya Monitoring for Kubernetes deployment process. This automation eliminates the need to manually define these common customizations in the appropriate `yaml` file.

Review the following notes of this automation:

- This functionality requires that you have the `yq` utility available on the host where the deployment scripts are run.
- This automation does not create the specified StorageClass. It only allows the deployment tooling to indicate the StorageClass that is used by the PVC.
- The StorageClass that is referenced must exist. If a StorageClass does not exist, the deployment process fails with an error message.
- The only validation that is done is to confirm that the specified StorageClass exists.

Use the following steps to provide this information to the appropriate Helm chart prior the deployment process:

- 1 Create your `$USER_DIR` directory that contains your sample files and `user.env` files. For information, see [“Create the Deployment Directory” on page 18](#).
- 2 To enable auto-generation of StorageClass, add the following two environment variables to the `$USER_DIR/user.env` file:

```
AUTOGENERATE_STORAGECLASS=true
STORAGECLASS=name_of_storage_class
```

The `$STORAGECLASS` environment variable is used for all PVCs when `AUTOGENERATE_STORAGECLASS` is set to `true`. You can optionally choose a specific StorageClass for each SAS Viya Monitoring for Kubernetes component. See Step 3.

Note: You can specify a single StorageClass to use for all of the PVCs or a different StorageClass for each PVC, or you can specify a combination of those two options.

- 3 **Optional:** If you have multiple StorageClasses that will be used for specific SAS Viya Monitoring for Kubernetes components, use the following environment variables to specify this:
 - Alertmanager:

`ALERTMANAGER_STORAGECLASS=name_of_storage_class_for_AlertManager`

- Grafana:

`GRAFANA_STORAGECLASS=name_of_storage_class_for_Grafana`

- Prometheus

`PROMETHEUS_STORAGECLASS=name_of_storage_class_for_Prometheus`

- Opensearch:

`OPENSEARCH_STORAGECLASS=name_of_storage_class_for_Opensearch`

- Prometheus Pushgateway:

`PUSHGATEWAY_STORAGECLASS=name_of_storage_class_for_Pushgateway`

The following table provides details about the environment variables that are available for configuring this functionality.

Table 7 Storage Class Environment Variables

Environment Variable	Default Value	Note
AUTOGENERATE_STORAGECLASS	false	When true the storageClass assigned to each PVC is auto-generated
STORAGECLASS	none	storageClass that is used for PVCs when a component is not explicitly assigned a storageClass.
ALERTMANAGER_STORAGECLASS	\$STORAGECLASS	storageClass for Alertmanager PVC
GRAFANA_STORAGECLASS	\$STORAGECLASS	storageClass for Grafana PVC
PROMETHEUS_STORAGECLASS	\$STORAGECLASS	storageClass for Prometheus PVC
OPENSEARCH_STORAGECLASS	\$STORAGECLASS	storageClass for OpenSearch PVC
PUSHGATEWAY_STORAGECLASS	\$STORAGECLASS	storageClass for Prometheus Pushgateway PVC

Customize Workload-Node Placement

(Log and metric monitoring)

About Workload-Node Placement

SAS Viya is deployed using a workload-node-placement strategy. The strategy uses the `workload.sas.com/class` taint to optimize the placement of its components on Kubernetes nodes.

By default, the log- and metric-monitoring components do not participate in this strategy. This is a best practice because it enables the components to function even if `workload.sas.com/class`

tainted nodes are scaled to zero (in other words, are shut down). Therefore, by default, most of the log- and metric-monitoring components are deployed to cluster nodes that do not have `workload.sas.com/class taints`.

Note: On Microsoft Azure, this might result in pods being deployed on nodes in the `system` nodepool.

Force Use of the Default Workload-Node-Placement Strategy

If you want one or both sets of the monitoring components to use the SAS Viya workload-node-placement strategy (instead of the best-practice deployment), complete the following steps:

- 1 Open your preferred `user.env` file:

Component that Uses the Setting	File Location
Log monitoring	<code>\$USER_DIR/logging/user.env</code>
Metric monitoring	<code>\$USER_DIR/monitoring/user.env</code>
Both components	<code>\$USER_DIR/user.env</code>

Where `$USER_DIR` is the location of your customization files.

- 2 Add or update the applicable line:

Log monitoring	<code>NODE_PLACEMENT_ENABLE=true</code>
Metric monitoring	<code>MON_NODE_PLACEMENT_ENABLE=true</code>

Prepare to Deploy SAS Viya Monitoring for Kubernetes in an Air Gap Environment

Introduction

This section is intended for users who are deploying SAS Viya Monitoring for Kubernetes in an air gap environment. These environments might also be called *dark sites*.

You can use the following methods to set up your private registry before you deploy SAS Viya Monitoring for Kubernetes:

- Create environment variables and then run the `setup_airgap.sh` script to automate the process. For the steps to use, see [“Use a Script to Populate Your Private Registry” on page 45](#).
- Perform manual steps to set up your environment, which allows for more control. For the steps to use, see [“Manually Populate Your Private Registry” on page 45](#).

The following tasks are performed regardless of the method that you choose:

- Container images and Helm charts are loaded into a private registry. You can use the same private registry for both the container images and Helm charts, or you can have a private registry for container images and a separate private registry for Helm charts.

The container images and Helm charts are listed here: ['Container Images and Helm Charts Used by SAS Viya Monitoring for Kubernetes' in ARTIFACT_INVENTORY.md](#).

Note: You can choose to deploy Helm charts from your local copies that you download. In this scenario, you must perform the manual steps that are located here: [“Manually Populate Your Private Registry” on page 45](#).

- The Prometheus Operator Custom Resource Definitions (CRDs) are downloaded to a **prometheus-operator-crd/version-specific** subdirectory in the `$USER_DIR/monitoring/` directory.
- The OpenSearch Grafana data source plug-in is downloaded to your `$USER_DIR/monitoring` directory. This task is done only if you want to configure your environment to view log messages in Grafana dashboards.
- Your `$USER_DIR/user.env` file is populated with the appropriate environment variables.

The following considerations are true regardless of which method you choose:

- This procedure is generic and not cloud-provider specific. Each cloud provider might have its own nuances. See your cloud provider's documentation.
- This procedure uses Docker commands. It assumes that Docker is installed and running.
- Typically, cloud providers provide their own command-line interfaces (CLI). A CLI might include the capability to load images into private registries. In some cases, a CLI might provide a more efficient process than Docker commands. For example, on Microsoft Azure using Azure Container Registry (ACR), three Docker commands can be replaced by a single Azure CLI command. See the cloud provider's documentation for more information.

Prerequisites

Review the following prerequisites:

- [Helm](#) version 3.8 or later is required for an air gap environment.
- A private registry on your cloud provider is required. To complete this configuration, you must have the following information from your private registry:
 - ☐ registry URL
 - ☐ registry user name
 - ☐ registry password

For information about private registries, see [“Create a Mirror Registry at an Air-Gapped Site” in SAS Viya Platform: Deployment Guide](#).

Use a Script to Populate Your Private Registry

The `setup_airgap.sh` script, that is available with version 1.2.46 (February 2026), automates the process of loading container images and Helm charts into your private registry before deploying SAS Viya Monitoring for Kubernetes into an air gap environment.

Perform the following steps:

- 1 Make sure that you have set up your `$USER_DIR` directory that contains your `yaml` files and `user.env` files. For more information, see [“Create the Deployment Directory” on page 18](#).

- 2 Add the following environment variables to the `$USER_DIR/user.env` file:

```
AIRGAP_REGISTRY=private_registry_URL
AIRGAP_REGISTRY_USERNAME=private_registry_username
AIRGAP_REGISTRY_PASSWORD=private_registry_password
```

Where:

- `private_registry_URL` is the location of your private registry.
- `private_registry_username` is the private registry user name.
- `private_registry_password` is the private registry password.

- 3 Run the following command from the directory that contains the copy of your repository:

```
/bin/setup_airgap.sh
```

- 4 Follow the steps in [“Prepare to Deploy” on page 50](#) to finish the air gap setup.

Manually Populate Your Private Registry

Perform the following steps to manually set up your private registry before deploying SAS Viya Monitoring for Kubernetes into an air gap environment.

Step 1: Import Images into Your Private Registry

Complete these steps to import images into your private registry:

- 1 Review [“Table 1. Container Images” in ARTIFACT_INVENTORY.md](#) to identify the components that you must import. You can import the components that are required for log monitoring, metric monitoring, or both.

- 2 Run the following set of commands for each image:

```
docker pull fully_qualified_container-image_name
docker tag fully_qualified_container-image_name registry_name/container-image_name
docker push registry_name/container-image_name
```

Where:

- `fully_qualified_container-image_name` is the entire value from the **Fully Qualified Container-Image Name** column in “Table 1. Container Images”. The value uses this format: `registry/repository/image_name:version`

IMPORTANT The fully qualified container-image name that you specify must be correct. Use the entire value shown in the **Fully Qualified Container-Image Name** column of "Table 1. Container Images".

- *registry_name* is the name of your private registry. Preserve the forward slash (/) between *registry_name* and *container-image_name*.
- *container-image_name* is only a subset of the value from the **Fully Qualified Container-Image Name** column in "Table 1. Container Images":

```
repository/image_name:version
```

IMPORTANT The repository, image name, and version number that you specify must be correct. Use the version number shown in "Table 1. Container Images".

Here is an example that imports the `Grafana:11.2.0` image into a Microsoft Azure private registry:

```
docker pull docker.io/grafana/grafana:11.2.0
docker tag docker.io/grafana/grafana:11.2.0 myacr.azurecr.io/grafana/grafana:11.2.0
docker push myacr.azurecr.io/grafana/grafana:11.2.0
```

Step 2: Enable Access to Helm Chart Repositories

Complete these steps to enable Helm to access each Helm chart repository at its specified location.

- 1 Review the following columns in "Table 2. Helm Chart Repositories" in [ARTIFACT_INVENTORY.md](#):

- Helm Repository
- Helm Repository URL

- 2 Run the following command for each Helm chart:

```
helm repo add Helm_repository Helm_repository_URL
```

Where:

- *Helm_repository* is the value from the **Helm Repository** column in "Table 2. Helm Chart Repositories".
- *Helm_repository_URL* is the value from the **Helm Repository URL** column in "Table 2. Helm Chart Repositories".

Step 3: Download the Helm Charts to Your Deployment System

Complete these steps to download each Helm chart to the system on which you are deploying:

- 1 Review the following columns in "Table 3. Helm Chart Information" in [ARTIFACT_INVENTORY.md](#):

- Helm Chart Version
- Helm Chart Repository
- Helm Chart Name

- 2 Run the following command for each Helm chart:

```
helm pull --destination local_Helm_directory --version Helm_chart_version
Helm_chart_repository/Helm_chart_name
```

Where:

- *local_Helm_directory* is the Helm directory on the system on which you are deploying.
- *Helm_chart_version* is the value from the **Helm Chart Version** column in "Table 3. Helm Chart Information".
- *Helm_chart_repository* is the value from the **Helm Chart Repository** column in "Table 3. Helm Chart Information". Be sure to preserve the forward slash (/) between *Helm_chart_repository* and *Helm_chart_name*.
- *Helm_chart_name* is the value from the **Helm Chart Name** column in "Table 3. Helm Chart Information".

Step 4: Push the Helm Charts to a Private Registry

In this step, you push Helm charts to the private registry that also includes your container images that you imported from step 1, or to a private registry that contains only Helm charts.

Note: If you push Helm charts to a private registry that is used only for Helm charts, you need to specify the location of these Helm charts in the `AIRGAP_HELM_REPO` environment variable. This step is documented in ["Prepare to Deploy" on page 50](#).

Note: Skip this step if you choose to deploy Helm charts from your local copies that you downloaded in the previous step. In this scenario, you need to set the `AIRGAP_HELM_FORMAT` and the `AIRGAP_HELM_REPO` environment variables. This step is documented in ["Prepare to Deploy" on page 50](#). Continue to ["Step 5: Download the Prometheus Operator CRDs" on page 48](#).

Use the following steps to push each Helm chart to the private registry:

- 1 Run the following command to log in to the private registry:

```
helm registry login -u private_registry_username -p private_registry_password
```

Where:

- *private_registry_username* is the private registry user name.
- *private_registry_password* is the private registry password.

The `helm registry login` command provides Helm with the credentials that are needed during the installation process.

Note: If you are using a cloud registry provider, use the cloud provider's CLI to access the user name and password for your cloud registry. The user account that is specified in the authentication must have permission to push images to the cloud registry.

- 2 Review the following columns in ["Table 3. Helm Chart Information" in ARTIFACT_INVENTORY.md](#):

- Helm Chart Repository
- Helm Archive File Name

- 3 Run the following command for each Helm chart:

```
helm push local_Helm_directory/Helm_archive_file_name oci://private_registry_URL/
Helm_chart_repository
```

Where:

- *local_Helm_directory* is the Helm directory on the system on which you are deploying. Preserve the forward slash (/) between *local_Helm_directory* and *Helm_archive_file_name*.
- *Helm_archive_file_name* is the value from the **Helm Archive File Name** column in "Table 3. Helm Chart Information".
- *private_registry_URL* is the URL to the private registry where the images and Helm charts reside. Preserve the forward slash (/) between *private_registry_URL* and *Helm_chart_repository*.
- *Helm_chart_repository* is the value from the **Helm Chart Repository** column in "Table 3. Helm Chart Information".

Step 5: Download the Prometheus Operator CRDs

The Prometheus Operator Custom Resource Definitions (CRDs) are used to deploy the metric-monitoring components.

Note: If you are deploying only log-monitoring components, skip this step.

Complete the following steps to download the Prometheus Operator CRDs:

- 1 See "Table 1. Container Images" in [ARTIFACT_INVENTORY.md](#).

Locate the row for **Prometheus Operator**. From the **Fully Qualified Container-Image Name** column, copy the version number including the "v" that precedes the number, for example:

```
quay.io/prometheus/prometheus-operator:v0.00.0
```

IMPORTANT The version number that you specify must be correct. Be sure to use the version number shown in "Table 1. Container Images".

- 2 Create the following subdirectory in your `$USER_DIR` directory:

```
mkdir $USER_DIR/monitoring/prometheus-operator-crd/prometheus_operator_version
```

Where:

- `$USER_DIR` is the location indicated by the `USER_DIR` environment variable.
- *prometheus_operator_version* is the Prometheus Operator version that you copied in the previous step.

IMPORTANT *prometheus_operator_version* must include the "v" that precedes the version number (for example, v0.00.0).

- 3 Download the following YAML files and save them to the directory that you created in the previous step.

IMPORTANT For each of these files, replace *prometheus_operator_version* with the value that you used in the previous step.

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_alertmanagerconfigs.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_alertmanagerconfigs.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_alertmanagers.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_alertmanagers.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_prometheuses.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_prometheuses.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_prometheusrules.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_prometheusrules.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_podmonitors.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_podmonitors.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_servicemonitors.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_servicemonitors.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_thanosrulers.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_thanosrulers.yaml)

[https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/
prometheus_operator_version/example/prometheus-operator-crd/
monitoring.coreos.com_probes.yaml](https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/prometheus_operator_version/example/prometheus-operator-crd/monitoring.coreos.com_probes.yaml)

Step 6: Download the OpenSearch Grafana Data Source Plug-in (Optional)

SAS Viya Monitoring for Kubernetes stores log messages in OpenSearch, and these log messages are viewed in OpenSearch Dashboards. However, you can optionally configure your environment to view log messages in Grafana dashboards. To enable this feature, you must run the `create_logging_datasource.sh` after you deploy SAS Viya Monitoring for Kubernetes. The script downloads the OpenSearch Grafana data source plug-in from the Grafana website. When you deploy into an air gap environment, you must manually download the plug-in from the Grafana website and make it available to the script. Complete the following steps:

- 1 Locate the OpenSearch Datasource Plugin (Grafana) in "Table 4. Miscellaneous Component Version Information" in `ARTIFACT_INVENTORY.md`.
- 2 Note the version number in the **Version** column and click the link in the **Project Repository** column to access the available releases from the *grafana-opensearch-datasource* project repository.
- 3 Scroll to the correct version in the *grafana-opensearch-datasource* project and expand the **Assets** section.
- 4 Locate the link to the ZIP file for the `linux_amd64` architecture. Here is an example: `grafana-opensearch-datasource-2.18.0.linux_amd64.zip`
- 5 Download this ZIP file into your `$USER_DIR/monitoring` directory.

Viewing log messages in Grafana dashboards requires that you run the `create_logging_datasource.sh` script AFTER you have deployed SAS Viya for Kubernetes in an air gap environment. See [“Run the create_logging_datasource.sh Script” on page 101](#) for instructions on running the script.

Step 7: Prepare to Deploy

- 1 Follow the steps in [“Prepare to Deploy” on page 50](#) to finish the air gap setup.

Prepare to Deploy

Make sure that you have completed the automatic or manual setup before completing the following steps:

- 1 Create the log-monitoring namespace:

```
kubectl create namespace log-monitoring_namespace
```

Where *log-monitoring_namespace* is the name that you specify for the log-monitoring namespace.

TIP The default name is logging.

Note: You can choose to deploy both the log-monitoring and metric-monitoring components into the same namespace.

- 2 If you prefer that the metric-monitoring components reside in their own namespace, use the following command. Otherwise, skip this step.

```
kubectl create namespace metric-monitoring_namespace
```

Where *metric-monitoring_namespace* is the name that you specify for the metric-monitoring namespace.

TIP The default name is monitoring.

- 3 Create an image pull secret for each namespace. Enter this command for each namespace that requires a pull secret:

```
kubectl create secret docker-registry -n namespace private_registry_secret_name --docker-server=private_registry_URL --docker-username=private_registry_username --docker-password=private_registry_password --docker-email=email@organization.com
```

Where:

- *namespace* is the name of the namespace for which the pull secret is created. You must enter this command for each namespace that requires a pull secret:
 - ☐ the log-monitoring namespace.
 - ☐ the metric-monitoring namespace, if you created its own namespace.
 - ☐ any SAS Viya namespaces where the SAS Viya Monitoring for Kubernetes components reside. Repeat the command for each applicable SAS Viya namespace.

- *private_registry_secret_name* is the name of the secret that contains the login credentials to the private registry.

TIP The default is `v4m-image-pull-secret`.

- *private_registry_URL* is the URL to the private registry where the images and Helm charts reside.
- *private_registry_username* is the private registry user name.
- *private_registry_password* is the private registry password.
- *email@organization.com* is your email address.

4 In the `$USER_DIR/user.env` file, set the following environment variables to the values shown:

```
AIRGAP_DEPLOYMENT=true
AIRGAP_REGISTRY=private_registry_URL
AIRGAP_IMAGE_PULL_SECRET_NAME=private_registry_secret_name
```

Where:

- *private_registry_URL* is the URL to the private registry where the images and Helm charts reside.
- *private_registry_secret_name* is the name of the secret that contains the login credentials to the private registry.

TIP The default is `v4m-image-pull-secret`.

Note: When you use the same private registry for your container images and your Helm charts, the `AIRGAP_HELM_REPO` is set by default to the same value as `AIRGAP_REGISTRY`.

Note: The `AIRGAP_REGISTRY` variable is already set if you following the steps in [“Use a Script to Populate Your Private Registry”](#) on page 45.

5 **Note:** Skip this step if you performed the steps in [“Use a Script to Populate Your Private Registry”](#) on page 45.

In the `$USER_DIR/user.env` file, you must set additional environment variables in the following scenarios:

- You performed the steps in [“Manually Populate Your Private Registry”](#) on page 45 and you chose to deploy using local copies of your Helm Charts. Create the following two variables:

```
AIRGAP_HELM_FORMAT=tgz
AIRGAP_HELM_REPO=local_Helm_directory
```

Where *local_Helm_directory* is the Helm directory on the system on which you are deploying. This is the location that you used in [“Step 3: Download the Helm Charts to Your Deployment System”](#) on page 46.

- You performed the steps in [“Manually Populate Your Private Registry” on page 45](#) and you chose a separate private repository that contains only your Helm charts. Create the following variable:

```
AIRGAP_HELM_REPO=private_registry
```

Where *private_registry* the private registry that includes your Helm charts.

Note: The `AIRGAP_HELM_FORMAT` variable identifies the format in which the Helm charts are stored. The default format is OCI. Your local copies of Helm charts are stored as TGZ files. If you uploaded your Helm charts to a private registry in [“Step 4: Push the Helm Charts to a Private Registry” on page 47](#), the Helm charts are stored as OCI files and you do not need to set this environment variable.

- 6 If you pushed Helm charts to your private registry, you might need to provide Helm with the credentials that are needed during the installation process. (This was done automatically through the `setup_airgap.sh` or in [“Step 4: Push the Helm Charts to a Private Registry” on page 47](#).)

Run the following command to log in to the private registry:

```
helm registry login -u private_registry_username -p private_registry_password
```

Where:

- *private_registry_username* is the private registry user name.
 - *private_registry_password* is the private registry password.
- 7 Complete any other pre-deployment tasks that you require. Then, go to [“Deploy SAS Viya Monitoring for Kubernetes” on page 72](#).
 - 8 (Optional) If you want to view log messages in Grafana dashboards, you need to run the `create_logging_datasource.sh` script as a post-deployment step. See [“Run the create_logging_datasource.sh Script” on page 101](#) for instructions on running the script.

Note: Before running the script, the OpenSearch Grafana data source plug-in must be downloaded to your `$USER_DIR/monitoring` directory. This was done through one of the following methods:

- With the `setup_airgap.sh` in Step 3 of [“Use a Script to Populate Your Private Registry” on page 45](#).
 - Step 6 of [“Manually Populate Your Private Registry” on page 45](#).
-

Customize the Retention Period for Log Messages

IMPORTANT Every organization's usage patterns are different. There is no simple rule for calculating the storage impact of adjusting the retention period for log messages.

You are encouraged to review the storage used by log messages after the initial deployment. Doing so enables you to make educated estimates to adjust the amount of storage required to store the log messages for your desired retention period. If you increase the retention period,

you must increase the size of the persistent volume claims (PVCs) associated with the OpenSearch pods.

Be sure to read [“Adjust the Retention Period for Log Messages” on page 93](#) and [“Increase Storage for Log Messages” on page 96](#).

To customize the retention period prior to the deployment of the log-monitoring components, complete these steps:

- 1 Open the file `$USER_DIR/logging/user.env` where `$USER_DIR` is the location of your customization files.
- 2 Adjust the applicable log-message retention-period environment variable:

Log-Message Source	Environment Variable	Default Retention Period
SAS Viya and Kubernetes pods	LOG_RETENTION_PERIOD	3 days
SAS Viya Monitoring components	OPS_LOG_RETENTION_PERIOD	1 day
Audit records of OpenSearch user activity	OS_SECAUDIT_RETENTION_PERIOD Note: The OS_SECAUDIT_RETENTION_POLICY_ENABLE environment variable can be set to <code>false</code> to disable the security audit log index policy. (Security audit logs indexes are not deleted if you change the value to <code>false</code> .)	90 days
(OpenShift only) OpenShift infrastructure namespaces	INFRA_LOG_RETENTION_PERIOD	1 day

Customize Security Configuration Files

(For log monitoring only)

About Security Configuration Files

Note: The OpenSearch security plug-in is configured to provide a default security configuration. This configuration meets most customers' needs. Although you can change that configuration in order to meet your needs and standards, very few need to do so. To prepare for security configuration, review the [OpenSearch Security documentation](#).

The security configuration for the OpenSearch security plug-in is specified in a set of YAML files, which are included in the SAS Viya Monitoring for Kubernetes repository. When you deploy the log-

monitoring components, the YAML files are loaded onto Kubernetes secrets. Then the secrets are mounted onto the OpenSearch and OpenSearch Dashboard pods so that they appear as files with the expected names and at the expected locations.

Table 8 Configuration Files and Associated Kubernetes Secrets

Configuration File	Kubernetes Secret
action_groups.yml	security-action-groups
config.yml	security-config
internal_users.yml ¹	security-internal-users
roles.yml	security-roles
roles_mapping.yml	security-roles-mapping

¹ A small number of internal users are created during deployment, primarily for interactions between SAS Viya Monitoring for Kubernetes components. If you need to create additional users, use the REST API, rather than modifying this file.

In most cases, you should not need to modify these files, and you can use the default values provided. If you do need to modify the values in the files, see [“Modify the Configuration File” on page 54](#).

CAUTION

Do not directly modify Kubernetes secrets for logging. This breaks the synchronization between the secrets and the information in the files, and errors are possible when the pods are restarted.

Modify the Configuration File

You should not need to modify the configuration file. For more information, see [“About Security Configuration Files” on page 53](#).

Note: To perform these tasks, you must have elevated Kubernetes permissions.

To modify the values in the configuration file, complete the following steps:

- 1 In your copy of the project repository, navigate to the `logging/opensearch/securityconfig` directory.
- 2 Locate the configuration file that you want to modify and copy it to the `$USER_DIR` directory.
- 3 Modify the configuration file.
- 4 Run the applicable deployment script for your environment.

For standard deployments `logging/bin/deploy_logging.sh`

For deployments on OpenShift	<code>logging/bin/deploy_logging_openshift.sh</code>
---------------------------------	--

Prevent Routing Network Traffic through a Proxy Server

Note: This topic applies to highly secure environments in which the deployment scripts are run on a machine whose network traffic is routed through a proxy server.

If you are deploying in this type of scenario, make sure that you include `localhost` to the list of hosts that are not routed through the proxy server. Do this on the `no_proxy` environment variable.

Run the `echo` command to see the existing list of hosts that are already assigned to the `no_proxy` environment variable.

```
echo no_proxy
```

Add `localhost` to the list of hosts on the `no-proxy` environment variable.

```
export no_proxy="localhost,the list of other hosts separated by commas"
```

The deployment scripts use port forwarding to make REST API calls to OpenSearch and OpenSearch Dashboards. If you do not include `localhost` in the list, the OpenSearch deployment fails.

Set Administrator User Passwords

About Administrator Users for Grafana and OpenSearch Dashboards

You can set a password for the following administrative users for Grafana and OpenSearch dashboards at deployment time:

Grafana administrator

The Grafana administrator user name is `admin`.

OpenSearch administrator

The OpenSearch administrator user name is `admin`.

OpenSearch `logadm` user

The `logadm` user is intended to be the primary account that is used for day-to-day log monitoring activities. It has access to all of the log messages that are collected by the log-monitoring components, which include all SAS Viya namespaces and any Kubernetes infrastructure namespaces. Although the `logadm` user can access all of the log messages, `logadm` can access only the `cluster_admins` OpenSearch Dashboards-tenant space.

Notes:

- If you do not set these passwords, default passwords for these users are randomly generated and displayed in a message during deployment. Different random strings are generated for each account.
- OpenSearch evaluates passwords for complexity. If you specify a password that does not meet the complexity standard, the script generates a password that does. The new password is

displayed as output. To change the password, use the OpenSearch Dashboards web application or the OpenSearch API.

- You can change these passwords after the deployment. See [“Change Administrator Passwords after Deployment” on page 82](#).

Specify Administrator Passwords

Complete the following steps:

For Grafana Admin:

- 1 Open the `$USER_DIR/monitoring/user.env` file where `$USER_DIR` is the location of your customization files.
- 2 Add or update this line:

```
GRAFANA_ADMIN_PASSWORD=your_password
```

Where `your_password` is the password that you specify.

For OpenSearch Admin and logadm User:

- 1 Open the `$USER_DIR/logging/user.env` file where `$USER_DIR` is the location of your customization files.
- 2 Add or update these lines:

```
ES_ADMIN_PASSWD=your_password
```

```
LOG_LOGADM_PASSWD=your_password
```

Where `your_password` is the password that you specify.

Note: These environment variables contain plain text passwords. You can choose to set these environment variables through some other mechanism to avoid having them saved in plain text in the file. Be sure that the deployment scripts can pick up these passwords if you use an alternative method.

Customize Alerts Managed by Grafana

About Grafana Alerting

Grafana includes capabilities that enable you to define the following alerting criteria:

alert rules

Define metric conditions and thresholds that trigger an alert.

contact points

Define where the resulting notifications are sent and how the message is formatted. You can route firing alerts through email, Slack, or to a webhook, for example. To configure your SMTP email server, see [“Configure Email Settings for Grafana Alerts” on page 58](#).

notification policies

Define how and when firing alerts are routed to the contact points.

You can configure alert rules, contact points, and notification policies through `YAML` files during the pre-deployment process. Sample `YAML` files are included in the project for creating Grafana-managed alert rules for SAS Viya environments. For more information, see [Sample Alerts](#).

Note: Alert rules, contact points, and notification policies can be configured through the **Alerting** page of Grafana after deploying SAS Viya Monitoring for Kubernetes. For instructions on how to do this, see [Configure Alert Rules](#) and [Configure Notifications](#) in Grafana documentation.

Provision Alert Rules, Contact Points, and Notification Policies

Configure alert rules, contact points, and notification policies, by adding customized `YAML` files to the `$USER_DIR/monitoring/alerting` subdirectory. Here are the steps:

- 1 Create one `YAML` file for each alerting configuration that you want to include.
 - **Alert rules:** SAS Viya Monitoring for Kubernetes includes sample files for adding Grafana alert rules that alert on metrics from your SAS Viya environments. The sample alert files are organized into subdirectories by a SAS Viya component or category. Each alert is stored in its own `YAML` file. Customize the alert rule `YAML` file to pertain to your environment. Some examples are:
 - ☐ namespace
 - ☐ alert thresholds
 - ☐ Persistent Volume Claims (PVCs)
 - ☐ container names

For more information about these sample alerts and the customizations that are needed, see the [Grafana Alerts README.md](#) file.

Note: Create one `YAML` file per alert that you want to provision.

Note: You can also use the Grafana Alerting interface to create alerts after you have initially deployed SAS Viya Monitoring for Kubernetes. Use the following steps:

- 1 Create an alert rule.
- 2 Export the alert rule in `YAML` format. You can provision this alert rule as is, or you can modify the properties in the exported `YAML` file to customize the alert.

For more information, see [Export Alerting Resources](#) and [Provision Alerting Resources](#) in Grafana documentation.

Note: Grafana-managed alerts that are provisioned cannot be modified through Grafana. You must export the alert in `YAML` format, make modifications, save the file in the `$USER_DIR/monitoring/alerting/` directory, and then deploy.

- Here is an example of a `YAML` file that configures a **custom contact point and notification policy for Slack**, a messaging platform. Alerts are routed to a specific contact point based on

matching labels. The following snippet example sends all alerts with *severity=critical* to the *slack-critical* custom contact point.

```
apiVersion: 1

contactPoints:
  - orgId: 1
    name: slack-critical
    receivers:
      - uid: slack-critical-receiver
        type: slack
        settings:
          url: https://hooks.slack.com/services/T00000000/B00000000/
XXXXXXXXXXXXXXXXXXXXXXXXXXXX
          recipient: '#alerts'
          disableResolveMessage: false

apiVersion: 1
notificationPolicies:
  - orgId: 1
    receiver: slack-critical
    matchers:
      - severity = critical
    group_by: ["alertname"]
    group_wait: 30s
    group_interval: 5m
    repeat_interval: 12h
```

Note: You can create multiple notification policies to target different types of alerts, such as routing warnings to email and critical alerts to Slack.

- 2 Save the file or files in the `$USER_DIR/monitoring/alerting/` directory where `$USER_DIR` is the location of your customization files.
- 3 After you deploy, alerts, contact points, or notification policies that you provisioned are displayed in **Grafana → Alerting**.

If your provisioned resources do not show up, use the following troubleshooting tips:

- 1 Check that your YAML syntax is valid.
- 2 Confirm that PromQL expressions return data by selecting **View** under the Actions column of an alert. You can then select **View in Explore**.
- 3 Look at Grafana pod logs for provisioning errors.
- 4 Ensure that the metrics that are used in your rules do exist.

Configure Email Settings for Grafana Alerts

If you choose to send alert notifications to an email address, you must configure your SMTP email server. You can choose to configure an SMTP email server through either of the following methods:

- “Automatic Configuration” on page 59
- “Manual Configuration” on page 60

Automatic Configuration

To enable this feature, add environment variables to your `$USER_DIR/user.env` file to automatically configure your SMTP email server. Here are the steps:

- 1 Make sure you have set up your `$USER_DIR` directory that contains your `yaml` files and `user.env` files. For more information, see [“Create the Deployment Directory” on page 18](#).
- 2 Add the following environment variables to the `$USER_DIR/user.env` file:

```
AUTOGENERATE_SMTP=true
SMTP_HOST=hostname SMTP_mail_server
SMTP_PORT=port SMTP_mail_server
SMTP_FROM_ADDRESS=FROM_email_address_of_generated_emails
SMTP_FROM_NAME=NAME_to_include_in_FROM_line_of_generated e-mails
```

Note: Error messages are generated if these variables are not set.

- 3 (Optional) You might need to set additional environment variables depending on your deployment. Here are examples:

```
SMTP_USER=username_to_connect_to_mailhost_server
SMTP_PASSWORD=password_to_connect_to_mailhost_server
SMTP_USER_SECRET=name_of_Kubernetes_secret_with_username/
password_to_connect_to_mailhost_server
```

If this secret already exists, the script uses it and ignores any values passed in through `SMTP_USER` and `SMTP_PASSWORD`. If the secret does not exist, the script creates the secret in the same namespace in which the monitoring components are being deployed. The default secret that is created for this environment variable is `grafana-smtp-user`.

```
SMTP_SKIP_VERIFY=true
```

When `SMTP_SKIP_VERIFY` is set to `true`, Grafana skips TLS verification when connecting to the mail host server. The default value is `false`.

```
SMTP_TLS_CERT_FILE=path_to_TLS_certificate_file_in_Grafana_container
```

The default value for `SMTP_TLS_CERT_FILE` is `/cert/tls.crt`. This is where TLS certificates are mounted when TLS is enabled.

```
SMTP_TLS_KEY_FILE=path_to_TLS_key_file_in_Grafana_container
```

The default value for `SMTP_TLS_KEY_FILE` is `/cert/tls.key`. This is where TLS certificates are mounted when TLS is enabled.

Note: If only `SMTP_USER` or `SMTP_PASSWORD` is set, an `ERROR` message is generated. However, it is acceptable that neither is set. You should set both variables or neither variable.

Manual Configuration

Use the following steps to manually configure an SMTP email server for alerting notifications:

- 1 Make sure you have set up your `$USER_DIR` directory that contains your `yaml` files and `user.env` files. For more information, see [“Create the Deployment Directory” on page 18](#).
- 2 Copy the `viya4-monitoring-kubernetes/monitoring/user-values-prom-operator.yaml` file to your `$USER_DIR/monitoring` subdirectory if you have not already done so for other customizations.
- 3 Modify the `$USER_DIR/monitoring/user-values-prom-operator.yaml` using the following example snippet. This example shows the **smtp** section that needs to be included in the **grafana:** **“grafana.ini”** stanza of that file:

```
grafana:
  "grafana.ini":
    smtp:
      enabled: true
      host: hostname SMTP_email_server
      from_address: grafanaAlerts@example.com
      from_name: Grafana_Alerting
      #skip_verify: false
      #user: user1
      #password: somepassword
      #cert_file: /cert/tls.crt
      #cert_key: /cert/tls.key
```

The keys that are highlighted in blue are required and the keys that are commented out are not required but they might be needed in your environment. For more information about the email server settings, see the [Configure Grafana Documentation](#).

Note: The name of the YAML file in **OpenStack** is `user-values-openshift-grafana.yaml` and the content of the file is slightly different. There is no top level `grafana` item and all other lines are indented less.

Validate SMTP Email Server Configuration in Grafana

If this feature has been enabled and the required values are configured correctly, you can send a test email from within Grafana to validate the configuration. Here are the steps:

- 1 Expand **Alerting** from Grafana's sidebar and select **Contact points**.
You should see a Contact Point that is labeled *grafana-default-email*.
- 2 Click **Edit** to the right of *grafana-default-email*.
- 3 Enter your email address in the **Addresses** box and click **Test** in the upper right of the box.
- 4 Click **Send test notification** in the Test contact point pop-up window.

If it fails, a window with a red error message appears with a message explaining why. If it succeeds, a window with a green success message appears. Check your inbox to confirm the email was sent. The email should be delivered in a minute or less. Confirm the `SMTP_FROM_ADDRESS` and `SMTP_FROM_NAME` values that you provided were used.

Configure Alertmanager

About Alertmanager Instances

In a typical deployment, each cluster uses its own instance of Alertmanager. However, using a single instance of Alertmanager that is external to the clusters can provide the following advantages:

- If you have multiple clusters, you might want the external instance of Alertmanager to generate the alerts for all of the clusters.
- If you have an existing instance of Alertmanager, you might prefer that the existing instance generate the alerts for your clusters.

Configure to Use an External Instance of Alertmanager

Use this procedure if you choose to use a single external instance of Alertmanager.

Complete the following steps:

- 1 Copy `samples/external-alertmanager/monitoring/alertmanager-endpoint.yaml` from your copy of the repository to `$USER_DIR`, which is the location of your customization files.

- 2 Edit `$USER_DIR/alertmanager-endpoint.yaml`.

- 3 In the **Service** section, replace `my-external-alertmanager` with the name of your external Alertmanager instance.

```
kind: Service
apiVersion: v1
metadata:
  name: my-external-alertmanager
```

- 4 In the **Endpoints** section, replace `my-external-alertmanager` with the same value that you used in step 3.

```
kind: Endpoints
apiVersion: v1
metadata:
  name: my-external-alertmanager
```

- 5 In the **subsets** section, replace `IP_value` with the IP address of your external instance of Alertmanager.

```
subsets:
- addresses:
  - ip: IP_value
  ports:
  - port: 9093
```

- 6 Save and close the file.

- 7 Edit `$USER_DIR/user-values-prom-operator.yaml` where `$USER_DIR` is the location of your customization files.

- 8 Turn off in-cluster use of Alertmanager. In the **alertmanager** section, update the `enabled` value to `false`.
- 9 In the **prometheus** section, replace *my-external-alertmanager* with the same value that you used in step 3.

Note: By default, the namespace is set to `monitoring`. If, when you deploy, you intend to use a different name for the metric-monitoring namespace, replace *monitoring* with that name.

```
prometheus:
  prometheusSpec:
    alertingEndpoints:
      - name: my-external-alertmanager
        namespace: monitoring
        port: http
        scheme: http
```

- 10 Save and close the file.

IMPORTANT After you deploy the metric-monitoring components, you must deploy the `alertmanager-endpoint.yaml` file to your metric-monitoring namespace. For the deployment procedure, see [“Deploy the External Alertmanager Configuration” on page 77](#).

Red Hat OpenShift Considerations

How SAS Viya Monitoring for Kubernetes Is Deployed on OpenShift

There are some important differences in how SAS Viya Monitoring for Kubernetes is deployed and how it operates on Red Hat OpenShift as compared to other cloud provider environments. These differences are explained in the following topics:

- [“Log Monitoring Components” on page 62](#)
- [“Metric Monitoring Components” on page 63](#)

Log Monitoring Components

The log-monitoring stack that is deployed by SAS Viya Monitoring for Kubernetes on OpenShift is the same stack that is deployed on other cloud provider environments. However, since OpenShift offers an optional log-monitoring solution, confirm that it has not been deployed before you deploy the SAS Viya Monitoring for Kubernetes log-monitoring components. Although you can deploy SAS Viya Monitoring for Kubernetes log-monitoring components when OpenShift logging is also deployed, this configuration is inefficient and it is not recommended for the following reasons:

- Each logging solution uses its own DaemonSet and each DaemonSet deploys its own log collection pods on every node in the cluster. This duplication wastes the limited CPU and memory resources that are available on each node in the cluster.
- Two separate storage systems store every log message. This duplication doubles the amount of storage used to store logs.

For information about OpenShift logging and monitoring, see [OpenShift Logging](#).

Note: There are two differences in how the SAS Viya Monitoring for Kubernetes components are configured on OpenShift:

- When you deploy on other supported Kubernetes distributions, the ServiceMonitors for the log-monitoring components are deployed as part of metric-monitoring components. On OpenShift, these ServiceMonitors are deployed as part of the log-monitoring components.
 - SAS Viya Monitoring for Kubernetes uses a third index management policy to govern log messages that are from namespaces that start with "openshift". Review the [log-message retention processing and index policies on page 93](#) for more information.
-

Metric Monitoring Components

Unlike other cloud providers, OpenShift deploys its own metric-monitoring stack that includes many of the same components that SAS Viya Monitoring for Kubernetes deploys. The OpenShift metric-monitoring stack includes Prometheus, Prometheus Operator, Node Exporter, and kube-state-metrics. (See [About OpenShift Container Platform monitoring](#) for the monitoring components that are deployed by default.)

The OpenShift metric-monitoring stack enables administrators to monitor the core OpenShift platform components with the same OpenShift web console that is used for other administrative tasks. These components can also be used to monitor user-defined projects and applications such as the SAS Viya platform. Because SAS Viya Monitoring for Kubernetes on OpenShift can integrate with these existing components, Grafana is the only metric-monitoring component that is installed with the project. It uses the Grafana Helm chart to deploy Grafana instead of the Kube-Prometheus Stack Helm chart that is used in other environments. Therefore, the customization process on OpenShift is different. For more information, see ["Customize Metric-Monitoring" on page 64](#).

SAS Viya Monitoring for Kubernetes on OpenShift does the following:

- Deploys the same set of Grafana dashboards for SAS Viya components that are deployed on other Kubernetes cluster environments.
- Deploys the same set of Prometheus recording rules to report metrics for SAS jobs.
- Uses OpenShift authentication to access Grafana. This enables administrators to seamlessly move between the OpenShift administrative web console and Grafana without having to sign in repeatedly. See ["OpenShift Authentication for Grafana" on page 64](#).

OpenShift Authentication for Grafana

Overview

Authentication is provided by the OpenShift oauth-proxy sidecar container in the Grafana pod and by default, SAS Viya Monitoring for Kubernetes uses this OpenShift authentication to log in to Grafana. This is done through the `OPENSIFT_AUTH_ENABLE` environment variable which has a default value of `true`. If OpenShift authentication is disabled, Grafana uses its internal user database for authentication.

Note: Regardless of the `OPENSIFT_AUTH_ENABLE` setting, in-cluster TLS (from the Ingress controller to Grafana) and Ingress TLS (from the web browser to the cluster) is enabled on the Grafana route. This route acts as the OpenShift version of Ingress. For more information, see the [OpenShift Route documentation](#).

Turn Off OpenShift Authentication for Grafana

To disable OpenShift authentication and instead use Grafana's internal user database for authentication, complete these steps:

- 1 Open the file `$USER_DIR/monitoring/user.env` where `$USER_DIR` is the location of your customization files.

- 2 Add or update this line:

```
OPENSIFT_AUTH_ENABLE=false
```

Customize Metric-Monitoring

When you deploy the metric-monitoring components on OpenShift, only Grafana is deployed. Therefore, only the following metric-monitoring customization files are supported on OpenShift:

- `$USER_DIR/user.env`
- `$USER_DIR/monitoring/user.env`
- `$USER_DIR/monitoring/user-values-openshift-grafana.yaml`

Where `$USER_DIR` is the location of your customization files.

See the [Grafana Helm chart](#) for more information about customizing the deployment. You can provide user settings for Grafana in the `$USER_DIR/monitoring/user-values-openshift-grafana.yaml` file.

No other customizations are required, even if you are using Ingress. The `deploy_monitoring_openshift.sh` script defines a route for Grafana. This route acts as an OpenShift version of Ingress.

Customize Routes on OpenShift

About Route Objects

Route objects is a feature unique to OpenShift. OpenShift uses route objects to access Grafana, OpenSearch Dashboards and (optionally) the OpenSearch API endpoint. This makes it unnecessary to configure Ingress objects or to use NodePorts. No customizations are required, even if you are using Ingress, because the OpenShift deployment scripts define a route for Grafana and OpenSearch Dashboards.

For more information about routes, see the [OpenShift: Route documentation](#).

Host- and Path-Based Routes

Routes can be configured to be host-based (default) or path-based.

- For host-based routes, the name of the application is part of the host name in the URL, for example:
 (for log monitoring) `https://v4m-application-logging.apps.my-openshift-cluster.com`
 (for metric monitoring) `https://v4m-grafana-monitoring.apps.my-openshift-cluster.com`
- For path-based routes, the application name appears only as part of the path at the end of the URL, for example:
 (for log monitoring) `https://v4m-logging.apps.my-openshift-cluster.com/application`
 (for metric monitoring) `https://v4m-monitoring.apps.my-openshift-cluster.com/grafana`

Where *application* is one of the following values:

- (for OpenSearch) `search`
- (for OpenSearch Dashboards) `osd`

The Host Name Used in the Route

You can specify a host name or use the host name that is generated by default.

The log- and metric-monitoring deployment processes can generate a default host name to use in the route. Depending on the route type, the host name is generated in the following way:

- **For host-based routes:**
`v4m-application-$namespace.$OPENSHIFT_ROUTE_DOMAIN`
- **For path-based routes:**
`v4m-$namespace.$OPENSHIFT_ROUTE_DOMAIN`

Where:

- *application* is one of the following values:
 - (for OpenSearch) `search`
 - (for OpenSearch Dashboards) `osd`

- (for Grafana) `grafana`
- `namespace` is the namespace in which the log- or metric-monitoring components are deployed.
- `OPENSIFT_ROUTE_DOMAIN` refers to the environment variable of that name. You can set this variable. If you do not, the log- or metric-monitoring deployment sets the variable based on the route defined for the OpenShift administrator's web console.

You can specify the host name portion of the URL that is used in host- or path-based routes to access OpenSearch, OpenSearch Dashboards, and Grafana. See [“Specify a Host Name” on page 66](#).

IMPORTANT OpenShift does not allow the use of the same route host name across namespaces. Do not use the same custom host name across the log-monitoring and metric-monitoring components.

Specify to Use Path-based Routes

By default, host-based routes are used.

To specify to use path-based routes, complete the following steps:

- 1 Open your preferred `user.env` file:

Component that Uses the Setting	File Location
Log monitoring	<code>\$USER_DIR/logging/user.env</code>
Metric monitoring	<code>\$USER_DIR/monitoring/user.env</code>
Both components	<code>\$USER_DIR/user.env</code>

Where `$USER_DIR` is the location of your customization files.

- 2 Add or update this line:

```
OPENSIFT_PATH_ROUTES=true
```

Specify a Host Name

For details, see [“The Host Name Used in the Route” on page 65](#).

You can specify the host name portion of the URL used to access the following applications:

- OpenSearch
- OpenSearch Dashboards
- Grafana

Complete the following steps:

- 1 Open your preferred `user.env` file:

Component that Uses the Setting	File Location
Log monitoring	<code>\$USER_DIR/logging/user.env</code>
Metric monitoring	<code>\$USER_DIR/monitoring/user.env</code>
Both components	<code>\$USER_DIR/user.env</code>

Where `$USER_DIR` is the location of your customization files.

2 Add or update the applicable lines:

- (for OpenSearch) `OPENSIFT_ROUTE_HOST_ELASTICSEARCH=myhost.example.com`
- (for OpenSearch Dashboards) `OPENSIFT_ROUTE_HOST_KIBANA=myhost.example.com`
- (for Grafana) `OPENSIFT_ROUTE_HOST_GRAFANA=myhost.example.com`

Where *myhost.example.com* is your specified host name.

Note: To maintain compatibility, these options have not been renamed to indicate their support of OpenSearch and OpenSearch Dashboards.

Enable Direct Access to OpenSearch on OpenShift (Optional)

About the OpenSearch API

The OpenSearch API allows users to access the following:

- log messages that are collected by SAS Viya Monitoring for Kubernetes
- OpenSearch configuration and its administrative controls

By default, external access to the OpenSearch API endpoint is disabled. This provides a more secure deployment by preventing direct external access to OpenSearch and requires users to use the OpenSearch Dashboards web application to access the log messages that are stored in OpenSearch. However, you can enable direct access to the OpenSearch API endpoint. If you have developed scripts or other tools that require direct access, you must enable access to the OpenSearch API endpoint.

To enable direct access during the deployment process, see [“Enable Access to the OpenSearch API Endpoint” on page 67](#). If you need to enable access after deployment, see [“Configure Access to OpenSearch \(OpenShift Only\)” on page 91](#).

For more information about the OpenSearch API, see [OpenSearch API documentation](#).

Enable Access to the OpenSearch API Endpoint

Complete these steps:

- 1 Open the `$USER_DIR/logging/user.env` file where `$USER_DIR` is the location of your customization files.
- 2 Add or update this line:

```
OPENSIFT_ES_ROUTE_ENABLE=true
```

Transition from Deploying SAS Viya Monitoring through the DaC Project

The [SAS Viya 4 Deployment repository on GitHub](#) (DaC) project included functionality to deploy SAS Viya Monitoring for Kubernetes in conjunction with its primary focus of deploying SAS Viya 4 software. The DaC removed this option starting at release 8.1.0 - April 7, 2025.

The DaC project provided capabilities to automatically configure Ingress and configure StorageClass references for PVCs that are necessary for some components of SAS Viya Monitoring for Kubernetes. However, you can continue to automatically deploy these customizations. For information about how to do this, see:

- [“Automatically Generate Ingress Definitions and Kubernetes Secrets” on page 26](#)
- [“Automatically Generate Storage Class References” on page 41](#)

The following table shows DaC configuration variables and the corresponding SAS Viya Monitoring for Kubernetes environment variables that provide similar functionality. In most cases, the DaC processing sets the values of these environment variables based the DaC configuration variable value.

Table 9 *DaC Configuration Variables And SAS Viya Monitoring for Kubernetes Environment Variables*

DaC Configuration Variable	SAS Viya Monitoring for Kubernetes Environment Variable	Notes
Ingress object definition		
V4M_ROUTING	ROUTING	The type of Ingress routing to create. Note that the environment variable values are different than the values used by the DaC Config Variables: ■ host versus host-based ■ path versus path-based
V4M_BASE_DOMAIN	BASE_DOMAIN	The host domain that is used in constructing

DaC Configuration Variable	SAS Viya Monitoring for Kubernetes Environment Variable	Notes
		URLs to web applications on this cluster.
V4M_CERT	INGRESS_CERT	The TLS certificate file to use for Ingress.
V4M_KEY	INGRESS_KEY	The TLS key file to use for Ingress.
V4M_ALERTMANAGER_FQDN	ALERTMANAGER_FQDN	The URL for accessing the Alertmanager web application.
V4M_ALERTMANAGER_CERT	ALERTMANAGER_INGRESS_CERT	The TLS certificate file to use for Alertmanager Ingress.
V4M_ALERTMANAGER_KEY	ALERTMANAGER_INGRESS_KEY	The TLS key file to use for Alertmanager Ingress.
V4M_ELASTICSEARCH_FQDN	OPENSEARCH_FQDN	The URL for accessing the OpenSearch API endpoint.
V4M_ELASTICSEARCH_CERT	OPENSEARCH_INGRESS_CERT	The TLS certificate file to use for OpenSearch Ingress.
V4M_ELASTICSEARCH_KEY	OPENSEARCH_INGRESS_KEY	The TLS key file to use for OpenSearch Ingress.
V4M_GRAFANA_FQDN	GRAFANA_FQDN	The URL for accessing Grafana web application.
V4M_GRAFANA_CERT	GRAFANA_INGRESS_CERT	The TLS certificate file to use for Grafana Ingress.
V4M_GRAFANA_KEY	GRAFANA_INGRESS_KEY	The TLS key file to use for Grafana Ingress.
V4M_KIBANA_FQDN	OSD_FQDN	The URL for accessing the OpenSearch

DaC Configuration Variable	SAS Viya Monitoring for Kubernetes Environment Variable	Notes
		Dashboards web application.
V4M_KIBANA_CERT	OSD_INGRESS_CERT	The TLS certificate file to use for OpenSearch Dashboards Ingress.
V4M_KIBANA_KEY	OSD_INGRESS_KEY	The TLS key file to use for OpenSearch Dashboards Ingress.
V4M_PROMETHEUS_FQDN	PROMETHEUS_FQDN	The URL for accessing the Prometheus web application.
V4M_PROMETHEUS_CERT	PROMETHEUS_INGRESS_CERT	The TLS certificate file to use for Prometheus Ingress.
V4M_PROMETHEUS_KEY	PROMETHEUS_INGRESS_KEY	The TLS key file to use for Prometheus Ingress.
StorageClass Specification		
V4M_STORAGECLASS	STORAGECLASS	The Kubernetes storageClass to use for PVCs. Note that DaC did NOT support specifying the storageClass for individual PVCs and did not handle the Pushgateway PVC.
Password		
V4M_GRAFANA_PASSWORD	GRAFANA_ADMIN_PASSWORD	The password for the Grafana admin (superuser) user.
V4M_KIBANA_LOGADM_PASSWORD	LOG_LOGADM_PASSWD	The password for the logadm OpenSearch user.
V4M_KIBANA_PASSWORD	LOG_ADMIN_PASSWD	The password for the OpenSearch/

DaC Configuration Variable	SAS Viya Monitoring for Kubernetes Environment Variable	Notes
		OpenSearch Passwords admin (superuser) user.
V4M_KIBANASERVER_PASSWORD	ES_KIBANASERVER_PASSWD	It is strongly recommended that you leave this unset and allow the deployment process to generate a random password.
V4M_LOGCOLLECTOR_PASSWORD	ES_LOGCOLLECTOR_PASSWD	It is strongly recommended that you leave this unset and allow the deployment process to generate a random password.
V4M_METRICGETTER_PASSWORD	ES_METRICGETTER_PASSWD	It is strongly recommended that you leave this unset and allow the deployment process to generate a random password.
Miscellaneous		
V4M_NODE_PLACEMENT_ENABLE	NODE_PLACEMENT_ENABLE	
V4_CFG_TLS_GENERATOR	CERT_GENERATOR	Valid values are openssl and cert-manager.
V4M_LOGGING_NAMESPACE	LOG_NS	The namespace in which to deploy log monitoring components.
V4M_MONITORING_NAMESPACE	MON_NS	The namespace in which to deploy metric monitoring components.
V4M_VERSION	<i>none</i>	Users can use git checkout command to select which version of SAS Viya Monitoring for Kubernetes to deploy. In all cases, users should

DaC Configuration Variable	SAS Viya Monitoring for Kubernetes Environment Variable	Notes
		deploy from the stable version.
V4M_CUSTOM_CONFIG_USER_DIR	USER_DIR	

Deploy

Deploy SAS Viya Monitoring for Kubernetes

Note: You can run the SAS Viya Monitoring for Kubernetes deployment scripts before or after you deploy the SAS Viya platform.

Deploy the Log-Monitoring Components

Complete the following steps:

- 1 Create a local copy of the repository. See [“Create the Deployment Directory” on page 18](#).
- 2 Prepare your [customizations on page 24](#).
- 3 Navigate to the root directory of your local copy of the repository. Enter the following command to deploy the SAS Viya log-monitoring components:

```
logging/bin/deploy_logging.sh
```

The script creates the namespace into which the components are deployed. By default, the components are deployed into the namespace `logging`.

Note: The script does not create the namespace for air-gapped environments. The air-gapped environment configuration created the namespace already.

- 4 When the deployment completes without errors, a message appears in the console window. This message contains the following information:
 - the web address for OpenSearch Dashboards

Note: The URL for OpenSearch Dashboards differs depending on whether you use NodePorts or an ingress controller to access OpenSearch Dashboards.

Note: If your networking rules alter how Kubernetes services are accessed, the displayed URL for OpenSearch Dashboards might not be correct. If this is the case, contact your Kubernetes administrator to determine the correct information to access OpenSearch Dashboards. The information might be the host, the port, the path, or a combination of two or more of these items.

- the web address for OpenSearch if you enabled access to OpenSearch
-

Note: You can use the `show_app_url.sh` script at any time to print the URLs for SAS Viya Monitoring for Kubernetes web applications. For more information, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

- 5 Access OpenSearch Dashboards and validate that the deployment was successful and confirm that the log-monitoring components are working. For more information, see [“Use OpenSearch Dashboards to Validate Logging” on page 79](#).
-

Note: The OpenSearch admin user is `admin`. If you did not set the `ES_ADMIN_PASSWD` environment variable to specify a password during deployment, (see [“Set Administrator User Passwords” on page 55](#)), the password for the OpenSearch `admin` user is randomly generated and displayed during deployment. To change the password after deployment, see [“Change Administrator Passwords after Deployment” on page 82](#).

Deploy the Metric-Monitoring Components

Complete the following steps:

- 1 Create a local copy of the repository. See [“Create the Deployment Directory” on page 18](#).
- 2 Prepare your [customizations on page 24](#).
- 3 Navigate to the root directory of your local copy of the repository. Enter the following command to deploy the metric-monitoring components:

```
monitoring/bin/deploy_monitoring_cluster.sh
```

The script creates the namespace into which the components are deployed. By default, the components are deployed into the namespace `monitoring`.

Note: The script does not create the namespace for air-gapped environments. The air-gapped environment configuration created the namespace already.

- 4 Enable monitoring of SAS Viya components.

IMPORTANT Typically, this step is done after deploying the SAS Viya platform. If you perform this step before deploying the SAS Viya platform, you will see only Kubernetes-related metrics until you deploy the SAS Viya platform. Also, you must create the namespace to which `VIYA_NS` points.

```
VIYA_NS=Viya_namespace monitoring/bin/deploy_monitoring_viya.sh
```

Where *Viya_namespace* is the namespace in which the SAS Viya platform is deployed.

Note:

- Instead of entering the `VIYA_NS=Viya_namespace` statement on the command line, you can add the statement to your `user.env` file.
 - If you have multiple namespaces that contain SAS Viya, repeat this command for each namespace.
-

- 5 If you have chosen to use a single external instance of Alertmanager, deploy the `alertmanager-endpoint.yaml` file. For the deployment procedure, see [“Deploy the External Alertmanager Configuration” on page 77](#).
- 6 When the deployment completes without errors, a message appears in the console window that contains the URL address for Grafana. This URL differs depending on whether you use ingress or NodePorts to access Grafana. (NodePorts are used by default.)

If you deployed using NodePorts and you enabled Prometheus and Alertmanager, these web applications are available at the following locations by default:

- Prometheus: Port 31090, for example, `http://master-node.yourcluster.example.com:31090`
- Alertmanager: Port 31091, for example, `http://master-node.yourcluster.example.com:31091`

NodePorts for these web applications are disabled by default. For more information, see [“Configure Access to Prometheus and Alertmanager via NodePorts” on page 36](#).

Note: You can use the `show_app_url.sh` script at any time to print the URLs for SAS Viya Monitoring for Kubernetes web applications. For more information, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

Note: The Grafana admin user is `admin`. If you did not set the `GRAFANA_ADMIN_PASSWORD` environment variable to specify a password during deployment, (see [“Set Administrator User Passwords” on page 55](#)), the default password is randomly generated and displayed during deployment. To change the password after deployment, see [“Change Administrator Passwords after Deployment” on page 82](#).

Deploy SAS Viya Monitoring for Kubernetes on Red Hat OpenShift

Note: You can run the SAS Viya Monitoring for Kubernetes deployment scripts before or after you deploy the SAS Viya platform.

Deploy the Log-Monitoring Components

Deploy the Log-Monitoring Components

Complete the following steps:

- 1 Create a local copy of the repository. See [“Create the Deployment Directory” on page 18](#).
- 2 Prepare your [customizations on page 24](#) and [customizations that are specific to OpenShift on page 62](#).

- 3 To log on to the cluster, enter the following command:

```
oc login cluster-hostname -u userID
```

Where:

- *cluster-hostname* is the host name of the cluster on which you want to deploy the components.
- *userID* is your user ID on the cluster.

- 4 Navigate to the root directory of your local copy of the repository. Enter the following command to deploy the SAS Viya log-monitoring components:

```
logging/bin/deploy_logging_openshift.sh
```

The script creates the namespace into which the components are deployed. By default, the components are deployed into the namespace `logging`.

Note: The script does not create the namespace for air-gapped environments. The air-gapped environment configuration created the namespace already.

- 5 When the deployment completes without errors, a message appears in the console window. This message contains the following information:

- the web address for OpenSearch Dashboards

Note: If your networking rules alter how Kubernetes services are accessed, the displayed URL for OpenSearch Dashboards might not be correct. If this is the case, contact your Kubernetes administrator to determine the correct information to access OpenSearch Dashboards. The information might be the host, the port, the path, or a combination of two or more of these items.

- the web address for OpenSearch if you enabled access to OpenSearch

Note: You can use the `show_app_url.sh` script at any time to print the URLs for SAS Viya Monitoring for Kubernetes web applications. For more information, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

Deploy the Metric-Monitoring Components

Deploy the Metric-Monitoring Components

Complete the following steps:

- 1 Create a local copy of the repository. See [“Create the Deployment Directory” on page 18](#).
- 2 Prepare your [customizations on page 24](#) and [customizations that are specific to OpenShift on page 62](#).
- 3 Navigate to the root directory of your local copy of the repository. Enter the following command to deploy the metric-monitoring components:

```
monitoring/bin/deploy_monitoring_openshift.sh
```

Note: The Grafana admin user is `admin` by default. If you did not set the `GRAFANA_ADMIN_PASSWORD` environment variable to specify a password during deployment, (see [“Set Administrator User Passwords” on page 55](#)), the default password is randomly generated and displayed during deployment. To change the password after deployment, see [“Change Administrator Passwords after Deployment” on page 82](#).

The script creates the namespace into which the components are deployed. By default, the components are deployed into the namespace `monitoring`.

Note: The script does not create the namespace for air-gapped environments. The air-gapped environment configuration created the namespace already.

Note: The existing Prometheus configuration is updated to start monitoring any user namespaces, including any SAS Viya namespaces.

- 4 Enable monitoring of SAS Viya components.

IMPORTANT Typically, this step is done after deploying the SAS Viya platform. If you perform this step before deploying the SAS Viya platform, you will see only Kubernetes-related metrics until you deploy the SAS Viya platform. Also, you must create the namespace to which `VIYA_NS` points.

```
VIYA_NS=Viya_namespace monitoring/bin/deploy_monitoring_viya.sh
```

Where `Viya_namespace` is the namespace in which the SAS Viya platform is deployed.

Note:

- Instead of entering the `VIYA_NS=Viya_namespace` statement on the command line, you can add the statement to your `user.env` file.

- If you have multiple namespaces that contain SAS Viya, repeat this command for each namespace.

- 5 When the deployment completes without errors, a message appears in the console window that contains the URL address for Grafana.

Note: You can use the `show_app_url.sh` script at any time to print the URLs for SAS Viya Monitoring for Kubernetes web applications. For more information, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

Deploy the External Alertmanager Configuration

Note: If you have chosen to use a single external instance of Alertmanager, you must complete the configuration after the deployment of the metric-monitoring components. For the pre-deployment steps, see [“Configure Alertmanager” on page 61](#).

After the metric-monitoring components are deployed, you must deploy the `alertmanager-endpoint.yaml` file to the metric-monitoring namespace.

Enter the following command:

```
kubectl apply -n namespace -f $USER_DIR/alertmanager-endpoint.yaml
```

Where *namespace* is the name of the metric-monitoring components namespace. By default, this value is `monitoring`. If you specified a different value when you deployed the metric-monitoring components, use that value.

Post-Deployment

Access Log-Monitoring and Metric-Monitoring Web Applications

When the log-monitoring and metric-monitoring deployment processes complete without errors, a message appears in the console window that displays the URLs to access web applications. You can also use a script to print out a list of URLs for accessing SAS Viya Monitoring for Kubernetes web applications.

The script is located in the top level `bin` directory of your local copy of the repository:

```
/bin/show_app_url.sh parameters
```

If you run the script without parameters, the URLs to access the following applications are returned:

- Grafana
- OpenSearch Dashboards
- OpenSearch

You can use the following parameters to identify specific applications:

- OPENSEARCHDASHBOARDS or OSD
- OPENSEARCH or OS
- GRAFANA or GR
- PROMETHEUS or PR
- ALERTMANAGER or AM
- ALL

This parameter returns the URLs for Grafana, OpenSearch Dashboards, and OpenSearch. Using `ALL` produces the same output as when you have no parameters on the script.

Use a space as a delimiter for more than one parameter.

Notes:

Direct access to OpenSearch, Prometheus and Alertmanager is not configured by default.

The script lists the URLs regardless of the method that you configured to access web applications: Contour or NGINX ingress controllers, NodePorts, or Routes (for OpenShift).

The script returns a message if the proper URL cannot be identified. This can occur if there are problems in the underlying ingress configuration, or if you are using an unsupported ingress controller.

If access to a web application is configured using Contour and the Contour HTTPProxy resource reports errors, the script displays additional information to help debug the issue.

Enable Access to Grafana Dashboards

About Grafana Dashboards

Grafana uses dashboards to visualize the metric data that is collected by Prometheus. The SAS Viya Monitoring for Kubernetes solution includes a set of Grafana dashboards from a variety of sources. For descriptions of available dashboards, see [“Dashboards for Kubernetes Resources” on page 134](#).

Use Permissions to Give Access to Grafana Dashboards

When you deploy the SAS Viya Monitoring for Kubernetes solution, all of the SAS Viya and Kubernetes dashboards for Grafana are available to the `admin` user. When adding users, the administrator can choose which dashboards each user can access.

For more information, see [Grafana: Manage Dashboard Permissions](#).

Use OpenSearch Dashboards to Validate Logging

To validate that the deployment was successful and confirm that all of the log-monitoring components are working, access OpenSearch Dashboards and review the log messages that are collected.

TIP When the deployment is working, you will see log messages with timestamps very close to the current time.

Complete these steps:

- 1 Start OpenSearch Dashboards. Enter the URL provided at the end of the deployment process into a web browser.
- 2 Enter your credentials.
- 3 Click  and then click **Dashboards**. The Dashboards application opens.
- 4 From the list of dashboards, click the **Log Message Volumes with Level** link (not the check box).
The Log Message Volumes with Level dashboard appears. It shows the log message volumes over time broken down by the source of the log messages.
- 5 To filter log messages, scroll down to the “Log Messages” table located below the charts. The newest messages are at the top of the table, based on the current filters. You can change the filters in any of the following ways:
 - Enter text in the **Search** field at the top of the page.
 - Click **Add filter**.
 - Click a log-source entry in the legend next to the “Message Volumes over Time by Source” chart.
- 6 Click  and then click **Discover**. The Discover application opens. The Discover window appears. Use this window to review the collected log messages.
You can use the **Search** field or click **Add filter** to filter the messages that are displayed.

Configure SAS Environment Manager to Access Monitoring

You can use SAS Environment Manager to access the log-monitoring application and the metric-monitoring application that is used by SAS administrators. The links to the applications must first be configured in SAS Environment Manager:

- [Configure SAS Environment Manager to Access Log Monitoring](#)

■ [Configure SAS Environment Manager to Access Metric Monitoring](#)

Note: You must have the URL for each monitoring application. When the log-monitoring and metric-monitoring deployment processes complete without errors, a message appears in the console window that displays the URLs to access web applications. You can also use a script at any time to print out a list of URLs for accessing SAS Viya Monitoring for Kubernetes web applications. For information, see [“Access Log-Monitoring and Metric-Monitoring Web Applications” on page 77](#).

Modify the Deployment

Migrate from Ingress NGINX to Contour

You can migrate from using Ingress NGINX as the ingress controller to using Contour by redeploying SAS Viya Monitoring for Kubernetes after you make adjustments for Contour. The modifications that are required depend on which method you chose to set up Ingress NGINX in your initial deployment.

Migrate from Auto-generated Ingress NGINX to Auto-generated Contour

If you had deployed your SAS Viya Monitoring for Kubernetes solution using [Ingress NGINX autogeneration](#) capabilities, you can migrate to Contour by modifying or adding environment variables to your `$USER_DIR/user.env` file. Run the deployment scripts after you have made the modifications to the files. Here are the steps to follow:

1 Modify or add the following environment variables in your `$USER_DIR/user.env` file:

- Add or modify `INGRESS_TYPE` from `ingress-nginx` to `contour`

```
INGRESS_TYPE=contour
```

The `INGRESS_TYPE` environment variable was introduced with the support of Contour (March, 2026). Add this variable if it is not in your `$USER_DIR/user.env` file.

- Add or modify the following environment variable:

```
USE_SEPERATE_CERTS=true
```

For more information about this environment variable, see *Step 3* of [Automatically Generate Ingress Definitions and Kubernetes Secrets](#).

The `USE_SEPERATE_CERTS` environment variable was introduced with the support of Contour (March, 2026). Add this variable if it is not in your `$USER_DIR/user.env` file.

- You might need to modify the `BASE_DOMAIN` environment variable to use the correct *host* in web application URLs. For example, the host portion of the URLs might need to change from `ingress-nginx.myserver.example.com` to `contour.myserver.example.com`.

`BASE_DOMAIN=host_domain_for_application_URLs`

- If you explicitly defined the fully qualified domain name for any of the web applications through the corresponding environment variables, for example, `GRAFANA_FQDN` and `OSD_FQDN`, adjust these values:

`GRAFANA_FQDN=fully_qualified_hostname`

`OSD_FQDN=fully_qualified_hostname`

This was an optional [step on page 28 in Automatically Generate Ingress Definitions and Kubernetes Secrets](#).

- 2 Run the deployment scripts to remove the existing Ingress NGINX resources and generate the Contour resources. For the deployment scripts to use, see [“Deploy SAS Viya Monitoring for Kubernetes” on page 72](#).
- 3 Verify that there are no Ingress NGINX resources in your deployment. See [“Verify Ingress NGINX Resources Are Removed After Migration to Contour” on page 82](#).

Migrate from Manually Configured Ingress NGINX to Auto-generated Contour

If you manually defined the ingress configuration for SAS Viya Monitoring for Kubernetes using Ingress NGINX, the project's developers recommend that you migrate to Contour using the auto-generation capabilities. This approach is easier than manually configuring the Contour ingress controller and reduces maintenance.

Follow these steps:

- 1 Kubernetes Ingress resources are not used with Contour. Remove any Ingress NGINX settings from your `$USER_DIR` directory. (These are `.yaml` files that are used to set Helm configuration values and consumed by Helm as part of its Helm chart processing.) Some examples are: `$USER_DIR/logging/user-values-opensearch.yaml`, `$USER_DIR/logging/user-values-osd.yaml`, and `$USER_DIR/monitoring/user-values-prom-operator.yaml`. If ingress-related customizations are the only content that is included in these user-values `.yaml` files, you can remove these files.
- 2 Use the steps that are listed in [“Automatically Generate Ingress Definitions and Kubernetes Secrets” on page 26](#).
- 3 Run the deployment scripts to remove the existing Ingress NGINX resources and generate the Contour resources. For the deployment scripts to use, see [“Deploy SAS Viya Monitoring for Kubernetes” on page 72](#).
- 4 Verify that there are no Ingress NGINX resources in your deployment. See [“Verify Ingress NGINX Resources Are Removed After Migration to Contour” on page 82](#).

Migrate from Manually Configured Ingress NGINX to Manually Configured Contour

If you manually defined the ingress configuration for SAS Viya Monitoring for Kubernetes using Ingress NGINX and you want to manually configure Contour, use the [Contours Sample](#) in the SAS Viya Monitoring for Kubernetes project for guidance.

Review the following key points:

- Kubernetes Ingress resources are not used with Contour. Therefore, you need to disable the creation of these resources. Depending on how you previously configured ingress, this can be done by setting the appropriate `ingress.enable` key to **false**, or by removing any Ingress NGINX settings from your `$USER_DIR` directory. Some examples are: `$USER_DIR/logging/user-values-opensearch.yaml`, `$USER_DIR/logging/user-values-osd.yaml`, and `$USER_DIR/monitoring/user-values-prom-operator.yaml`. If ingress-related customizations are the only content that is included in these user-values .yaml files, you can remove these files.
- Review the sample user-value .yaml files in this sample carefully to ensure you have the correct set of files and that the keys within those files are set properly. There are differences in what is required between Ingress NGINX and Contour technologies.
- After updating the files and settings, run the deployment scripts to remove the existing Ingress NGINX resources and generate the Contour resources. For the deployment scripts to use, see [“Deploy SAS Viya Monitoring for Kubernetes” on page 72](#).

After you deploy, verify that Ingress NGINX resources are removed.

Verify Ingress NGINX Resources Are Removed After Migration to Contour

The migration process typically removes Ingress NGINX resources automatically. Verify that this is true. Otherwise, you need to manually delete these Ingress NGINX resources. For example, there should be no Ingress NGINX resources related to the following components:

- logging: OpenSearch, OpenSearch Dashboards
- monitoring: Grafana, Prometheus or Alertmanager

Change Administrator Passwords after Deployment

Note: You can set Administrator user passwords at deployment time. For more information, see [“Set Administrator User Passwords” on page 55](#).

Change the Grafana Administrator Password

The default Grafana administrator user is `admin`. To change the Grafana admin password, complete the following steps:

- 1 Navigate to the `/local_directory/viya4-monitoring-kubernetes` directory.
- 2 Submit the following command:

```
./monitoring/bin/change_grafana_admin_password.sh --password new_password
```

 Where *new_password* is the new password that you want to specify.

Change Log-Monitoring Passwords

To change a log-monitoring password after deployment, you must use the `change_internal_password.sh` script in the project repository. This script ensures that the password in OpenSearch and the corresponding Kubernetes secret remain synchronized.

Complete the following steps:

- 1 Navigate to the `/local_directory/viya4-monitoring-kubernetes` directory.
- 2 Submit the following command:

```
./logging/bin/change_internal_password.sh user_name [password]
```

Where:

- *user_name* is the user whose password you want to change. This parameter is required. Use one of the following values for *user_name*:
 - ☐ `admin`
 - ☐ `logadm`
- *password* is the new password that you want the user account to use.
 - ☐ When specifying the `logadm` user, you must specify a value for *password*.
 - ☐ When specifying the `admin` user, if you do not specify a value for *password*, a randomly generated string is used as the password. That new password is displayed in a message.

Note: A small number of internal users and credentials are defined in the `internal_users.yml` file. These users are required in order to initialize the system and to ensure that the log-monitoring components can communicate with each other. Do not modify these users. For reference, see [“Internal Logging Users” on page 127](#).

Control User Access to Log Monitoring

The logadm User

The deployment process automatically creates the `logadm` user. It is intended to be the primary account that is used for day-to-day log-monitoring activity. The `logadm` user has access to all of the log messages that are collected by the log-monitoring components, which include the following messages:

- log messages from all SAS Viya namespaces
- log messages from all namespaces that are not SAS Viya including the Kubernetes infrastructure namespaces

Although the `logadm` user can access all of the log messages, `logadm` can access only the `cluster_admins` OpenSearch Dashboards-tenant space.

The access controls are designed to provide `logadm` users with full cluster access to perform the log-monitoring activities needed for their job. Limiting `logadm` users to the `cluster_admins` OpenSearch Dashboards-tenant space helps in the following ways:

- It simplifies the OpenSearch Dashboards user experience.
- It eliminates the potential confusion that could otherwise occur if the user could access OpenSearch Dashboards content in the other OpenSearch Dashboards-tenant spaces.

Note: For information about setting the `logadm` password, see the following:

- During the deployment process: [“Set Administrator User Passwords” on page 55](#)
 - Post deployment: [“Change Administrator Passwords after Deployment” on page 82](#)
-

Control Access to Logs by Using Roles and Role Mapping

Note: As of release 2023.10 of the SAS Viya platform, application multi-tenancy, which comprises multiple tenants in the same SAS Viya platform deployment, is no longer supported. Customers who support use cases that require separate tenants are encouraged to deploy the SAS Viya platform into more than one namespace on the same Kubernetes cluster.

About Roles and Role Mapping

The OpenSearch security plug-in uses roles to control access to the logging information from a cluster, an index, or a namespace. A role contains permissions for actions (such as cluster access and index access) that correspond to sources and types of log information. You can assign users to roles in order to grant them the permissions that are defined in the role.

A *back-end role* enables you to point to multiple roles. The security plug-in links roles and back-end roles through *role mappings*. Role mappings can also be used to define links between users and roles and between users and back-end roles. By assigning a single back-end role to a user, the user can be assigned to multiple roles at once.

By default, the log-monitoring capability in SAS Viya Monitoring for Kubernetes enables an administrator with an OpenSearch account to view log information about everything in your Kubernetes cluster.

However, there might be instances where you want to restrict the information that certain administrators can access. If you have multiple SAS Viya deployments in your cluster (for example, to support Development, Test, and Production instances), you can limit an administrator's access so that they can access the log information only for a specified namespace.

This access is controlled by a combination of OpenSearch Dashboards namespace -tenant spaces and OpenSearch roles, back-end roles, and role mappings. Scripts are provided to create everything that is needed for each scenario. For details, see [“Manage Access Controls for a Namespace” on page 87](#).

How Roles and Role Mappings Control Access for the logadm User

The following items are created during the deployment process:

- a set of role-based access controls (RBACs)
- the logadm user, which is linked to the set of RBACs

The following table shows the roles that are linked to the back-end role. The table describes the access granted to the RBAC roles by the OpenSearch security plug-in. When you link users to the back-end role `V4MCLUSTER_ADMIN_kibana_users` via the security plug-in, those users are granted the access described in the table.

Table 10 *The RBAC Roles Linked to the V4MCLUSTER_ADMIN_kibana_users Back-End Role and Their Access*

Role	Access
v4m_kibana_user	Grants user access to the required OpenSearch Dashboards Functionality. (This role is not unique to the logadm user).
search_index	Grants access to all collected log messages.
tenant_cluster_admins	Grants access to the cluster_admins OpenSearch Dashboards-tenant space.

Use Roles and Role Mappings to Control Access in a Namespace

Note: As of release 2023.10 of the SAS Viya platform, application multi-tenancy, which comprises multiple tenants in the same SAS Viya platform deployment, is no longer supported. Customers who support use cases that require separate tenants are encouraged to deploy the SAS Viya platform into more than one namespace on the same Kubernetes cluster.

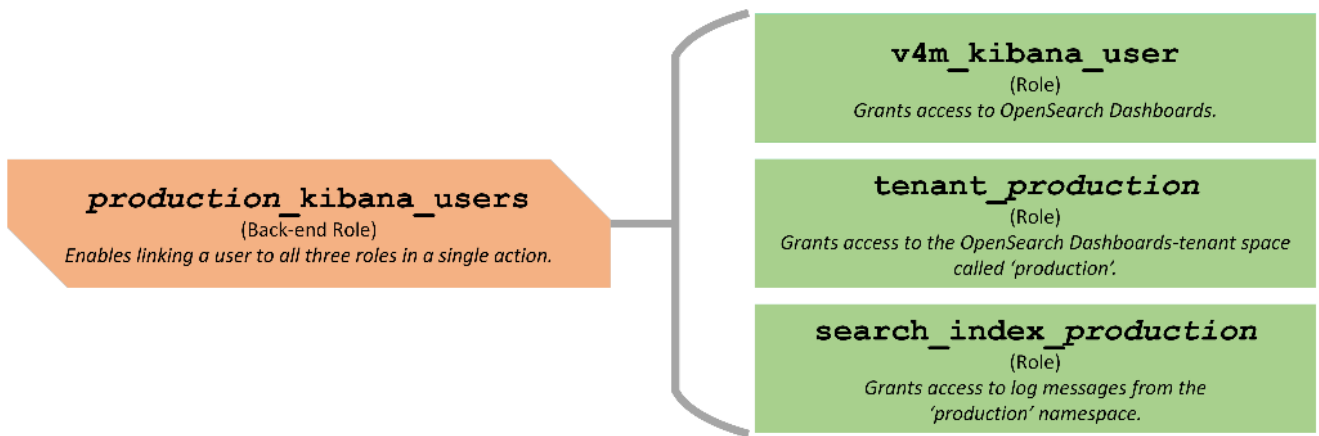
You can use roles and role mappings to control access for other users also. You can ensure that a user can access log messages for a specific namespace only. For example, if you are the provider administrator in a SAS Viya multi-namespace environment, you can implement access controls so that the administrators of each SAS Viya platform namespace can view only the log messages associated with the namespace and access only the OpenSearch Dashboards resources for the namespace.

Namespace Example

For example, consider a SAS Viya instance in a namespace called `production`. You can create a new back-end role with the following command:

```
onboard.sh -ns production
```

The back-end role is called `production_kibana_users`. The following figure shows the roles that are linked to the `production_kibana_users` back-end role.



The following table shows the roles that are linked to the `production_kibana_users` back-end role. The table also shows the access granted to the RBAC roles by the security plug-in. The back-end role is granted access to all of the RBAC roles.

Table 11 The RBAC Roles Linked to the `production_kibana_users` Back-End Role and Their Access

Role	Access
<code>v4m_kibana_user</code>	Grants access to OpenSearch Dashboards
<code>tenant_production</code>	Grants access to the production OpenSearch Dashboards-tenant space
<code>search_index_production</code>	Grants access to log messages from the production namespace

After these access controls have been defined, you can assign the back-end role of `production_kibana_users` to a user. This back-end role enables the user to access only:

- log messages collected from the pods associated with the SAS Viya platform instance in the production namespace

■ OpenSearch Dashboards-tenant space *production*

In some cases, you might want to create users who cannot log in to OpenSearch Dashboards but can access the log messages collected from a specific namespace. For example, you might need these users to enable an automated process to extract log messages through API calls. To do this, assign the `search_index_production` role to the user, rather than the `production_kibana_users` back-end role. Because the user has not been assigned the `v4m_kibana_user` or `tenant_production` roles, the user cannot log in to OpenSearch Dashboards and cannot access the production OpenSearch Dashboards-tenant space.

Manage Access Controls for a Namespace

Note: As of release 2023.10 of the SAS Viya platform, application multi-tenancy, which comprises multiple tenants in the same SAS Viya platform deployment, is no longer supported. Customers who support use cases that require separate tenants are encouraged to deploy the SAS Viya platform into more than one namespace on the same Kubernetes cluster.

Create Access Controls for a Namespace

The `onboard.sh` script implements logging access for a namespace by using the following process:

- 1 Loads OpenSearch Dashboards content such as visualizations and dashboards into the OpenSearch Dashboards namespace.
- 2 Creates the access controls to limit access to only the appropriate back-end data.
- 3 Links the access controls to the OpenSearch Dashboards namespace.
- 4 (Optional) Creates a new user that can access only the information in this OpenSearch Dashboards namespace. .

Use the `onboard.sh` script from the `logging/bin` subdirectory in the repository to implement logging access controls for a namespace.

Here is the syntax for the script:

```
logging/bin/onboard.sh --namespace namespace [--user] [user_name] [--password password]
```

Where:

- *namespace* specifies the Kubernetes namespace to which access limitations are applied. This parameter is required.
- *user* specifies to create an initial user with access to the OpenSearch Dashboards namespace.
- (optional) *user_name* specifies a user name for the account.
- *password* specifies the password for the account.

Note: As of release 1.2.15 (July 2023), OpenSearch evaluates passwords for complexity. If you specify a password that does not meet the complexity standard, the script generates a password that does. The new password is displayed as output. To change the password, use the OpenSearch Dashboards web application or the OpenSearch API.

Remove Access for a Namespace

The `offboard.sh` script removes logging access for a namespace by using the following process:

- 1 Removes the OpenSearch Dashboards namespace.
- 2 Removes the access controls for the OpenSearch Dashboards namespace.

Use the `offboard.sh` script from the `logging/bin` subdirectory in the repository to remove logging access controls for a namespace.

Note: The `offboard.sh` script does not remove the initial user, if one was created by the `onboard.sh` script. To do so, see [“Delete a User Account” on page 89](#).

Here is the syntax for the script:

```
logging/bin/offboard.sh --namespace namespace
```

Where:

- *namespace* specifies the Kubernetes namespace for which the corresponding OpenSearch Dashboards namespace and access controls are removed. This parameter is required.

Create a User Account

Note: Perform this task after you have run the `onboard.sh` script to implement access controls for the OpenSearch Dashboards. The user accounts that you create are bound by these access controls.

Here is the syntax for the script to add a user account:

```
logging/bin/user.sh CREATE --namespace namespace [--user] [user_name] [--password password]
```

Where:

- *namespace* specifies the Kubernetes namespace which the user can access. This parameter is required.
- *user_name* specifies a user name for the user account.
- *password* specifies the password for the account.

Note: As of release 1.2.15 (July 2023), OpenSearch evaluates passwords for complexity. If you specify a password that does not meet the complexity standard, the script generates a password that does. The new password is displayed as output. To change the password, use the OpenSearch Dashboards web application or the OpenSearch API.

You create new users and link them to the access controls by running the `user.sh` script. For example, to create a user called “chen” and link the user to the access controls, enter the following command:

```
logging/bin/user.sh CREATE --user chen --password chenspassword --namespace mynamespace
```

Grant logadm-level Access to Another User

You can grant a user the same level of access as a `logadm` user. In your repository directory, enter the following command:

```
logging/bin/user.sh CREATE --user user_name --password password --namespace _all_
```

Where:

- *user_name* is the user to whom you want to grant the same level of access as a `logadm` user.
- *password* is the password associated with *user_name*.

Delete a User Account

Note: Perform this task after you have run the `onboard.sh` script to implement access controls for the OpenSearch Dashboards namespace.

Here is the syntax for the script to delete a user account:

```
logging/bin/user.sh DELETE --user user_name
```

Where *user_name* specifies the user account to delete.

Configure Access to Web Applications via NodePorts

About Access via NodePorts

You can configure access via NodePorts for the following logging and monitoring applications:

- OpenSearch
- OpenSearch Dashboards
- Grafana
- Prometheus
- Alertmanager

While you can configure access via NodePorts, this type of configuration is typically used only in test environments.

IMPORTANT While it is possible to make Prometheus and Alertmanager web applications accessible, you might not want to because these web applications do not include built-in user authentication mechanisms. Therefore, making them accessible without implementing an authentication mechanism is a security risk.

Note: You can configure access via NodePorts before you deploy SAS Viya Monitoring for Kubernetes by using environment variables. For more information see the following pre-deployment customizations:

- [“Configure Access to OpenSearch Dashboards via NodePorts” on page 36](#)
- [“Configure Access to Prometheus and Alertmanager via NodePorts” on page 36](#)

Enable Access via NodePorts

IMPORTANT This script cannot be used on OpenShift. See [“Configure Access to OpenSearch \(OpenShift Only\)” on page 91](#) and [“Customize Routes on OpenShift” on page 65](#).

Complete these steps:

- 1 Make sure you are in the directory that contains the copy of the repository.
- 2 To enable a NodePort connection, enter the following command:

```
bin/configure_nodeport.sh application [port_number]
```

Where:

- *application* is the application for which you want to configure NodePort accessibility. It can be one of the following case-insensitive values:
 - ☐ opensearch (or the shortcut value *os*)
 - ☐ opensearchdashboards (or the shortcut value *osd*)
 - ☐ grafana
 - ☐ prometheus
 - ☐ alertmanager
- *port_number* is an optional argument that can be one of the following values:
 - ☐ 0 (zero) indicates to use a random port for the NodePort.
 - ☐ a number between 30000 – 32767 that requests a specific port number to use for the NodePort.

Note: An error message is generated for the following situations:

- You specify a value outside of the number range.
- You specify a port number that is already in use.

Note: If you do not specify a port number when configuring OpenSearch, the script assigns a random port number.

When the command completes, a message such as this appears in the console window:

****Accessing the monitoring applications****

You can access OpenSearch Dashboards via the following URL:

`http://myk8snode:port_number`

Note: The URL might be incorrect if your Ingress configuration, another network configuration, or both include options that this script does not process.

Where:

- **myk8snode** is the host name of a Kubernetes node in your deployment.
- **port_number** is the port assigned to the application in your deployment. In this example, the message provides the URL for the OpenSearch Dashboards application.

Disable Access via NodePorts

Complete these steps to disable access via NodePorts:

- 1 Make sure that you are in the directory that contains the copy of the repository.
- 2 To disable a NodePort connection, enter the following command:

```
bin/configure_nodeport.sh application disable
```

Where *application* is the application with the NodePort that you want to disable. It can be one of the following case-insensitive values:

- `opensearch` (or the shortcut value `os`)
- `opensearchdashboards` (or the shortcut value `osd`)
- `grafana`
- `prometheus`
- `alertmanager`

Configure Access to OpenSearch (OpenShift Only)

About Access to OpenSearch

IMPORTANT These procedures cannot be used on standard deployments. Instead, see [“Configure Access to Web Applications via NodePorts” on page 89](#).

You can enable users to access the OpenSearch application so that they can issue queries using API calls or scripts. For more information about the OpenSearch API, see [OpenSearch API documentation](#).

For information about enabling access to the OpenSearch API before deployment, see [“Enable Direct Access to OpenSearch on OpenShift \(Optional\)” on page 67](#).

Enable Access to the OpenSearch API

To enable access after deployment, enter the following command:
`logging/bin/create_openshift_route.sh OPENSEARCH`

Remove Access to the OpenSearch API

To remove access after deployment, enter the following command:
`oc -n $LOG_NS delete route v4m-es`

Limit Access in a Multi-Tenant Environment

Note: As of release 2023.10, application multi-tenancy, which comprises multiple tenants in the same SAS Viya platform deployment, is no longer supported. Customers who support use cases that require separate tenants are encouraged to deploy the SAS Viya platform into more than one namespace on the same Kubernetes cluster.

In a SAS Viya platform multi-tenant environment, you want to ensure that each tenant administrator can access the resources and messages only for their tenant.

If you are the provider administrator in a SAS Viya platform multi-tenant environment, you can limit access in the following ways:

- Each administrator can view the log messages associated only with their tenant.
- Each administrator can access the OpenSearch Dashboards resources only for their tenant.

Use the following scripts to manage the access controls for SAS Viya platform tenants. For detailed information about these scripts, see [“Control User Access to Log Monitoring” on page 84](#).

To implement logging access limitations for a SAS Viya platform tenant.	<code>onboard.sh</code>
To remove logging access for a SAS Viya platform tenant.	<code>offboard.sh</code>
To create or delete OpenSearch accounts that have access to the log messages and OpenSearch Dashboards-tenant space associated only with a specified SAS Viya platform tenant.	<code>user.sh</code>

Adjust the Retention Period for Log Messages

Log-Message Retention

The SAS Viya Monitoring for Kubernetes solution is focused on operational monitoring. That is, helping administrators in their day-to-day work to ensure that SAS Viya is meeting their users' needs. This is especially true with regard to the log message monitoring. Log messages are collected and stored for day-to-day operational purposes with all of the collected log messages made equally available.

This solution is not designed to provide long-term storage or archival of log messages. Log message retention should be thought of in terms of days rather than weeks or months.

Note: Although you can configure the underlying OpenSearch technology to support longer time frames, doing so involves developing more complex policies and storage configurations. Refer to the [OpenSearch documentation](#) for more information.

Log-Message Retention Processing and Index Policies

Messages flow through log-retention processing in the following way:

- An incoming message is loaded into memory and placed in the *hot state*. Messages in this state are available to be searched.
- After the retention period has passed, the message moves to the *doomed state*. Messages in this state are deleted from memory and the back-end data store. Once deleted, the message is no longer available and cannot be included in search results.

Note: If you increase the retention period, you must increase the size of the persistent volume claims (PVCs) associated with the OpenSearch pods. For more information, see [“Increase Storage for Log Messages” on page 96](#).

Log-message retention processing is implemented via index policies. Index policies are a feature of the Index State Management plug-in to OpenSearch. SAS Viya Monitoring for Kubernetes deployments use the index policies that are listed in the following table:

Index Management Policy	Policy Definition	Default Retention Period
<code>viya_logs_idxmgt_policy</code>	The process and retention period for indexes that contain log messages collected from SAS Viya and other Kubernetes pods on the cluster.	3 days
<code>viya_ops_idxmgt_policy</code>	The process and retention period for indexes that contain log messages	1 day

Index Management Policy	Policy Definition	Default Retention Period
	from internal monitoring components (that is, Grafana, OpenSearch, and so on).	
security-auditlog_idxmgmt_policy	The process and retention period for audit indexes that contain audit records for OpenSearch user activity. The naming convention of these log messages is <code>security-auditlog-date</code>. This policy is new starting with 1.2.53 (August, 2026). Although these indexes are typically small, they consume OpenSearch shards, a limited internal resource. To help manage shard usage, this policy automatically deletes audit log indexes after 90 days. You can disable the policy using an environment variable. Note that <code>security-auditlog</code> indexes are not deleted without this index policy. For more information, see “Customize the Retention Period for Log Messages” on page 52.	90 days
For OpenShift deployments: viya_infra_idxmgmt_policy	Deployments on OpenShift have an additional policy to manage log messages from OpenShift infrastructure namespaces, which can generate a large number of log messages. (Any namespace that starts with “openshift” is an OpenShift infrastructure namespace.)	1 day

You can modify these policies or create your own. For more information, see [Index State Management in the OpenSearch documentation](#).

Changing the Log-Message Retention Period

Prior to deploying, you can modify the retention period for index management policies through environment variables. For more information, see [“Customize the Retention Period for Log Messages” on page 52](#).

If you perform an upgrade-in-place, which does not require that you remove the existing deployment, modifying these environment variables has no effect. New values do not replace any existing index management policies. In this situation, modify the retention period through OpenSearch Dashboards.

For more information, see [“Edit the Policy” on page 95](#) and [“Apply an Updated Policy to an Existing Index” on page 96](#).

Note: If you modify retention periods through OpenSearch Dashboards and you are maintaining customized configuration information in `user.env` files, be sure to update the retention period environment variables so that the new values are used in future re-deployments.

OpenSearch performance tuning is very complicated. It is dependent on unique use patterns and activity. To ensure the best performance in your specific environment, you might need to make the following changes at the same time that you are changing the log-message retention period:

- When you increase the retention period, you must also increase storage. For information about estimating storage use, see [“Increase Storage for Log Messages” on page 96](#).
- Update the Java memory settings of the OpenSearch nodes.
- Modify the number of OpenSearch nodes.
- Modify the specific mix of OpenSearch node types.

Policy Updates and Pre-existing Indexes

When you edit a policy, the change is not applied retroactively to the indexes that already exist.

- Newly created indexes use the updated retention policy.
- Existing indexes continue to be retained for the period previously defined in the index retention policy.

If you want the updated policy to apply to existing indexes, you must apply the updated policy to those existing indexes as well. For more information, see [“Apply an Updated Policy to an Existing Index” on page 96](#).

Edit the Policy

To review and edit the index management policies in OpenSearch Dashboards, complete the following steps:

- 1 Click  and then click **Index Management**.
- 2 Select the policy that you want to edit.
- 3 Click **Edit**. The Edit policy window appears.
- 4 In the **Define policy** pane, locate the **min_index_age** value.
- 5 Change the existing value to the new value.
- 6 Click **Update**.

The updated policy is used for any newly created indexes.

Apply an Updated Policy to an Existing Index

When you edit a policy, the change is not applied retroactively to the indexes that already exist. To apply an updated policy to an existing index, complete the following steps:

- 1 Click  and then click **Index Management**.
- 2 Select **Managed Indices** from the subsequent main menu.
- 3 Click **Change Policy**. It is located in the upper right area of the window. The Change policy window appears.
- 4 In the **Managed indices** list box located in the **Choose managed indices** pane, enter: `viya_logs-`.

TIP The interface might suggest `viya_logs-*`. Accept the suggestion by pressing the Enter key.

- 5 Select **viya_logs_idxmgmt_policy** from the **New policy** list box.
- 6 Make sure the **Keep indices in their current state after the policy takes effect** option is selected.
- 7 Click **Change**.

A message appears that indicates how many indexes were updated.

TIP The back-end processing for handling log roll-off runs periodically, and the changes might take effect immediately.

Increase Storage for Log Messages

About Retention and Storage

Although you can modify the length of time that log messages are retained, it is important to understand that doing so can impact the amount of storage needed for OpenSearch.

You must increase the size of the persistent volume claims (PVCs) associated with the OpenSearch pods if you choose to retain log messages for a longer period of time. If you reduce the retention period or increase it by a small amount, such as one or two days, there is no need to modify the PVCs.

For information about the retention period, see [“Adjust the Retention Period for Log Messages” on page 93](#).

For information about adjusting storage before deployment, see [“Customize StorageClass” on page 40](#).

Identify Current Storage Use

There is no simple formula to calculate how a change in the retention period can impact the required storage. However, you can determine how much space currently is being consumed by the OpenSearch indexes (the storage structures used by OpenSearch). Using that information, you can estimate a size increase.

To see the amount of storage consumed by each OpenSearch index, complete the following steps in OpenSearch Dashboards:

- 1 From the main menu, navigate to the Index Management window.
- 2 Select **Indices** from the menu on the left side. A list of all of the indexes currently stored in OpenSearch is displayed. The list might be multiple pages.
- 3 Adjust the number of indexes shown per page by using the control at the bottom of the table.
- 4 In the **Search** field above the list of indexes, enter the filter criteria to view all of the indexes that were created for one day. For example, enter `2022*01*10` to view indexes created on January 10, 2022.
- 5 For each index, the **Total Size** column shows the amount of storage consumed by the index. Add the values in the **Total Size** columns for one day. The sum is the storage required for that day's worth of log messages.

Note: The volume of log messages varies day-by-day based on use. Calculate the amount of storage consumed for a few days before making any assumptions.

Example Estimate

IMPORTANT This example and its sums are relevant only for the example cluster. The sums for your cluster can be different and potentially significantly different.

The Indices window shows the indexes for a single day (January 24, 2022) on a cluster.

Indices

🔍 2022*01*24

☐ Index	Health	Managed by Policy	Status	Total size ↓	Primaries size	Total docume...
☐ viya_logs-production-pit-stable-2...	● green	Yes	open	20.7gb	10.3gb	26202319
☐ viya_logs-production-pit-stable-2...	● green	Yes	open	5.8gb	2.9gb	9368059
☐ viya_logs-ingress-nginx-2022-01...	● green	Yes	open	1013.1mb	505.4mb	1697382
☐ viya_logs-kube-system-2022-01-...	● green	Yes	open	326.1mb	164.3mb	358678
☐ viya_logs-cert-manager-2022-01...	● green	Yes	open	212mb	105.7mb	278783
☐ security-auditlog-2022.01.24	● green	No	open	32.8mb	16.4mb	16626
☐ viya_logs-kubernetes-dashboard...	● green	Yes	open	5mb	2.5mb	12958
☐ viya_logs-missing_ns-2022-01-24	● green	Yes	open	1.6mb	830.1kb	953
☐ viya_logs-default-2022-01-24	● green	Yes	open	1.1mb	584.9kb	1129
☐ viya_logs-monitoring-2022-01-24	● green	Yes	open	301.1kb	136.1kb	121
☐ viya_logs-logging-2022-01-24	● green	Yes	open	220.2kb	110.1kb	33
☐ viya_logs-cadstblctestmanager-2...	● green	Yes	open	183.1kb	91.5kb	37

TIP To navigate to the Indices window, click  and then select **Index Management** ⇨ **Indices**.

The filter criteria (2022*01*24) limits the list to that day. The sum of the **Total Size** columns shows that about 28 GB of log messages were collected on that day. Performing the same exercise for other days shows comparable sums. To store 5 days of logs, you need at least 140 GB (5 days x 28 GB per day) of storage. Then, increasing the estimate by 50 or 100 percent is not unreasonable. The extra margin can allow for logging due to increased user activity or problems.

The current default configuration creates three OpenSearch nodes (that is, the pods responsible for storing the OpenSearch indexes) and configures a 30 GB PVC for each node. This means there is a maximum of 90 GB of storage available. The default configuration also sets the log-retention period to 3 days. Although this default configuration might work for the sample cluster, it will not work when expanding the log-retention period to 5 days. And, even if the current retention period is unchanged, there is no extra margin. Any increase in logging levels might fill up the available space.

Consequences of Failing to Increase Storage

When all available space has been consumed, the OpenSearch cluster fails:

- No new log messages are stored.
- The housekeeping process that purges old log messages does not run.

- The cluster becomes unusable.

Increase the Storage for the OpenSearch Nodes

Use the following information to increase the storage for the existing OpenSearch nodes.

Verify Support for Expansion

Note: To perform these tasks, you must have elevated Kubernetes permissions.

Some Kubernetes storageClass resources do not support expansion. To determine whether the type of storageClass resources used by the OpenSearch data node PVCs supports expansion, enter the following command:

```
kubectl describe storageClass your_storageClass
```

Where *your_storageClass* is the name of the storageClass that is used for the PVCs that you are checking. For example, to describe the default storageClass, enter the following command:

```
kubectl describe storageClass default
```

Review the command output. If the `AllowVolumeExpansion` property is `true`, the expansion of existing PVCs is supported and the following procedure should work.

Increase the Storage

Note: To perform these tasks, you must have elevated Kubernetes permissions.

To increase storage, complete the following steps:

- 1 Scale down the statefulSet that controls the OpenSearch nodes by entering the following command:

```
kubectl -n logging scale statefulset v4m-search --replicas=0
```

Where *logging* is the namespace into which the log-monitoring components have been deployed in your installation.

This command terminates the existing `v4m-search-*` pods.

- 2 Resize the existing PVCs by entering the following command (all on one line):

```
kubectl -n logging patch pvc pvc_name -p '{"spec":{"resources":{"requests":{"storage":"nnGiB"}}}}'
```

Where:

- *logging* is the namespace into which the log-monitoring components have been deployed in your installation.
 - *pvc_name* is the PVC to resize (for example, `v4m-search-v4m-search-0`).
 - *nnGiB* is the amount (for example, `70GiB`) to increase the PVC storage.
- 3 Repeat the command in [Step 2](#) for each of the PVCs associated with the OpenSearch nodes (that is, `v4m-search-v4m-search-0`, `v4m-search-v4m-search-1`, and `v4m-search-v4m-search-2`).

- 4 Scale up the statefulSet that controls the OpenSearch nodes by entering the following command:

```
kubectl -n logging scale statefulset v4m-search --replicas=3
```

Where *logging* is the namespace into which the log-monitoring components have been deployed in your installation.

This command results in the creation of three new `v4m-search-*` pods that are linked to the existing (but now larger) PVCs.

TIP If you are maintaining customized configuration information (that is, using the `$USER_DIR` environment variable), consider updating the contents of the `logging/user-values-opensearch.yaml` file to reflect the larger PVC size. Doing so ensures that your updated configuration is re-created if you redeploy the log-monitoring components. For information about using the `$USER_DIR` environment variable, see [“Set the \\$USER_DIR Environment Variable” on page 20](#).

IMPORTANT This procedure adjusts only the size of the existing PVCs and does not change the PVC specification included in the statefulSet definition. If you scale up the statefulSet to increase the number of OpenSearch nodes beyond the existing three pods, the new pods are linked to PVCs using the original size. At that point, you must repeat this procedure to increase the size of the PVCs linked to the new pod.

Display Log Messages in Grafana Dashboards

Overview

SAS Viya Monitoring for Kubernetes stores log messages in OpenSearch, and these log messages are viewed in OpenSearch Dashboards. However, you can optionally configure your environment to view log messages in Grafana dashboards. Log messages can provide context that helps in interpreting the displayed metrics within the dashboards.

To create the log-enabled versions of the dashboards, use the `create_logging_datasource.sh` script. The script performs the following tasks:

- Deploys the OpenSearch Grafana data source plug-in, which configures the data source that enables Grafana to include log messages that are stored in OpenSearch.
- Enables Grafana to connect to OpenSearch.
- Replaces some SAS Viya dashboards with log-enabled dashboards. The log-enabled versions of the dashboards display the same metrics and visualization as the original dashboards. And these dashboards include log messages that are collected from the selected components during the same time period.

Note: Only some of the Grafana dashboards display log messages after you run this script. However, once you have created the data source for Grafana to include log messages in its dashboards, you

can incorporate log messages into new dashboards that you build in Grafana. See [“Add More Grafana Dashboards” on page 101](#).

Grafana also provides an Explore tool that enables users to review data from a data source, including the OpenSearch datasource.

Run the create_logging_datasource.sh Script

Complete these steps:

- 1 For an air gap environment only, you must have downloaded the OpenSearch Grafana data source plug-in through one of the following methods:

- [“Step 6: Download the OpenSearch Grafana Data Source Plug-in \(Optional\)” on page 49](#).
- Step 3 of [“Use a Script to Populate Your Private Registry” on page 45](#).

Set the `AIRGAP_DEPLOYMENT` environment variable to `true`:

```
AIRGAP_DEPLOYMENT=true
```

You can set it on the command line before you run the script, in the same command as the script, or in the `$USER_DIR/user.env` file.

- 2 Run the following command from the directory that contains the copy of the repository:

```
/monitoring/bin/create_logging_datasource.sh
```

Add More Grafana Dashboards

Download Dashboards

Although the SAS Viya Monitoring for Kubernetes solution includes a set of Grafana dashboards, you can also add your own dashboard. Many dashboards are available at [Grafana's site of community dashboards](#).

To use a dashboard from the Grafana site, you must first download the JSON file for the dashboard and save it using a file name that is a valid Kubernetes resource name. For information about names that you can use, see [Kubernetes: Object Names and IDs](#).

After you save the dashboards that you want to use, choose one of these methods to add them to your deployment:

- [import each dashboard into Grafana manually. on page 102](#)
- [add dashboards using a script. on page 102](#)
- [automatically deploy dashboards stored in the dashboards directory. on page 103](#)

Note: The scripts described in this section act upon only the first organization in your Grafana deployment.

Dashboards That Require a License or Enablement

SAS Workload Orchestrator

The following dashboards report metric data from the SAS Viya platform feature SAS Workload Orchestrator:

- SAS Launched Jobs - User Activity
- SAS Launched Jobs - Node Activity

To use these dashboards, SAS Workload Orchestrator must be enabled in your SAS Viya platform deployment. For details, see [“Disabling or Enabling SAS Workload Orchestrator” in SAS Viya Platform: Workload Management](#).

IMPORTANT

- As of SAS Viya release 2023.08, SAS Workload Orchestrator is enabled, by default, as part of the SAS Viya platform deployment. A license is no longer required.
- Releases 2023.07 and earlier require a license to use SAS Workload Orchestrator.

SAS Micro Analytic Service

The SAS Micro Analytic Service dashboard requires that the SAS Viya platform deployment has a license for SAS Micro Analytic Service. Contact SAS to purchase a license if you require one.

Import Dashboards Manually

You can use Grafana's import function to import dashboards.

The advantage of using the import function is that Grafana recognizes whether the dashboard's data source is not valid for your environment and prompts you to select your environment's data source.

The disadvantage is that the process is not automated, so you must re-import the dashboards each time you redeploy the monitoring components. If you have many dashboards to import, the process could also be time-consuming.

Use a Script to Add Dashboards

You can use the following script to deploy a single dashboard or a directory of dashboards:

```
deploy_dashboards.sh [dashboard | dashboard_directory]
```

The advantage of using the script is that the process can be automated, so you can automatically redeploy all your custom dashboards whenever you redeploy the monitoring components.

Here are key points to remember when using this script:

- Grafana cannot validate the data source of the dashboards that are deployed using this script, so you must either manually change the data source or use Grafana to create a dashboard file with the data source resolved. See [“Resolve Dashboard Data Sources” on page 103](#).
- Some detailed dashboards might be too large to deploy using the script, so must be imported manually. If the dashboard is too large, a message is displayed during deployment.

Deploy Dashboards in the Dashboards Directory

Dashboards that are stored in the `$USER_DIR/monitoring/dashboards` directory are added automatically when you deploy the metric-monitoring components. These dashboards must meet the following criteria:

- be in JSON format
- cannot be in a subdirectory under `$USER_DIR/monitoring/dashboards`

Here are key points to remember when using this script:

- Grafana cannot validate the data source of the dashboards that are deployed using this script, so you must either manually change the data source or use Grafana to create a dashboard file with the data source resolved. See [“Resolve Dashboard Data Sources” on page 103](#).
- If you remove the metric-monitoring components using the `remove_monitoring_cluster.sh` script, the dashboards are removed as well.

Resolve Dashboard Data Sources

Grafana cannot validate the data source of the dashboards that are deployed using the following methods:

- `deploy_dashboards.sh` script
- the `$USER_DIR/monitoring/dashboards` directory

Without a validated data source, the dashboards display incomplete information.

You can specify the data source in either of the following ways:

- manually change the data source in the dashboard's JSON file.
- manually import the dashboard into Grafana. Doing so prompts you to select a data source, and then export the dashboard with the valid data source specified.

To manually change the data source, complete the following steps:

- 1 Edit the dashboard's JSON file.
- 2 Locate the following statement:


```
"datasource": "${DS_PROMETHEUS}"
```
- 3 Replace the original statement with this new statement:

```
"datasource": "Prometheus"
```

To use Grafana's import and export functions to create a JSON file for the dashboard with data sources resolved, complete the following steps:

- 1 Import the dashboard into Grafana and resolve the data source.
- 2 In Grafana, select **Share**, and then select the **Export** tab.
- 3 Ensure that the **Export for sharing externally** option is not selected.
- 4 Select **Save to file** in order to save the JSON file for the dashboard with the data source resolved.
- 5 Use the script or directory to import the dashboard during future deployments.

Additional Metric-Monitoring Customizations

About Alert Notifications

If you use Prometheus and Alertmanager as part of your metric-monitoring strategy, you can use these applications to create an alert-notification strategy. Prometheus evaluates alerts and sends them to Alertmanager, which then sends user-defined notifications.

Although some Prometheus alerts are provided, you can also define your own by creating alerting rules in a PrometheusRule.yaml file. For information about defining alerts, see [Alerting Rules](#) in the Prometheus documentation.

Alertmanager enables you to specify the action that is taken when a defined Prometheus alert is triggered. Notifications are defined in notification templates. The templates specify the following information:

- the receiver of a notification.
- the information that is sent with the notification.

For more information, see the following resources:

- [Notification Template Reference](#) in the Prometheus documentation
- [PrometheusRule](#) in the Prometheus Operator documentation.

About Recording Rules

If you did not deploy SAS Viya Monitoring for Kubernetes but are using your own version of Prometheus Operator, you can apply the rules-sas-launcher-job.yaml file in order to use the rules in your own dashboards and applications.

If you are not using Prometheus Operator, you can use the contents of the rule definitions in `rules-sas-launcher-job.yaml` to create and apply your own rules.

For information about the available recording rules, see [“Recording Rules Provided for SAS Workload Manager” on page 129](#).

About ServiceMonitors

Prometheus uses ServiceMonitors to determine how to discover resources and to read metrics for those resources. ServiceMonitors for SAS Viya resources are provided by SAS and are installed as part of the monitoring infrastructure. For the provided ServiceMonitors, see [“ServiceMonitors Provided for SAS Viya” on page 128](#).

There might be cases where you want to create your own SAS Viya ServiceMonitors. For example, you might want to use different labels. SAS recommends that you use the provided ServiceMonitors as examples for creating your own.

Upgrade in Place

Note: To perform these tasks, you must have elevated Kubernetes permissions.

You can perform an upgrade-in-place for the monitoring components. Note that you do not need to remove your existing deployment to perform an upgrade-in-place.

TIP Also, instead of running the deployment script for a new release, you can run the deployment script for your current release in order to adjust a configuration setting.

Complete these steps:

- 1 Select the release that you want to use for the update. Copy the repository of that release. See [“Copy the Repository” on page 18](#).
- 2 Make sure you are in the directory to which you copied the repository in step 1.
- 3 Set the `$USER_DIR` environment variable to point to the customization files from your initial deployment. See [“Set the \\$USER_DIR Environment Variable” on page 20](#).
- 4 For the log-monitoring deployment, enter the applicable command for your deployment.

For standard deployments	<code>logging/bin/deploy_logging.sh</code>
--------------------------	--

For deployments on OpenShift	<code>logging/bin/deploy_logging_openshift.sh</code>
------------------------------	--

- 5 For the metric-monitoring deployment, enter the applicable command for your deployment.

For standard deployments	<code>monitoring/bin/deploy_monitoring_cluster.sh</code>
--------------------------	--

For deployments on OpenShift	<code>monitoring/bin/deploy_monitoring_openshift.sh</code>
------------------------------	--

- 6 To update the SAS Viya metric-monitoring components, enter the following command:

```
deploy_monitoring_viya.sh
```

When complete, the script displays a success message. The latest versions of all components, indexes, and dashboards are installed.

Remove the Monitoring Components

Objects That the Script Does and Does Not Remove

The removal script removes components, indexes, and dashboards.

By default, the removal script does not remove all of the associated Kubernetes objects. The following objects are not removed:

- The Kubernetes configuration maps (ConfigMaps), secrets, and persistent volume claims (PVC) in the `logging` and `monitoring` namespaces that were not created by the deployment process.

Because these objects were not created by the deployment process, it is presumed that they were created by you or your organization. These objects are not removed in case you want to redeploy the log- and metric-monitoring components without losing data or having to regenerate TLS certificates.

To override the default behavior of the removal script, you can specify the following environment variables:

- ☐ (for log monitoring) `LOG_DELETE_PVCS_ON_REMOVE`
- ☐ (for metric monitoring) `MON_DELETE_PVCS_ON_REMOVE`

When set to `true` and you run the removal script, the PVCs are removed as part of the removal process.

- The `logging` and `monitoring` namespaces.

By default, these objects are not removed in case you deployed the log- or metric-monitoring components into a namespace that also hosts other content. If your instance of the `logging` or `monitoring` namespace does not host other content, you can choose to enable the removal script to remove the namespace.

To override the default behavior of the removal script, you can specify the following environment variables:

- ☐ (for log monitoring) `LOG_DELETE_NAMESPACE_ON_REMOVE`
- ☐ (for metric monitoring) `MON_DELETE_NAMESPACE_ON_REMOVE`

When set to `true` and you run the removal script, the associated namespace is removed as part of the removal process.

Note: To use the environment variables, declare them on the command line or add them to your `user.env` file before you run the removal script. See [“Customize Variables and Parameters” on page 22](#).

Remove the Monitoring Components

Note: To perform these tasks, you must have elevated Kubernetes permissions.

Complete the following steps:

- 1 Make sure you are in the directory where your copy of the repository is located.
- 2 To remove the log-monitoring components, enter the applicable command for your deployment.

For standard deployments	<code>logging/bin/remove_logging.sh</code>
For deployments on OpenShift	<code>logging/bin/remove_logging_openshift.sh</code>

- 3 To remove the cluster metric-monitoring components, enter the applicable command for your deployment.

For standard deployments	<code>monitoring/bin/remove_monitoring_cluster.sh</code>
For deployments on OpenShift	<code>monitoring/bin/remove_monitoring_openshift.sh</code>

- 4 To remove the SAS Viya metric-monitoring components, enter the following command for each namespace that contains SAS Viya:

```
VIYA_NS=<your_viya_namespace> monitoring/bin/remove_monitoring_viya.sh
```

Where *your_viya_namespace* is the namespace that contains SAS Viya in your environment.

Renew Transport Layer Security (TLS) Certificates

Introduction

TLS security certificates have an expiration date. If you use your own certificates, you must renew your certificates when they expire. For the steps that you perform, see [“Renew Your Own Certificates” on page 108](#).

If you use deployment-generated certificates for in-cluster secure communications, run the [“renew-tls-certs.sh Script” on page 108](#). The script:

- regenerates certificates.
- re-creates Kubernetes secrets.
- restarts log-monitoring and metrics-monitoring pods so that the new secrets are used.

Note: For an overview of TLS for SAS Viya Monitoring for Kubernetes see 'Link to System Requirements-Security Options - TLS'.

renew-tls-certs.sh Script

If you use deployment-generated certificates for in-cluster secure communications, run the [renew-tls-certs.sh](#) script:

- 1 Navigate to your SAS Viya Monitoring for Kubernetes repository. The directory is named `viya4-monitoring-kubernetes`.

- 2 Run the following command:

```
./bin/renew-tls-certs.sh -t ALL-MON/ALL-LOG/ALL
```

Where:

- ALL-MON indicates to process Alertmanager, Prometheus and Grafana.
- ALL-LOG indicates to process OpenSearch and OpenSearch Dashboards.
- ALL indicates to process all five applications.

Note: For usage information about the script, run the script with only the Help option (`-h`).

Renew Your Own Certificates

If you supplied your own certificates to be used for communications into the cluster through Ingress or for in-cluster communication, you must:

- delete the Kubernetes secrets.
- regenerate the Kubernetes secrets with valid certificates.
- restart the pods where the secrets are mounted (for in-cluster communications only).

For communication into the cluster with Ingress, perform the following steps:

- 1 Delete existing Kubernetes secrets that are using expired certificates. Run the following command for each secret that must be regenerated:

```
kubectl -n namespace delete secret secret_name
```

Where:

- *namespace* is the namespace that is used during the deployment process. The following values are used by default:

Log monitoring	<i>logging</i>
Metric monitoring	<i>monitoring</i>

If you change the default, be sure to specify the name of your namespace instead of the default.

- *secret_name* is one of the following values:

For log monitoring:

- `elasticsearch-ingress-tls-secret`
- `kibana-ingress-tls-secret`

For metric monitoring:

- `grafana-ingress-tls-secret`
- `prometheus-ingress-tls-secret`
- `alertmanager-ingress-tls-secret`

- 2 Complete the steps listed here: [“Manually Configure Ingress Definitions and Enable TLS” on page 33.](#)

For in-cluster communications, perform the following steps:

- 1 Delete existing Kubernetes secrets that are using expired certificates. Run the following command for each secret that must be regenerated:

```
kubectl -n namespace delete secret secret_name
```

Where:

- *namespace* is the namespace that is used during the deployment process. The following values are used by default:

Log monitoring	<i>logging</i>
Metric monitoring	<i>monitoring</i>

If you change the default, be sure to specify the name of your namespace instead of the default.

- *secret_name* is one of the following values:

For log monitoring:

- `es-transport-tls-secret`
- `es-rest-tls-secret`
- `es-admin-tls-secret`
- `kibana-tls-secret`

For metric monitoring:

- `prometheus-tls-secret`
- `alertmanager-tls-secret`
- `grafana-tls-secret`

- 2 Complete the steps listed here: [“Use Your Own Certificates to Create Kubernetes Secrets” on page 38.](#)

- Restart the pods where the secrets are mounted. To do this, run the following script from your `viya4-monitoring-kubernetes` directory:

```
viya4-monitoring-kubernetes/bin/renew-tls-certs.sh -t ALL-MON/ALL-LOG/ALL -r
```

Where:

- ALL-MON indicates to process Alertmanager, Prometheus and Grafana.
- ALL-LOG indicates to process OpenSearch and OpenSearch Dashboards.
- ALL indicates to process all five applications.

TIP The `-r` parameter restarts only the target resources so that the newly generated secrets are mounted in the pods. (New certificates are not generated.)

Troubleshooting

Log Messages Require More Storage

Problem:

Log messages run out of storage.

Explanation:

You might need to increase storage in the following situations:

- The persistent volume claims (PVCs) associated with the OpenSearch nodes (pods) that are used for storage (that is, the nodes with a role of "data") are running low on space.
- You are considering increasing the retention period for log messages. Longer retention periods require more storage.

TIP In the default configuration, these pods have names that start with "v4m-search-".

Solution:

See the following topics:

- ["Increase Storage for Log Messages" on page 96](#)
- ["Adjust the Retention Period for Log Messages" on page 93](#)

Fluent Bit Log Messages Are Not Available

Problem:

There are no Fluent Bit log messages collected or available from pods running on a particular node.

Explanation:

By default, the monitoring components are deployed so that the pods run on the appropriate nodes and are not generally subject to eviction. However, it is possible for the Fluent Bit pods to be at risk of eviction due to a lack of resources.

Solution:

To prevent the eviction, you can specify the pod's priority. Complete the following procedure.

Note: To perform these tasks, you must have elevated Kubernetes permissions.

- 1 Open the `USER_DIR/logging/user-values-fluent-bit-opensearch.yaml` file where `USER_DIR` is the location of your customization files.
- 2 Add the following line:

```
priorityClassName:priorityClassName
```
- 3 Specify the priority by setting `priorityClassName` to *system-cluster-critical*.
 The Fluent Bit pod is included with the pods that are scheduled first along with the Kubernetes core components.
- 4 Make sure you are in the directory to which you copied the repository.
- 5 Enter the applicable command for your deployment.

For standard deployments	<code>logging/bin/deploy_logging.sh</code>
--------------------------	--

For deployments on OpenShift	<code>logging/bin/deploy_logging_openshift.sh</code>
------------------------------	--

TIP You can also define your own `priorityClass`. See the [Kubernetes documentation about priorityClass](#).

Logging Deployment Fails Due to OpenSearch REST Endpoint Not Being Accessible

Problem:

The `viya4-monitoring-kubernetes/logging/bin/deploy_logging.sh` script fails to deploy the logging components with the following log messages:

```
INFO The OpenSearch REST endpoint does not appear to be quite ready [000];
sleeping for [30] more seconds before checking again.
```

There might be 7-8 of these `INFO` messages generated over 4 minutes. If the script is unable to connect after 4 minutes, it exits with the following error:

```
INFO Loading Content into OpenSearch
ERROR The OpenSearch REST endpoint has NOT become accessible in the expected time; exiting.
ERROR Review the OpenSearch pod's events and log to identify the issue and resolve it before trying again.
ERROR Exiting script [deploy_logging.sh] due to an error executing the command [logging/bin/deploy_opensearch]
```

Explanation:

The deployment scripts use port forwarding to make REST API calls to OpenSearch and OpenSearch Dashboards. If you do not include `localhost` in the list of hosts that are not routed through the proxy server, the OpenSearch deployment fails. This is a topic in pre-deployment customizations: [“Prevent Routing Network Traffic through a Proxy Server” on page 55](#).

To confirm that the proxy settings are the issue, you can use `curl` commands to replicate an HTTP REST call similar to those made by the deployment scripts. Use the commands in the following steps:

- 1 Run the following command and note the port number that is returned and the PID:

```
kubect1 -n logging port-forward --address localhost svc/v4m-search :9200 &
```

Here is an example of the output that is used in subsequent steps:

```
211969
```

```
Forwarding from [::1]:45933 -> 9200
```

- 2 Next, run the following command. If the proxy setting is the issue, the command should fail with an exit code of 000.

```
curl -s -o /dev/null -w '%{http_code}' -XGET https://localhost:45933 --user
'admin:bad_password' --insecure
```

Notes:

- Replace the port number in the URL above with the port number from the first command.
- You can use an incorrect password as it still gives some type of response other than 000 if port-forwarding is working correctly. (If you do know the admin password, you can include it in the `--user` option.)

- 3 Run the following command to confirm that the proxy settings are the issue by using the `-noproxy` argument.

```
curl -s --noproxy "*" -o /dev/null -w '%{http_code}' -XGET https://localhost:45933 --user
'admin:bad_password' --insecure
```

Example output:

```
Handling connection for 45933
401
```

Note that the command returns 401 since we provided an incorrect password. If we had provided the correct password, the return code would be 200.

4 Use the PID that is output from the first command and run this command:

```
kill 211969
```

Example output:

```
Terminated          kubect1 -n logging port-forward --address localhost svc/v4m-search :9200
```

Solution:

Modify the `no_proxy` environment variable to include `localhost` to the list of hosts that are not routed through the proxy server.

Here is an example:

Run the `echo` command to see the existing list of hosts that are already assigned to the `no_proxy` environment variable.

```
echo no_proxy
```

Add `localhost` to the list of hosts on the `no-proxy` environment variable:

```
export no_proxy="localhost,list of other hosts separated by commas"
```

Grafana Log In Fails after Updating the Grafana Admin Password

Note: As of release 1.2.17 (September 2023), this issue is resolved. The `monitoring/bin/change_grafana_admin_password.sh` script changes the Grafana password and you are able to log in with the updated password.

Problem:

After you run the `monitoring/bin/change_grafana_admin_password.sh` script, you can no longer log in to Grafana with either the old password or the new password.

Explanation:

An issue has been identified with Grafana.

Resolution if you have not run the script:

If you have not run the script, you can use this workaround:

- 1 Log in to the Grafana web application using your unchanged admin password.
- 2 Click the **User** icon in the upper right corner.
- 3 Navigate to **Change password**. Update your password using this interface.
- 4 To update the back-end, run the `monitoring/bin/change_grafana_admin_password.sh` script. Use the updated password from step 3.

Resolution if you have already run the script:

If you ran the `monitoring/bin/change_grafana_admin_password.sh` script without using the workaround (in the Grafana web application), you are locked out. The only resolution is to remove and redeploy the metric-monitoring components:

- **First:** Enter the applicable command for your deployment.

For standard deployments	<code>monitoring/bin/remove_monitoring_cluster.sh</code>
--------------------------	--

For deployments on OpenShift	<code>monitoring/bin/remove_monitoring_openshift.sh</code>
------------------------------	--

- **Second:** Enter the applicable command for your deployment.

For standard deployments	<code>monitoring/bin/deploy_monitoring_cluster.sh</code>
--------------------------	--

For deployments on OpenShift	<code>monitoring/bin/deploy_monitoring_openshift.sh</code>
------------------------------	--

Metric-Monitoring Pods Are Not Running

Problem:

One or more of the following symptoms occur:

- No metrics are available.
- Metrics related to Kubernetes nodes are not available.
- Grafana is not available.
- One or more of the expected pods related to metric monitoring are not running:
 - ☐ alertmanager-v4m-alertmanager
 - ☐ prometheus-v4m-prometheus
 - ☐ v4m-grafana
 - ☐ v4m-kube-state-metrics
 - ☐ v4m-operator
 - ☐ a v4m-node-exporter pod for each node
 - ☐ prometheus_pushgateway

Note: This pod is seen only if you run the `deploy_monitoring_viya` script to deploy the metric-monitoring component in a SAS Viya namespace.

Explanation:

By default, the monitoring components are deployed so that the pods run on the appropriate nodes and are not generally subject to eviction. However, it is possible for pods (such as Prometheus or Alertmanager pods) to be at risk of eviction due to a lack of resources.

Solution:

To prevent the eviction, you can specify the pod's priority. Complete the following procedure.

Note: To perform these tasks, you must have elevated Kubernetes permissions.

- 1 Open the `$USER_DIR/monitoring/user-values-prom-operator.yaml` file where `$USER_DIR` is the location of your customization files.
- 2 Add the following lines:

```
prometheusOperator:
  priorityClassName: system-cluster-critical
  admissionWebhooks:
    patch:
      priorityClassName: system-cluster-critical
prometheus:
  prometheusSpec:
    priorityClassName: system-cluster-critical
alertmanager:
  alertmanagerSpec:
    priorityClassName: system-cluster-critical
```

The *system-cluster-critical* priority ensures that the metric-monitoring pods are included with the pods that are scheduled first, along with the Kubernetes core components.

Note: If you have existing content in the `$USER_DIR/monitoring/user-values-prom-operator.yaml` file, be sure to merge these lines in carefully to ensure that you do not duplicate existing lines. Secondly, preserve the indentation of these new lines.

- 3 Make sure you are in the directory to which you copied the repository.
- 4 Enter the applicable command for your deployment.

For standard deployments	<code>monitoring/bin/deploy_monitoring_cluster.sh</code>
--------------------------	--

For deployments on OpenShift	<code>monitoring/bin/deploy_monitoring_openshift.sh</code>
------------------------------	--

TIP You can also define your own *priorityClassName*. See the [Kubernetes documentation about priorityClass](#).

The Kubernetes-Proxy Dashboard Does Not Display Any Data

Problem:

The Grafana dashboard for Kubernetes-Proxy does not display any data.

Explanation:

The kube-proxy metrics are used in the Kubernetes-Proxy dashboard. Some clusters are deployed with the kube-proxy metrics listening address set to 0.0.0.0:10249. This address prevents Prometheus from collecting metrics. Therefore, the dashboard does not display any data.

It is also possible that the cluster does not use kube-proxy. To determine whether a cluster uses kube-proxy, enter the following command:

```
kubectl -n kube-system get pods | grep proxy
```

If no results are returned, kube-proxy is not used in the cluster.

Solution:

To enable collection of the kube-proxy metrics, complete the following steps:

- 1 To edit the YAML file of the configMap for kube-proxy, enter the following command:

```
kubectl edit cm -n kube-system kube-proxy
```

Note: This command opens the system default editor.

- 2 Locate `metricsBindAddress` in the file. If it is set to `0.0.0.0:10249`, replace that setting with `127.0.0.1:10249`.

- 3 To re-create the pods so that they use the new configuration, enter the following command:

```
kubectl delete po -n kube-system -l k8s-app=kube-proxy
```

SAS Launched Jobs Dashboards Do Not Work

Problem:

The following dashboards do not have metrics:

- SAS Launched Jobs — Node Activity
- SAS Launched Jobs — User Activity

Explanation:

Both of these dashboards require that either SAS Workload Orchestrator is enabled or that the feature has a license in your SAS Viya platform deployment.

Solution:

See [“Dashboards That Require a License or Enablement”](#) on page 102 for details.

Avoid Higher Resource Use (Istio Only)

Note: Istio is not supported by SAS Viya at this time.

If your Kubernetes environment uses Istio, be aware that Istio includes an instance of Prometheus. The default configuration of this instance of Prometheus monitors the entire Kubernetes cluster. If you also deploy an instance of Prometheus that is part of the metric-monitoring components, you risk higher resource usage.

This is because the Istio instance of Prometheus discovers and scrapes pods that have the `prometheus.io/*` annotations. That includes SAS Viya components. The metric-monitoring instance of Prometheus also scrapes these pods. Because both instances of Prometheus are scraping the same pods, resource use is higher.

To prevent this situation, disable cluster-wide scraping of pods and services that contain the `prometheus.io/*` annotations. In the Istio instance of Prometheus, disable the following jobs:

- `kubernetes-pods`
- `kubernetes-service-endpoints`

Reference

Components for Deployment

Note: The SAS Viya Monitoring for Kubernetes solution deploys the following components. Your use of each component is governed by its applicable open-source license.

Log-Monitoring Components

The log-monitoring solution uses the following components:

Fluent Bit

collects and distributes log messages that are streamed from all Kubernetes pods on the cluster. SAS provides configuration files so that Fluent Bit can locate messages from all SAS Viya sources. As Fluent Bit collects the log messages, it performs some limited transformations and then passes them to OpenSearch. See the [Fluent Bit documentation](#) for more information about Fluent Bit.

OpenSearch

receives the collected messages from Fluent Bit, indexes the message contents, and directs the messages to the persistent store. The SAS logging solution is implemented with OpenSearch storage and search technology. For more information, see the [OpenSearch documentation](#).

BusyBox (OpenSearch)

is used for the initContainer that checks configuration prior to launching OpenSearch.

OpenSearch Dashboards

provides a graphical user interface, which enables users to search, filter, chart, and display the log messages that are stored in OpenSearch. See [“Performing Basic Tasks in OpenSearch Dashboards” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#) for more information about how to use OpenSearch Dashboards.

SAS provides a set of dashboards for SAS Viya, and you can create your own. For information about these dashboards, see [“OpenSearch Dashboards Provided by SAS” on page 130](#).

Elasticsearch Exporter

collects performance and operational metrics from OpenSearch and makes them available to Prometheus. This enables administrators to monitor the health and performance of this key log-monitoring component.

See the [Elasticsearch Exporter community on GitHub](#).

Helm charts

used to deploy the components. For a list of the Helm charts used, see [“Helm Charts Used by the Log-Monitoring Components” on page 125](#).

Metric-Monitoring Components

The metric-monitoring solution uses the following components:

Prometheus

locates resources, collects and stores metrics, and generates alerts based on metrics. Prometheus uses Cluster Alert Definitions to determine when to generate an alert. See the [Prometheus website](#) for more information.

Alertmanager

manages alerts based on metric data. See the [Alertmanager documentation](#) for more information.

Prometheus Operator

manages the Prometheus and Alertmanager instances. See the [Prometheus Operator GitHub website](#) for information.

Grafana

displays the collected metric data. See the [Grafana documentation](#) for more information. SAS provides a set of dashboards for SAS Viya, and you can create your own. For information about these dashboards, see [“Dashboards for Kubernetes Resources” on page 134](#).

BusyBox (Grafana)

is used for the initContainer that checks configuration prior to launching Grafana.

Admission Webhook

validates configuration information when the Prometheus Operator is deployed. For more information, see [Admission Webhook](#).

Node Exporter

collects hardware and operating system metrics from Linux kernels and makes the metrics available to Prometheus. For more information, see the [Node Exporter GitHub website](#).

Kube State Metrics

collects metrics about the status of the deployment, its nodes, and its pods and makes the metrics available to Prometheus. For more information, see the [kube-state-metrics GitHub website](#).

Configuration Reloader (Alertmanager, Prometheus)

automatically reloads configuration information for Alertmanager and Prometheus when changes are detected.

Prometheus Pushgateway

collects metrics from ephemeral and batch jobs and makes the metrics available to Prometheus. For more information, see the [Prometheus Pushgateway GitHub website](#).

Auto-load Sidecars (Grafana)

provides auto-provisioning functionality for dashboards and data sources to Grafana.

Helm charts

used to deploy the components. For a list of the Helm charts used, see “[Helm Charts Used by the Metric-Monitoring Components](#)” on page 126.

Sample Files for Customization

Samples are provided for several common deployment scenarios as well as a generic sample. Each sample includes the customization files for the scenario. For information about using these samples, see “[Copy Customization Sample Files](#)” on page 20.

The following samples are available:

[Azure Deployment of Log-Monitoring and Metric-Monitoring Components](#)

shows the customizations that are necessary if you deploy on Microsoft Azure Kubernetes Service (AKS). The sample assumes that NGINX Ingress is used, but you can modify the sample for other solutions.

[External Alertmanager](#)

shows how to configure monitoring to use an external instance of Alertmanager.

In a typical deployment, each cluster uses a separate instance of Alertmanager. If you have multiple clusters, you might want alerts from all clusters to come from a single instance of Alertmanager. You might also have an existing instance of Alertmanager and want alerts to come from the existing instance.

[Generic Base for Log-Monitoring and Metric-Monitoring Components](#)

provides a template `USER_DIR` containing a full set of customization files with explanatory comments.

[Grafana Alerts](#)

includes the customizations for adding Grafana alert rules for monitoring SAS Viya environments. The alert `YAML` files are organized into subdirectories by component or category.

Ingress Configuration

The following samples are available for configuring access to the web applications that are deployed as part of the SAS Viya Monitoring for Kubernetes solution:

- [Contour Configuration](#)

demonstrates how to configure Kubernetes Contour HTTPProxy resources for accessing the web applications that are deployed. Options for either host-based and path-based ingress are included.

- [Ingress NGINX Configuration](#)

demonstrates how to configure Kubernetes Ingress NGINX resources for accessing the web applications. Options for either host-based and path-based ingress are included.

Note: You can choose to automate this process. For information, see [“Configure Access to Web Applications” on page 24](#).

Note: In November, 2025, the Kubernetes community announced that it would retire Ingress NGINX. For information, see [Ingress NGINX Retirement: What You Need to Know](#). SAS recommends that you change the ingress controller in existing SAS Viya platform deployments to Contour ingress. For information, see [“Overview” in SAS Viya Platform: Changing Ingress Controllers in a Deployment](#).

Namespace Monitoring

demonstrates how to customize a monitoring deployment to separate cluster monitoring from SAS Viya (namespace) monitoring.

SAS SpeedyStore Monitoring

demonstrates how to extend SAS Viya Monitoring for Kubernetes to monitor the SingleStore instance that is embedded within SAS SpeedyStore. This allows administrators to monitor their SingleStore cluster using the same Grafana instance that they use to monitor the rest of their SAS Viya deployment.

The following samples demonstrate how cloud providers' monitoring solutions can be used to monitor SAS Viya:

[Amazon CloudWatch](#)

describes how to configure Amazon CloudWatch to collect metrics from the services and components in SAS Viya. Complete the deployment and configuration steps in this sample to enable Amazon CloudWatch to collect and visualize metric information from both SAS Viya as well as the metric-monitoring components. This sample currently does not provide information for using Amazon applications to view SAS Viya log messages.

[Azure Monitoring](#)

shows how to configure Azure Monitor to automatically discover and scrape Prometheus-compatible HTTP or HTTPS metric endpoints that are associated with SAS Viya components.

[GKE Monitoring](#)

shows how to deploy the components needed to enable Google Cloud Operations to view SAS Viya metric data.

Google GKE uses Google Cloud Operations to view metric data. In order for Google Cloud Operations to view metric data collected in Prometheus (such as from SAS Viya services) that data must be collected from Prometheus, converted, and exported Google Cloud Operations.

CAUTION

Sample files show plain text passwords as examples. For your production environment, SAS recommends that you set these passwords with a more secure mechanism.

Grafana Alerts

Overview

SAS Viya Monitoring for Kubernetes includes [sample alerts](#) in YAML format for configuring Grafana alert rules that address potential problems with SAS Viya components. You might need to modify elements in the sample alert YAML files before you provision them. Some examples are namespaces and PVCs that are specific to your SAS Viya environment. Alternatively, you might want to modify the condition to customize the alert. This information can be found in the [Alert Rules Structure README.md](#) file.

The following tables list the Grafana-managed alerts that are available as samples.

For more information about provisioning these alerts during the deployment, see [“Customize Alerts Managed by Grafana” on page 56](#).

Configurable Grafana Alerts

Table 12 CAS Alerts

Alert Name	Description	If Alert is Firing
CAS Memory Usage High	This alert checks the CAS memory usage. If it is greater than 300 GB, the alert is fired. Currently, maximum memory is 512 GB. It can be used as an early warning sign to investigate large memory usage and prevent OOMkill of CAS. <pre>condition: (cas_node_mem_size_bytes{type="physical"} - cas_node_mem_free_bytes{type="physical"})/ 1073741824 > 300</pre>	CAS server memory usage is high. Usage should be less than the threshold. Investigate the causes, such as a program or pipeline that is using all of the memory.
CAS Restart Detected	This alert checks that the CAS pod (sas-cas-server-default-controller) existed for less than 15 minutes. This implies that the CAS pod has restarted. <pre>condition: cas_grid_uptime_seconds_total < 15m</pre>	Further investigate the cause of the CAS pod restarts. For help with troubleshooting the CAS Server, see the following documentation: ■ SAS Viya Platform: SAS Cloud Analytic Services

Alert Name	Description	If Alert is Firing
		■ “Monitoring SAS CAS Server” in SAS Viya Platform: Troubleshooting ■ “Monitoring SAS CAS Server” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes

Table 13 Database Alerts

Alert Name	Description	If Alert is Firing
Catalog DB Connections High	This alert checks if the active catalog database connections are greater than 21. The default database connection pool is 22. <pre>condition: sas_db_pool_connections{container="sas-catalog-services", state="inUse"} > 21</pre>	If the maximum number of database connections is reached, the RabbitMQ queues start to fill up with ready messages and this causes issues with SAS Model Studio pipelines. For troubleshooting help, see the troubleshooting section in SAS Viya: Machine Learning User's Guide.
Crunchy Backrest Repo	This alert checks to see if /pgbackrest/rep01 file system is more than 50% full. <pre>condition: ((kubelet_volume_stats_capacity_bytes{persistentvolumeclaim=~"sas-crunchy-platform-postgres-repo1"} - kubelet_volume_stats_available_bytes{persistentvolumeclaim=~"sas-crunchy-platform-postgres-repo1"}) / kubelet_volume_stats_capacity_bytes{persistentvolumeclaim=~"sas-crunchy-platform-postgres-repo1"}) * 100</pre>	This typically happens when the number of archived WAL logs is increasing because they are not expiring and clearing out. For troubleshooting help, see “Avoid

Alert Name	Description	If Alert is Firing
		Database Issues in SAS Viya Platform: Troubleshooting .
Crunchy PGData Usage High	This alert checks to see if /pgdata filesystem is more than 50% full. condition: <pre>((kubernetes_io_v1_volume_stats_capacity_bytes{persistentvolumeclaim=~"sas-crunchy-platform-postgres-00-.*"} - kubernetes_io_v1_volume_stats_available_bytes{persistentvolumeclaim=~"sas-crunchy-platform-postgres-00-.*"}) / kubernetes_io_v1_volume_stats_capacity_bytes{persistentvolumeclaim=~"sas-crunchy-platform-postgres-00-.*"}) * 100</pre>	This typically happens when the number of WAL logs are increasing because they are not clearing out. For troubleshooting help, see “Avoid Database Issues” in SAS Viya Platform: Troubleshooting.

Table 14 Other Alerts

Alert Name	Description	If Alert is Firing
NFS Share Usage High	This alert checks if the NFS share that is attached to CAS is more than 85% full. condition: - <pre>((kubernetes_io_v1_volume_stats_capacity_bytes{persistentvolumeclaim="cas-default-data"} - kubernetes_io_v1_volume_stats_available_bytes{persistentvolumeclaim="cas-default-data"}) / kubernetes_io_v1_volume_stats_capacity_bytes{persistentvolumeclaim="cas-default-data"}) * 100 > 85</pre>	Your users might be filling up their home directory or backups.

Table 15 Viya Platform Alerts

Alert Name	Description	If Alert is Firing
RabbitMQ Ready Queue Backlog	This alert checks if there are more than 10,000 RabbitMQ ready messages. This can include potential orphan queues, the bottleneaking of queues due to catalog service, or both. condition: rabbitmq_queue_messages_ready > 10000	RabbitMQ has a large backlog of messages due to high activity, which can be temporary, or something

Alert Name	Description	If Alert is Firing
		might have gone wrong. For information about the SAS Message Broker, see “SAS Message Broker” in SAS Viya Platform: Infrastructure Servers.
RabbitMQ Unacknowledged Queue Backlog	This alert checks if RabbitMQ has a high number of unacknowledged messages. This means that consumers are taking a long time to process messages or have failed. condition: <code>rabbitmq_queue_messages_unacked > 5000</code>	This might indicate either stuck consumers or performance issues. For information about the SAS Message Broker, see “SAS Message Broker” in SAS Viya Platform: Infrastructure Servers.
Viya Pod Restart Count High	This alert checks the restart count of pods. condition: <code>kube_pod_container_status_restarts_total{namespace="viya"} > 20</code>	The number of pod restarts is greater than 20. Service pods have restarted too many times. One possible cause is OOMkill. This might mean that you need to increase the memory limits for service pods. For more information, see “Troubleshoot Pod Failures” in SAS Viya

Alert Name	Description	If Alert is Firing
<i>Platform: Troubleshooting.</i>		
Viya Readiness Probe Failed	This alert checks for the Ready state of sas-readiness pod. condition: kube_pod_container_status_ready{container="sas-readiness"}	One or more Viya pods are not in a ready state since the sas-readiness pod reflects the health of Viya services. To troubleshoot, see “Verify That Pods Are Ready” in SAS Viya Platform: Troubleshooting.

Table 16 Other Alerts

Alert Name	Description	If Alert is Firing
NFS Share Usage High	This alert checks if the NFS share that is attached to CAS is more than 85% full. condition: - ((kubelet_volume_stats_capacity_bytes{persistentvolumeclaim="cas-default-data"} - kubernetes_volume_stats_available_bytes{persistentvolumeclaim="cas-default-data"}) / kubernetes_volume_stats_capacity_bytes{persistentvolumeclaim="cas-default-data"}) * 100 > 85	Your users might be filling up their home directory or backups.

Helm Charts

Helm Charts Used by the Log-Monitoring Components

Most of the components of the SAS Viya Monitoring for Kubernetes solution are deployed via Helm Charts. Helm Charts provide the mechanism to define configuration options that the project uses. Administrators can modify these configurations in order to better fit their organization's needs.

Use the following links in the table to the Helm Charts and the corresponding default values file. Note that the links are to the *latest* version of the chart and values file. This might not be the version

that SAS Viya Monitoring for Kubernetes deploys. Verify the version of the Helm Chart in the [ARTIFACT_INVENTORY.md](#) within the project repository.

Table 17 *Helm Charts Used By Log-Monitoring Components*

Helm Chart	Default Values from values.yaml File
OpenSearch	Default values
OpenSearch Dashboards	Default values
Fluent Bit	Default values
Elasticsearch Exporter	Default values

Helm Charts Used by the Metric-Monitoring Components

The components of the SAS Viya Monitoring for Kubernetes solution are deployed via Helm Charts. Helm Charts provide the mechanism to define configuration options that the project uses. Administrators are able to modify these configurations in order to better fit their organization's needs.

Use the following links in the table to the Helm Charts and the corresponding default values file. Note that the links are to the *latest* version of the chart and values file. This might not be the version that SAS Viya Monitoring for Kubernetes deploys. Verify the version of the Helm Chart in the [ARTIFACT_INVENTORY.md](#) within the project repository.

Table 18 *Helm Charts Used By Metric-Monitoring Components*

Component	Helm Chart	Default Values from values.yaml File
Prometheus	kube-prometheus-stack	Default values
Grafana		
AlertManager		
Kube-State Metrics		
Node Exporter		
Prometheus Operator		
Prometheus Pushgateway	prometheus-pushgateway	Default values

Internal Logging Users

A small number of user credentials are defined in the `internal_users.yml` file. These users are required in order to initialize the system and to ensure that the log-monitoring components can communicate with each other.

IMPORTANT Do not create additional users in the `internal_users.yml` file. If you need to create additional users to interact with log monitoring, use the OpenSearch security API. For more information, see the [OpenSearch API documentation](#).

This table shows the internal users that are created from the `internal_users.yml` file.

Table 19 *Internal Logging Users, Their Default Passwords, and Their Purposes*

User Name	Default Password	Purpose
admin	■ set by the <code>ES_ADMIN_PASSWD</code> environment variable. ■ if not set, a randomly generated string¹	This is a superuser with complete access to everything in OpenSearch Dashboards. These user credentials should be used only when unlimited access is required. This account should be used only for OpenSearch Dashboards administration tasks, such as reviewing security, adjusting index management policies, or when the <code>logadm</code> account cannot be used.
kibanaserver	a randomly generated string ¹	This user is used by OpenSearch Dashboards to interact with OpenSearch. Do not use this user for any other purpose. It has access to internal OpenSearch Dashboards indexes but does not have access to any other indexes.
logadm	■ set by the <code>LOG_LOGADM_PASSWD</code> environment variable. ■ If no password is set for the <code>logadm</code> user, but a password is set for the <code>admin</code> user, its password is used for the <code>logadm</code> user also. Note: Setting the <code>logadm</code> password by way of the <code>admin</code> user will be discontinued in a later release. ■ If neither password is set, a randomly generated string.¹	This is the primary user account used for log-monitoring activity. The OpenSearch Dashboards administrator account (<code>admin</code>) should be used only for OpenSearch Dashboards administration tasks.

logcollector	a randomly generated string ¹	This user is used by Fluent Bit to load log messages into OpenSearch. Do not use this user for any other purpose.
metricgetter	a randomly generated string ¹	This user is used by Prometheus to collect operational and performance metrics from OpenSearch. Do not use this user for any other purpose.

¹The randomly generated passwords are created when the deployment script is run. These passwords are never displayed in logs. If the password for the `admin` or `logadm` account is randomly generated, it is displayed in a message during deployment. Different random strings are generated for each account.

Note: You can change passwords through the following methods:

- During the deployment process: [“Set Administrator User Passwords” on page 55](#)
 - Post deployment: [“Change Administrator Passwords after Deployment” on page 82](#)
-

ServiceMonitors Provided for SAS Viya

The following ServiceMonitors are provided by SAS Viya Monitoring for Kubernetes. The ServiceMonitors are located in the [SAS Viya Monitoring for Kubernetes](#) repository on GitHub.

logging/serviceMonitor-elasticsearch.yaml

ServiceMonitor for OpenSearch

logging/serviceMonitor-fluent-bit-v2.yaml

ServiceMonitor for Fluent Bit

viya/serviceMonitor-sas-arke.yaml

ServiceMonitor for the Arke message broker

viya/serviceMonitor-sas-cas-operator.yaml

ServiceMonitor for the SAS Viya CAS operator

viya/serviceMonitor-sas-cas-server.yaml

ServiceMonitor for the SAS Viya CAS server

viya/serviceMonitor-sas-go-services.yaml

ServiceMonitor for SAS Viya services that are written in Go

viya/serviceMonitor-sas-java-services.yaml

ServiceMonitor for SAS Viya services that are written in Java

viya/serviceMonitor-sas-postgres.yaml

ServiceMonitor for Postgres

viya/serviceMonitor-sas-pushgateway.yaml

ServiceMonitor for Prometheus Pushgateway

viya/serviceMonitor-sas-rabbitmq.yaml
ServiceMonitor for RabbitMQ

Recording Rules Provided for SAS Workload Manager

A recording rule is a Prometheus rule type that contains a PromQL expression to calculate or process already collected metrics. Essentially, the rule runs the PromQL query on existing metrics, and the resulting value can be queried and collected as if it were a metric. The rules can be run periodically so that the values can be tracked over time in Grafana dashboards or surfaced in other environments. For more information, see [Prometheus: Recording Rules](#).

The SAS Viya Monitoring for Kubernetes solution provides a set of recording rules that are used to provide data for the following Grafana dashboards:

- SAS Launched Jobs - User Activity
- SAS Launched Jobs - Node Activity

IMPORTANT See “Dashboards That Require a License or Enablement” on page 102 for details.

See “[SAS Launched Jobs – User Activity](#)” on page 133 for information about these dashboards.

The following rules are defined in the rules-sas-launcher-job.yaml file. This file is located in the [SAS Viya Monitoring for Kubernetes](#) repository on GitHub.

`sas_launcher_pod_info`

provides information about the pods that are running jobs.

`sas_launcher_pod_status`

provides data about the number of jobs that are currently running. This rule populates the **Workload Distribution by Node** and **Workload Distribution by User** charts and is used by all other recording rules in the set.

`sas_launcher_pod_cpu_core_usage`

provides data about the CPU that is used by jobs. This rule populates the **CPU Usage by Node**, **CPU Usage by User**, and **Total CPU Usage** charts.

`sas_launcher_pod_mem_working_set_bytes`

provides data about the memory that is used by node. This rule populates the **Memory Usage by Node** and **Total Memory Usage** charts in the SAS Launched Jobs - Node Activity dashboard.

`sas_launcher_pod_mem_usage_bytes`

provides data about the memory that is used by user. This rule populates the **Memory Usage by User** and **Total Memory Usage** charts in the SAS Launched Jobs - User Activity dashboard.

`sas_launcher_pod_fs_read_bytes_total`

provides data about the file Read operations that are performed by jobs. This rule populates the **File I/O** chart.

`sas_launcher_pod_fs_write_bytes_total`

provides data about the file Write operations that are performed by jobs. This rule populates the **File I/O** chart.

`sas_launcher_pod_net_receive_bytes_total`

provides data about the network receive operations that are performed by jobs. This rule populates the **Network I/O** chart.

`sas_launcher_pod_net_transmit_bytes_total`

provides data about the network transmit operations that are performed by jobs. This rule populates the **Network I/O** chart.

OpenSearch Dashboards Provided by SAS

SAS provides the following dashboards for use with OpenSearch Dashboards:

Log Message Volumes With Level

This dashboard displays information about all log messages from the Kubernetes cluster that are grouped by the source of the log message, such as a SAS Viya service. The dashboard contains charts that display the message volume over time, the number of messages by level, and the message volume over time by source. The dashboard also contains a table that displays the time, level, source, and message for each captured log message. This dashboard is useful to SAS Viya administrators for routine administration.

Log Message Volumes With Level (Pods/Containers)

This dashboard displays information about all log messages from the Kubernetes cluster that are grouped by the source pod and container. The dashboard contains charts that display the message volume over time, the number of messages from each pod, and the message volume over time by source. The dashboard also contains a table that displays the time, level, source, and message for each captured log message. This dashboard is most useful to Kubernetes administrators for routine administration.

Kubernetes Events

This dashboard displays information about Kubernetes events. It contains a chart that displays the number of warning-level events per pod, a chart that displays the number of warning-level events per reason, and a table that contains detailed information about all events.

Top 10 Errors

This dashboard displays the top ten most common messages at the ERROR or higher level. You can select a message to see details about the message and information about individual instances of the message, including the Kubernetes pod and container from which each instance was collected.

Operations Processing Activity

This dashboard displays information about the processing of log messages. This dashboard is useful when working with SAS Technical Support to troubleshoot issues with missing log messages.

Update Checker

This dashboard displays all of the results of the Update Checker reports that have run over the last seven days. For more information, see [“Update Checker Report” in SAS Viya Platform Operations: Updating Software Tasks](#).

Grafana Dashboards Descriptions

The following topics provide descriptions of available Grafana dashboards. To learn the file names of the dashboards for deployment, see [“Grafana Dashboards Location and File Names” on page 139](#).

Dashboards for SAS Viya Deployments

Overview

Grafana dashboards give you a comprehensive view of the consumption of resources of all pods in a SAS Viya namespace that are deployed on the nodes within a cluster. These dashboards are tagged with `sas-viya` and `sas-workload-orchestrator`. Metrics are collected from the container level, and then aggregated at pods, nodes, and cluster levels. With these dashboards, you can do the following:

- View the availability of your SAS Viya deployment
- View the resource consumption of your Go and Java services
- Monitor SAS Cloud Analytic (CAS) Server CPU and Memory
- Monitor the performance of your Kubernetes cluster and namespaces
- Monitor Crunchy Data (PostgreSQL)
- Monitor RabbitMQ

This section includes descriptions of dashboards that you can use to monitor your SAS Viya deployments.

Note: Because your Kubernetes administrator can set up Grafana to display only a limited set of dashboards, all of these dashboards might not be available. See [Manage Dashboard Permissions](#) in Grafana documentation.

Dashboards for SAS Viya Components

OpenSearch

provides information about the instance of OpenSearch that is used by the SAS Information Catalog component in the SAS Viya deployment.

PostgreSQL

displays performance and usage metrics for one or more PostgreSQL databases. Use the **PGCluster** field at the top of the dashboard to select the database whose metrics you want to view.

PostgreSQL Database

displays performance and usage metrics for one or more PostgreSQL databases. Use the **Database** field at the top of the dashboard to select the databases whose metrics you want to view.

RabbitMQ-Erlang

collects and visualizes metrics that are related to the Erlang VM memory. See [runtime tuning](#) in RabbitMQ documentation.

RabbitMQ-Overview

displays performance and usage metrics for RabbitMQ, which collects metric data.

SAS Arke Dashboard

displays events that are produced and consumed by microservices. The Arke service is a message broker proxy that provides a generic subscribe and publish interface. Clients connect to Arke and provide message broker connection information.

SAS CAS Overview or SAS CAS Overview with Logs

displays CPU usage and memory usage as open files for a selected CAS server. Use the **CAS Server** menu at the top of the dashboard to select a CAS server.

If you ran the `/project_repository/monitoring/bin/create_logging_datasource.sh` script, log messages that are associated with CAS server are also included in the dashboard. The script creates a data source that enables Grafana to access log messages from OpenSearch and then replaces the original SAS CAS Overview dashboard with SAS CAS Overview with Logs dashboard.

SAS Go Service Details or SAS Go Service Details with Logs

displays performance and usage metrics for a selected SAS Viya service that is written in Go or for all SAS Go services. Use the **Application** menu to select the service whose metrics you want to view.

If you ran the `/project_repository/monitoring/bin/create_logging_datasource.sh` script, log messages that are associated with Go microservices are also included in the dashboard. The script creates a data source that enables Grafana to access log messages from OpenSearch and then replaces the original SAS Go Service Details dashboard with SAS Go Service Details with Logs dashboard.

SAS Java Services or SAS Java Services with Logs

displays performance and usage metrics for a selected SAS Viya service that is written in Java or for all SAS Java services. Use the **Application** menu to select the service whose metrics you want to view.

If you ran the `/project_repository/monitoring/bin/create_logging_datasource.sh` script, log messages that are associated with Java microservices are also included in the dashboard. The script creates a data source that enables Grafana to access log messages from OpenSearch and then replaces the original SAS Java Services dashboard with SAS Java Services with Logs dashboard.

SAS Launched Jobs – Node Activity

displays information organized by node about SAS jobs that use the launcher service. The report displays the number of jobs that are pending and running, the amount CPU and memory that are used by jobs, and the network I/O of the jobs. It also charts the workload distribution by node and the CPU and memory usage by node. You can filter the results to display information for only specified namespaces (SAS Viya deployments), nodes, job types, users, and jobs. See [“Monitoring Jobs Started by SAS Launcher Service” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#) for more information about this dashboard.

IMPORTANT See [“Dashboards That Require a License or Enablement”](#) on page 102 for details.

TIP The dashboard is tagged with `sas-workload-orchestrator`.

SAS Launched Jobs – User Activity

displays information organized by user about SAS jobs that use the launcher service. The report displays the number of jobs that are pending and running, the amount CPU and memory that are used by jobs, and the network I/O of the jobs. It also charts the workload distribution by user and the CPU and memory usage by user. You can filter the results to display information for only specified namespaces (SAS Viya deployments), nodes, job types, users, and jobs. See [“Monitoring Jobs Started by SAS Launcher Service” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#) for more information about this dashboard.

IMPORTANT See [“Dashboards That Require a License or Enablement” on page 102](#) for details.

TIP The dashboard is tagged with `sas-workload-orchestrator`.

SAS Micro Analytic Service

displays metrics about the performance and usage of SAS Micro Analytic Service. The dashboard displays information about throughput, latency, and scores organized by module, instance, and client identifier. Additional metrics about the health and operation of SAS Micro Analytic Service pods are also reported. Note that most of this dashboard's charts report “No data” when SAS Micro Analytic Service is idle.

IMPORTANT See [“Dashboards That Require a License or Enablement” on page 102](#) for details.

TIP The dashboard is tagged with `sas-micro-analytic-service`.

The following dashboards were developed by SAS teams for performance tuning:

Perf / Analysis

This dashboard was developed by an internal testing team at SAS. It consolidates a large number of metrics to allow the team to monitor many aspects of their cluster and SAS Viya deployments from a single dashboard. The Bird's Eye View panel at the top displays critical metrics. More detailed metrics are arranged in additional rows. Most of the panels include notes that explain the metric that is displayed and how to interpret the information.

Perf / Container Utilization

monitors requests (the amount of memory requested for a container) made per pod on a node and limits (the maximum amount of memory a container can use on a pod). This dashboard enables you to view the balance between requests and limits.

Perf / Kubernetes Headroom

displays metrics for memory and CPU usage for the cluster, the requests made for each namespace, and the requests made for each node

Perf / Node Utilization

displays metrics for CPU usage, memory usage, network performance disk I/O, and disk space for nodes. The metrics do not account for requests or limits. Select a node ID on a graph's legend in order to display metrics for only the selected node.

Perf / Node Utilization Detail

displays detailed metrics for CPU usage, memory usage, network performance disk I/O, and disk space for nodes. The metrics do not account for requests or limits. Select a node ID on a graph's legend in order to display metrics for only the selected node.

Dashboards for Kubernetes Resources

Overview

Because of the complex nature of a SAS Viya platform deployment on Kubernetes, no single dashboard provides a view of everything you need to know about your Kubernetes system. One strategy is to start by using dashboards that provide a high-level view (such as a cluster or namespace view) in order to get an overall view of system metrics. Then, you can use more focused dashboards to identify any problems or anomalies that you notice.

The following sections include descriptions of the dashboards that can be used for monitoring your Kubernetes cluster. They are grouped by the metrics that are monitored in the dashboard.

Note: Because your Kubernetes administrator can set up Grafana to display only a limited set of dashboards, all of these dashboards might not be available. See [Manage Dashboard Permissions](#) in Grafana documentation.

Dashboards for Kubernetes Resources

These dashboards provide high-level views that you can use to locate problems with Kubernetes resources:

Kubernetes Cluster

provides a comprehensive view of the metrics for each layer of a Kubernetes cluster. Panels on this dashboard enable you to view metrics for CPU usage, memory usage, and network I/O for the following levels:

- cluster
- the namespaces in the cluster
- the pods in a namespace
- the containers in a pod
- the processes in a container

Kubernetes Deployment metrics

displays metrics for a selected Kubernetes deployment or for all deployments, which include pods and replicas of pods. If you are viewing metrics for all deployments, select a deployment ID on a graph's legend in order to display metrics for only the selected deployment. You can also use the **Deployment** menu at the top of the dashboard to select a deployment.

Kubernetes/Persistent Volumes

displays usage metrics for selected persistent volume claims.

Perf/Analysis

This dashboard was developed by an internal testing team at SAS. It consolidates a large number of metrics to allow the team to monitor many aspects of their cluster and SAS Viya deployments from a single dashboard. The Bird's Eye View panel at the top displays critical metrics. More detailed metrics are arranged in additional rows. Most of the panels include notes that explain the metric that is displayed and how to interpret the information.

Perf/Container Utilization

monitors requests (the amount of memory requested for a container) made per pod on a node and limits (the maximum amount of memory a container can use on a pod). This dashboard enables you to view the balance between requests and limits.

Perf/Kubernetes Headroom

displays metrics for memory and CPU usage for the cluster, the requests made for each namespace, and the requests made for each node

Perf/Node Utilization

displays metrics for CPU usage, memory usage, network performance disk I/O, and disk space for nodes. The metrics do not account for requests or limits. Select a node ID on a graph's legend in order to display metrics for only the selected node.

Perf/Node Utilization Detail

displays detailed metrics for CPU usage, memory usage, network performance disk I/O, and disk space for nodes. The metrics do not account for requests or limits. Select a node ID on a graph's legend in order to display metrics for only the selected node.

Nodes Exporter/Nodes

displays CPU usage, memory usage, disk I/O, and networking metrics for the nodes in the cluster.

Node Exporter/USE Method/Cluster

displays high-level information about the cluster using the Utilization, Saturation, and Errors (USE) method. See [The USE Method](#) for information about this methodology.

Node Exporter/USE Method/Node

displays high-level information about a node using the Utilization, Saturation, and Errors (USE) method. See [The USE Method](#) for information about this methodology.

Dashboards for Kubernetes Compute Services

These dashboards provide usage and performance metrics for Kubernetes resources.

Kubernetes/Compute Resources/Multi-Cluster

provides cluster-level metrics.

Kubernetes/Compute Resources/Cluster

provides metrics for CPU and memory usage for the cluster as well as for the namespaces that are running on the cluster. You can select a namespace in one of the tables on the dashboard in order to drill down to the Kubernetes / Compute Resources / Namespace (Pods) dashboard for the selected namespace.

Kubernetes/Compute Resources/Namespace (Pods)

displays CPU and memory usage for a selected namespace and for the pods in the namespace. Use the **namespace** menu at the top of the dashboard to select the namespace whose metrics you want to view. Select a pod in one of the tables on the dashboard in order to drill down to the Kubernetes / Compute Resources / Pod dashboard for the selected pod.

Kubernetes/Compute Resources/Namespace (Workloads)

displays CPU usage, memory usage, and network metrics for a selected namespace and for the workloads in the namespace. Use the **namespace** menu at the top of the dashboard to select the namespace whose metrics you want to view. Select a workload in one of the tables on the dashboard in order to drill down to the Kubernetes / Compute Resources / Workload dashboard for the selected workload.

Kubernetes/Compute Resources/Node (Pods)

displays CPU and memory usage for a selected node and for the pods in the node. Use the **node** menu at the top of the dashboard to select the node whose metrics you want to view. Select a pod in one of the tables on the dashboard in order to drill down to the Kubernetes/Compute Resources/Pod dashboard for the selected pod.

Kubernetes/Compute Resources/Pod

displays CPU usage, memory usage, and networking metrics for a selected pod. Use the **pod** menu at the top of the dashboard to select the pod whose metrics you want to view. (This dashboard can also be accessed from the following dashboards: Kubernetes/Compute Resources/Namespace (Pods), Kubernetes/Compute Resources/Node (Pods), Kubernetes/Compute Resources/Workload.)

Kubernetes/Compute Resources/Workload

displays CPU usage, memory usage, and network metrics for a selected workload and for the pods in the workload. Use the **workload** menu at the top of the dashboard to select the workload whose metrics you want to view. Select a pod in one of the tables on the dashboard in order to drill down to the Kubernetes/Compute Resources/Pod dashboard for the selected pod.

Note: Most of the Kubernetes compute services dashboards contain drill-down capabilities. Here is an example:

- In the **Kubernetes/Compute Resources/Cluster** dashboard, select a namespace in a data table (CPU Quota, Memory Requests by Namespaces, Current Network Usage, or Current Storage IO), to surface the following:
 - **Kubernetes/Compute Resources/Namespace (Pods)** dashboard. Select a pod in a data table (CPU Quota, Memory Quota, Current Network Usage, or Current Storage IO) to display the following dashboard:
 - **Kubernetes/Compute Resources/Pod dashboard.** (This dashboard is also accessible through these dashboards: **Kubernetes/Compute Resources/Node (Pods)**, **Kubernetes/Compute Resources/Workload**.)
-

Dashboards for Kubernetes Networking

These dashboards provide networking metrics for Kubernetes resources. Some basic network performance metric information might also be provided in other dashboards for Kubernetes resources.

Kubernetes/Networking/Cluster

displays networking metrics for the cluster and the namespaces within the cluster. Select a namespace in the **Current Status** table to drill down to the Kubernetes/Networking/Namespace (Pods) dashboard for the selected namespace.

Kubernetes/Networking/Namespace (Pods)

displays networking metrics for a selected namespace and the pods within the namespace. Use the **namespace** menu at the top of the dashboard to select the namespace whose metrics you want to view. Select a pod in the **Current Status** data table to surface the Kubernetes/Networking/Pod dashboard for the selected pod.

Kubernetes/Networking/Pod

displays networking metrics for a selected pod. Use the **pod** menu at the top of the dashboard to select the pod whose metrics you want to view.

Kubernetes/Networking/Namespace (Workload)

displays networking metrics for a selected namespace and for the workloads in the namespace. Use the **namespace** menu at the top of the dashboard to select the namespace whose metrics you want to view. Select a pod in the **Current Status** data table to surface the Kubernetes/Networking/Workload dashboard for the selected pod.

Kubernetes/Networking/Workload

displays networking metrics for a selected workload. Use the **workload** menu at the top of the dashboard to select the workload whose metrics you want to view.

Note: Kubernetes networking dashboards contain drill-down capabilities. Here is an example:

- In the **Kubernetes/Networking/Cluster** dashboard, select a namespace in the **Current Status** data table to display the following:
 - **Kubernetes/Networking/Namespace (Pods)** dashboard. Select a pod in the **Current Status** data table to display the:
 - **Kubernetes/Networking/Pod** dashboard.
-

Dashboards for Kubernetes Control Plane Components

These dashboards provide metrics for controller resources in the Kubernetes control plane.

CoreDNS

provides metrics for requests, responses, and cache for the CoreDNS server that is running in your Kubernetes cluster. The CoreDNS server is a core component in your Kubernetes system.

etcd

displays performance and usage metrics for the etcd server. This server is a Kubernetes control plane component that provides a backing store for cluster data.

Kubernetes/API server

provides usage and performance metrics for the API server. This server is a Kubernetes control plane component that exposes the Kubernetes API.

Kubernetes/Controller Manager

displays usage and performance metrics for the Kubernetes controller manager. This manager is a Kubernetes control plane component that runs controller processes.

Kubernetes/Kubelet

displays metrics for the kubelet component. This component is a node component that ensures that containers are running in a pod. You can view metrics for kubelets on selected nodes or on all nodes.

Kubernetes/Proxy

provides metrics for the Kubernetes proxy component. This component is a node component that provides networking services for a node. You can view metrics for proxies on selected nodes or on all nodes.

Kubernetes/Scheduler

provides metrics for the Kubernetes scheduler component. This component is a Kubernetes control plane component that determines which node the pods should run on, based on available resources.

NGINX Ingress controller

displays performance and usage metrics for Ingress traffic inside of the cluster.

Dashboards for SAS Viya Monitoring for Kubernetes Solutions

Dashboards for SAS Viya Monitoring for Kubernetes Components

These dashboards provide performance metrics for components that are included in the SAS Viya Monitoring for Kubernetes solution for monitoring and logging.

Note: Metrics are available only when the associated component is deployed.

Alerts

provides information about alerts that are triggered in Prometheus. This dashboard is the only way to view Alertmanager alerts in Grafana. For information about the alerts, see the [Alertmanager documentation](#).

Alertmanager / Overview

provides metrics on the state and health of Alertmanager.

Elasticsearch

displays metrics about the instance of OpenSearch that is part of SAS Viya Monitoring for Kubernetes.

Fluent Bit - Log Message Collection

provides metrics for the Fluent Bit process. This dashboard enables you to see which pods have unusually high levels of logging activity, which can then help you better filter log messages in OpenSearch Dashboards.

Grafana Overview

provides metrics on the state and health of Grafana.

Prometheus Overview

displays performance metrics for the Prometheus service.

Grafana Dashboards Location and File Names

Overview

Grafana dashboards that are deployed with the SAS Viya Monitoring for Kubernetes project have been either created by the SAS team, derived from open-source community projects, or they are from a third-party source.

The project migrated to Grafana 11 as part of the [1.2.27 release](#). This version of Grafana dropped support for some older visualizations plug-ins. Therefore, dashboards that used the older visualization technology had to be migrated to newer visualizations. A migration was done for all of the dashboards maintained by this project. Some of the dashboards that are maintained by third-parties have also been migrated by those third-parties. However, others were not and currently rely on the auto-migration features of Grafana 11. All of the dashboards, regardless of who maintains them, should be functional.

The appearance of the original dashboards has been maintained, where possible. Some minor cosmetic differences might be displayed. These differences include the following:

- Column labels, for example : **current** is now **Last*** and **avg** is now **mean**.
- order of columns.
- the appearance of values that are used in the X and Y axis, and their associated reference lines.
- colors that are used for some elements such as plots, bars, or column headers.
- To increase readability, legends might have moved from below the chart to the right side of the chart or vice versa.
- Missing values might appear in a different order in tables.

The following tables include the dashboard name, its origin, and the location of the definition file in the project if it is created and maintained by the project team.

For descriptions of the dashboards, see these topics:

- [“Dashboards for SAS Viya Deployments” on page 131](#)
- [“Dashboards for Kubernetes Resources” on page 134](#)
- [“Dashboards for SAS Viya Monitoring for Kubernetes Solutions” on page 138](#)

Grafana Dashboards That Are Maintained by the SAS Viya Monitoring for Kubernetes Project

Note: The location of the definition file is included if you need to reload a dashboard definition. You can do this by using the `deploy_dashboard.sh` script that is located in the `project_repository/monitoring/bin/` subdirectory. For example, to reload the Kubernetes Cluster dashboard, run the following command:

```
/project_repository/monitoring/bin/deploy_dashboards.sh /project_repository/
```

```
monitoring/dashboards/kube/k8s-cluster-dashboard.json
```

The command above should be run on one line and without spaces in the directory path.

Table 20 *Grafana Dashboards That Are Maintained by the SAS Viya Monitoring for Kubernetes Project*

Dashboard Name	Origin	Location of Definition File in Project Repository (subdirectory of project_repository/ monitoring/dashboards/)
Alerts	Grafana community dashboard	kube/prometheus-alerts.json
Elasticsearch	Grafana community dashboard (modified to add additional filters, such as datasource and namespace)	logging/elasticsearch-dashboard.json
Fluent Bit - Log Message Collection	Grafana community dashboard	logging/fluent-bit.json
Kubernetes Cluster	Grafana community dashboard	kube/k8s-cluster-dashboard.json
Kubernetes Deployment metrics	Grafana community dashboard	kube/k8s-deployment-dashboard.json
NGINX Ingress controller	Grafana community dashboard (earlier version)	nginx/nginx-dashboard.json
OpenSearch	Grafana community dashboard (earlier version)	viya/sas-opensearch.json
Perf/Analysis	SAS internal testing team	kube/perf-k8s-analysis.json
Perf/Container Utilization	SAS internal testing team	kube/perf-k8s-container-util.json
Perf/Kubernetes Headroom	SAS internal testing team	kube/perf-k8s-headroom.json
Perf/Node Utilization	SAS internal testing team	kube/perf-k8s-node-util-detail.json
Perf/Node Utilization Detail	SAS internal testing team	kube/perf-k8s-node-util.json

Dashboard Name	Origin	Location of Definition File in Project Repository (subdirectory of project_repository/ monitoring/dashboards/)
PostgreSQL	pgmonitor project (modified to add namespace filter)	pgmonitor/pg-details.json
PostgreSQL Database	Grafana community dashboard	viya/postgres- dashboard.json
RabbitMQ-Erlang	Grafana community dashboard (Revision #7 27DEC23, modified for migration to Grafana 11 and to include the Datasource filter)	rabbitmq/erlang-memory- allocators.json
RabbitMQ-Overview	Grafana community dashboard (Revision #14 27DEC23, modified for migration to Grafana 11 and to include the Datasource filter)	rabbitmq/rabbitmq- overview.json
SAS Arke Dashboard	SAS Arke development team	viya/sas-arke- dashboard.json
SAS CAS Overview	SAS Viya Monitoring project team	viya/cas-dashboard.json
SAS Go Service Details	SAS Viya Monitoring project team	viya/go-service- dashboard.json
SAS Java Services	SAS Viya Monitoring project team	viya/java-service- dashboard.json
SAS Launched Jobs - Node Activity	SAS Viya Monitoring project team	viya/sas-launched-jobs- node.json
SAS Launched Jobs - User Activity	SAS Viya Monitoring project team	viya/sas-launched-jobs- users.json
SAS Micro Analytic Service	SAS Micro Analytic Service development team	viya/sas-mas- dashboard.json
SAS Viya Welcome	SAS Viya Monitoring project team	welcome/viya-welcome- dashboard.json
SAS CAS Overview with Logs	SAS Viya Monitoring project team	viya-logs/cas-dashboard.json

Dashboard Name	Origin	Location of Definition File in Project Repository (subdirectory of project_repository/ monitoring/dashboards/)
SAS Go Service Details with Logs	SAS Viya Monitoring project team	viya-logs/go-service-dashboard.json
SAS Java Services with Logs	SAS Viya Monitoring project team	viya-logs/java-service-dashboard.json

Grafana Dashboards That Are Maintained by a Third Party

Note: For documentation on the following Grafana dashboards, see [Prometheus Monitoring Mixins](#).

Table 21 Grafana Dashboards That Are Maintained by a Third-Party

Dashboard Name	Origin
Alertmanager/Overview	node-exporter-mixin via the kube-prometheus-stack Helm Chart
CoreDNS	kube-prometheus-stack-helm-charts , derived from comparable dashboard from kubernetes-mixin
etcd	etcd-mixin via the kube-prometheus-stack Helm Chart
Grafana Overview	grafana-mixins via the kube-prometheus-stack Helm Chart
Kubernetes/API server	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Compute Resources/Cluster	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Compute Resources/Multi-Cluster	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Compute Resources/Namespace (Pods)	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Compute Resources/Namespace (Workloads)	kubernetes-mixin via the kube-prometheus stack Helm Chart

Dashboard Name	Origin
Kubernetes/Compute Resources/Node (Pods)	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Compute Resources/Pod	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Compute Resources/Workload	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Controller Manager	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Kubelet	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Networking/Cluster(1)	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Networking/Namespace (Pods) ¹	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Networking/Namespace (Workload) ¹	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Networking/Pod	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Networking/Workload	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Persistent Volumes	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Proxy	kubernetes-mixin via the kube-prometheus stack Helm Chart
Kubernetes/Scheduler	kubernetes-mixin via the kube-prometheus stack Helm Chart
Node Exporter/Mac OS	kubernetes-mixin via the kube-prometheus stack Helm Chart
Node Exporter/Nodes	node-exporter-mixin via the kube-prometheus-stack Helm Chart
Node Exporter/USE Method/Cluster	node-exporter-mixin via the kube-prometheus-stack Helm Chart

Dashboard Name	Origin
Node Exporter/USE Method/Node	node-exporter-mixin via the kube-prometheus-stack Helm Chart
Prometheus/Overview ¹	kubernetes-mixin via the kube-prometheus stack Helm Chart

¹Compatibility issues between these dashboards and Grafana 11.x were found during the upgrade effort. As a temporary fix, SAS Viya Monitoring for Kubernetes is deploying its own versions of these dashboards that are derived from earlier official versions. If you do not want to deploy this temporary fix, you can deploy the official versions of these dashboards in spite of the compatibility issues. Set the `V4M_TEMP_REPLACE_PROBLEMATIC_MIXIN_DASHBOARDS` environment variable to `false` before you run the `/project_repository/monitoring/bin/deploy_monitoring_cluster.sh` script. SAS Viya Monitoring for Kubernetes will deploy the official versions of the dashboards when the compatibility issues have been resolved.

Monitoring and Logging on Cloud-Provider Platforms

Introduction

This section contains information about using SAS Viya Monitoring for Kubernetes in cloud-provider platforms (Azure, Amazon, and Google) that are supported by SAS Viya platform.

SAS Viya Monitoring for Kubernetes can be deployed and can run in any of the cloud provider's environments. The applications must be deployed using Ingress, but minimal additional customization is required. Because the applications provided for SAS Viya work the same way across cloud platforms, administrators can have a consistent experience when moving between cloud platforms.

As an alternative to using SAS Viya Monitoring for Kubernetes, you might choose to use the cloud provider's native application for viewing metric data and log messages. However, there are differences with coverage and configuration to consider if you choose to use a native-monitoring application with SAS Viya.

The configurations for SAS Viya Monitoring for Kubernetes are focused on detailed metrics and messages from the applications and services that make up SAS Viya. In some cases, the cloud provider's native-monitoring application might not support the same level of detail. For administrators who are interested in metrics and log messages from SAS Viya services and components, SAS Viya Monitoring for Kubernetes provides the most comprehensive and detailed view of this information. For administrators who are primarily interested in information from cluster resources, the cloud provider's native-monitoring application might provide the level of log-monitoring and metric-monitoring support that is required.

Configuration and customization steps are required in order to integrate the collected SAS Viya metric data and log messages with native-monitoring applications. Samples are included in the [SAS](#)

[Viya Monitoring for Kubernetes](#) GitHub repository that provide the steps required to display SAS Viya metrics and log messages on some native-monitoring applications, but samples are not provided for all possible scenarios.

Microsoft Azure

Deploying SAS Viya Logging and Monitoring on Azure

The default Azure StorageClass is not large enough for use with the SAS Viya Monitoring for Kubernetes solution. You can use a StorageClass other than the default, or you can create a custom StorageClass. The procedures in the Azure sample provide information about creating a custom StorageClass.

In order to use SAS Viya Monitoring for Kubernetes in an Azure environment, you must deploy the components using Ingress. The Azure Deployment sample includes deployment using Ingress. The procedure in the Ingress sample also provides the same information about deployment using Ingress.

Alternative Monitoring Approach: Azure Monitor

As an alternative to using SAS Viya Monitoring for Kubernetes, you can use Azure Monitor, which is Azure's monitoring application. In order to determine which solution is right for your environment, you must consider the monitoring needs of both the cluster administrator and the SAS Viya administrator and compare the capabilities of the monitoring solutions. See ["Choosing a Monitoring Strategy for an Azure Environment"](#) on page 146 for more information.

Amazon Elastic Kubernetes Service (EKS)

Deploying SAS Viya Monitoring for Kubernetes on Amazon EKS

In order to use SAS Viya Monitoring for Kubernetes in an Amazon EKS environment, you must deploy monitoring and logging using Ingress. See the Ingress sample for Azure Deployment information.

No other customization is needed to deploy and run SAS Viya Monitoring for Kubernetes on Amazon EKS.

Alternative Monitoring Approach: CloudWatch

As an alternative to using SAS Viya Monitoring for Kubernetes, you can use CloudWatch, which is Amazon EKS's monitoring application.

If you choose to use CloudWatch to monitor SAS Viya metrics, you must do the following:

- Deploy the CloudWatch agent for Prometheus.
- Make configuration changes to convert the Prometheus metrics collected from SAS Viya into performance log events.
- Map the log events to CloudWatch metrics.

Follow the steps in the Amazon CloudWatch Integration sample (in the samples/cloudwatch subdirectory of the GitHub repository) to use CloudWatch to monitor a selected group of SAS Viya metrics. Not all SAS Viya metrics are supported by this sample, because supporting all SAS Viya

metrics would require every possible combination of labels from every SAS Viya source and every metric to be mapped to CloudWatch metric types.

Amazon CloudWatch can also enable you to monitor the log messages produced by SAS Viya applications and services. However, CloudWatch does not present all of the messages produced by SAS Viya applications and services in a single consistent format. The SAS Viya log-monitoring application does. Some components of SAS Viya, especially several of the third-party components, do not produce log messages in the same format as the rest of the SAS Viya components. Although the SAS Viya log-monitoring components standardize these messages to use the same format as the rest of the SAS Viya messages, CloudWatch does not. This discrepancy might make working with log messages more difficult in CloudWatch than in the SAS Viya log-monitoring application.

Google Kubernetes Engine (GKE)

Deploying SAS Viya Monitoring for Kubernetes on GKE

In order to use SAS Viya Monitoring for Kubernetes in a GKE environment, you must deploy monitoring and logging using Ingress. See the Ingress sample for deployment information.

Alternative Monitoring Approach: Google Cloud's Operations Suite

As an alternative to using SAS Viya Monitoring for Kubernetes, you can use Google Cloud's operations suite, which is Google's set of monitoring applications.

In future releases, samples might be developed to provide procedures for integrating SAS Viya Monitoring for Kubernetes with the applications in Google Cloud's operations suite.

Choosing a Monitoring Strategy for an Azure Environment

If you deploy SAS Viya in an Azure environment, you can monitor and view log information by using either Azure Monitor or the components in the SAS Viya Monitoring for Kubernetes GitHub site. This topic provides an overview of the differences between the strategies in order to help you decide which one to use.

Consider Administrator Roles

In a SAS Viya environment, you must consider the monitoring needs of both the cluster administrators and the SAS Viya administrators:

- Cluster administrators need to see metric and log information from the cluster, but not from SAS resources.
- SAS administrators need to see metric and log information from SAS Viya resources and applications, but not from cluster resources.

Azure Monitor enables you to monitor metrics and log messages from the cluster and from SAS Viya resources such as nodes, pods, and services.

However, Azure Monitor does not provide the following support:

- It does not parse log messages into a common structure for consistent queries.
- It does not support monitoring SAS Viya applications.
- It does not include metrics from services (such as CAS or SAS launcher jobs) or internal metrics from SAS Viya services.

SAS Viya Monitoring for Kubernetes enables the following support:

- Log messages from different sources are parsed into a common structure for consistent queries.
- You can monitor metrics and log messages from both the cluster and from SAS Viya resources and applications.
- You can monitor metrics from services (such as CAS or SAS launcher jobs) and internal metrics from SAS Viya services.

Overview of Azure Monitor

Azure Monitor provides a built-in solution for cluster monitoring through views that enable you to easily check the availability of each level of a cluster or a multi-cluster environment. You can check metrics by drilling down through the cluster, node, pod, namespace, and process levels. It also enables you to view metrics and log messages in the same interface. See the [Azure Monitor documentation](#) for more information.

However, Azure Monitor does not provide support for monitoring SAS Viya resources. By default, Azure Monitor does not provide support for surfacing metrics that are collected from SAS Viya resources. Adding support for these metrics would require a great deal of code.

In addition, Azure Monitor does not provide a method for separating cluster metrics from SAS Viya metrics. In order to view metrics from SAS Viya, SAS administrators would also have to view metrics from all resources in the cluster. This quantity of information makes it harder for SAS administrators to find the information that they need, and it also gives them access to information that they might not have permission to view.

Log messages from SAS Viya are collected and displayed in Azure Monitor, but are not parsed into a common structure that simplifies queries. Like the metric data, SAS Viya log messages are not separated from log messages that come from cluster resources.

Azure Monitor provides cluster monitoring and log viewing that you can use with minimal setup, but it does not provide support for monitoring metrics from SAS Viya applications. Also, it does not parse log messages into a structure that makes it easy to find the information that SAS administrators need.

Overview of SAS Viya Monitoring for Kubernetes

Although the SAS Viya Monitoring for Kubernetes solution is not deployed and configured by default when you deploy SAS Viya platform, it provides the capability to monitor both the cluster and SAS Viya resources. See [“About SAS Viya Monitoring for Kubernetes” on page 3](#) for more information.

For monitoring Kubernetes clusters, Grafana dashboards enable you to view availability and performance information at the cluster, node, pod, and process levels.

For monitoring SAS Viya, dashboards enable you to view metrics for SAS Viya resources such as CAS, SAS services, and SAS launcher jobs.

You can deploy the monitoring solution so that SAS administrators have access to metric data and Grafana dashboards only for SAS Viya resources. This solution makes it easier for SAS administrators to find the information that they need and prevents them from accessing information that they do not need to view.

Log messages are collected, parsed, and normalized, so they can be displayed in OpenSearch Dashboards consistently. You can also use roles to enable SAS administrators to view log messages only from SAS Viya resources and only from a specified SAS Viya namespace.

Comparison Details

This table provides details about the comparison between Azure Monitor and SAS Viya Monitoring for Kubernetes.

Table 22 *Comparison of Azure Monitor and SAS Viya Monitoring for Kubernetes*

Features	Azure Monitor	SAS Viya Monitoring for Kubernetes
Main function	Supports logging and monitoring for a single cluster, and custom views can provide data from multiple clusters. Both logging and monitoring data is viewed through a single interface (Azure Portal).	Supports logging and monitoring for a single cluster and for the SAS Viya components in the cluster. Logging data is viewed through OpenSearch Dashboards and monitoring data is viewed through Grafana.
Cluster monitoring	Provides availability views for the cluster, node, pod, container, and process levels. Provides approximately 40 metrics for CPU, memory, disk, and network performance.	Provides approximately 1,500 metrics, many with multiple time series. Cluster availability dashboards are not yet provided.
SAS Viya monitoring	No monitoring is provided for SAS Viya resources, although the metric data can be scraped from the SAS Viya metric endpoints.	Monitoring is provided for SAS Viya resources, and dashboards are provided to display SAS metric data for CAS, SAS Java microservices, SAS Go microservices, SAS launcher jobs, RabbitMQ, and Postgres.
Data storage	Supports managed, off-cluster storage through the Azure Log Analytics workspace.	Uses in-cluster data volumes by default.
Data retention	Retains data for between 30 and 730 days, and supports a daily data cap with alert notification.	By default, retains monitoring data for seven days or 25 GB, whichever comes first, and retains logging data for three days. You can configure the monitoring and logging retention periods.

Features	Azure Monitor	SAS Viya Monitoring for Kubernetes
Monitoring dashboards	Dashboards are provided for clusters, nodes, controllers, deployments, and containers, and some workbooks (such as Node Disk Usage and Node Disk I/O) are also provided. Support is provided for creating your own dashboards using the Kusto Query Language (KQL). A small number of dashboards are provided by the Azure user community.	More than 40 Grafana dashboards are provided, including eight specifically for SAS Viya resources. Support is provided for creating your own dashboards through PromQL. Many dashboards, including custom dashboards, are provided by the Grafana user community.
Sharing and exporting monitoring data	Supports exporting chart data to Excel and sharing links to dashboards or charts.	Supports exporting data from Grafana to a CSV file, sharing chart images, and sharing links to dashboards and charts.
Logging	Collects all cluster logs, stores them in the Azure Log Analytics workspace, and searches them in Azure Monitor using KQL. KQL can parse structured SAS JSON logs and search by field.	Collects all cluster logs, parses them into a common schema using Fluent Bit, stores them in OpenSearch, and searches them using OpenSearch Dashboards. The field names of most logs are already known, so they can be parsed and searched easily.
User account security	Supports unified enterprise user account and subscription management through Microsoft.	By default, provides separate accounts for Grafana (monitoring) and OpenSearch Dashboards (logging). Can be configured to integrate with security back ends.
High availability	Supported by default through the Azure infrastructure.	Partially supported by default. You can use external components to provide full high availability.
Cluster resource usage	Uses approximately 350 MB of memory and less than 1 CPU core	Uses approximately 20 GB of memory and 3 CPU cores (although this can vary).
Alerting	Provides 11 predefined alerts as a preview, which can be integrated with SMS text messages, email, or a webhook. Alerts are supported for both metrics and logs.	Provides approximately 100 predefined alerts, which can be integrated with Slack, Microsoft Teams, SMS text messages, email, and other channels. Alerts are supported only for metrics.
Cross compatibility	Supports metric endpoints from Prometheus through Insight Metrics, but cannot read Prometheus data directly.	Supports Azure Monitor as a data source, and community-created Azure dashboards are available
Namespace data separation	Monitoring and logging data is not separated by namespace.	By default, monitoring data is not separated by namespace, but a sample is provided for a custom deployment that separates monitoring data by namespace.

Features	Azure Monitor	SAS Viya Monitoring for Kubernetes
		You can use custom permissions to separate logging data by namespace.
Cost	Provided at a base cost, plus additional costs for API calls, alert rules, and storage of logs and metric data.	Provided at no additional cost except for normal compute and storage costs for cluster usage.

